

Paper 2: Algorithms, programming and logic

Chapter 8: Programming

8.1 Programming concepts

8.2 Arrays

8.3 File handling

By: Mr. Nouredine Tadjerout

Learning Objectives:

8.1 Programming concepts

Candidates should be able to:

- 1 Declare and use variables and constants
- 2 Understand and use the basic data types

- 3 Understand and use input and output
- 4 (a) Understand and use the concept of sequence
(b) Understand and use the concept of selection

(c) Understand and use the concept of iteration

(d) Understand and use the concepts of totalling and counting

(e) Understand and use the concept of string handling

Notes and guidance

- Including:
 - integer
 - real
 - char
 - string
 - Boolean

- Including:
 - IF statements
 - CASE statements
- Including:
 - count-controlled loops
 - pre-condition loops
 - post-condition loops

- Including:
 - length
 - substring
 - upper
 - lower
- The first character of the string can be position zero or one

Learning Objectives:

8.1 Programming concepts continued

Candidates should be able to:

- (f) Understand and use arithmetic, logical and Boolean operators

5 Understand and use nested statements

6 (a) Understand what is meant by procedures, functions and parameters

(b) Define and use procedures and functions, with or without parameters

(c) Understand and use local and global variables

7 Understand and use library routines

Notes and guidance

- Arithmetic, limited to:

- +
- −
- /
- *
- ^ (raised to power of)
- MOD
- DIV

- Logical, limited to:

- =
- <
- <=
- >
- >=
- <> (not equal to)

- Boolean, limited to:

- AND
- OR
- NOT

- Including nested selection and iteration

- Candidates will **not** be required to write more than three levels of nested statements

- Procedures and functions may have up to two parameters

- Including:

- MOD
- DIV
- ROUND
- RANDOM

Learning Objectives:

8.1 Programming concepts continued

Candidates should be able to:

- 8 Understand how to create a maintainable program

Notes and guidance

- Including appropriate use of:
 - meaningful identifiers
 - the commenting feature provided by the programming language
 - procedures and functions
 - relevant and appropriate commenting of syntax
- Use meaningful identifiers for:
 - variables
 - constants
 - arrays
 - procedures and functions

Learning Objectives:

8.2 Arrays

Candidates should be able to:

- 1 Declare and use one-dimensional (1D) and two-dimensional (2D) arrays
- 2 Understand the use of arrays
- 3 Write values into and read values from an array using iteration

Notes and guidance

- Including the use of variables as indexes in arrays
- The first index can be zero or one
- Including nested iteration

8.3 File handling

Candidates should be able to:

- 1 Understand the purpose of storing data in a file to be used by a program
- 2 Open, close and use a file for reading and writing

Notes and guidance

- Including:
 - read and write single items of data
 - read and write a line of text

Resources:



<https://www.hoddereducation.co.uk/subjects/general/products/14-16/cambridge-igcse-and-o-level-computer-science-secon>

05/10/2023

By: Nouredine Tadjerout

Resources:



Craig 'n' Dave For Students

CRAIG 'N' DAVE

To explore the entire series,
visit <https://craigndave.org/free-videos>

8.1.1 Declare and use variables and constants

Click the link below to watch the video for the variables and constants

<https://www.youtube.com/watch?v=Jot4qdPjieA>

8.1 Programming

Variables and constants

CAMBRIDGE
IGCSE
(0478/0984)

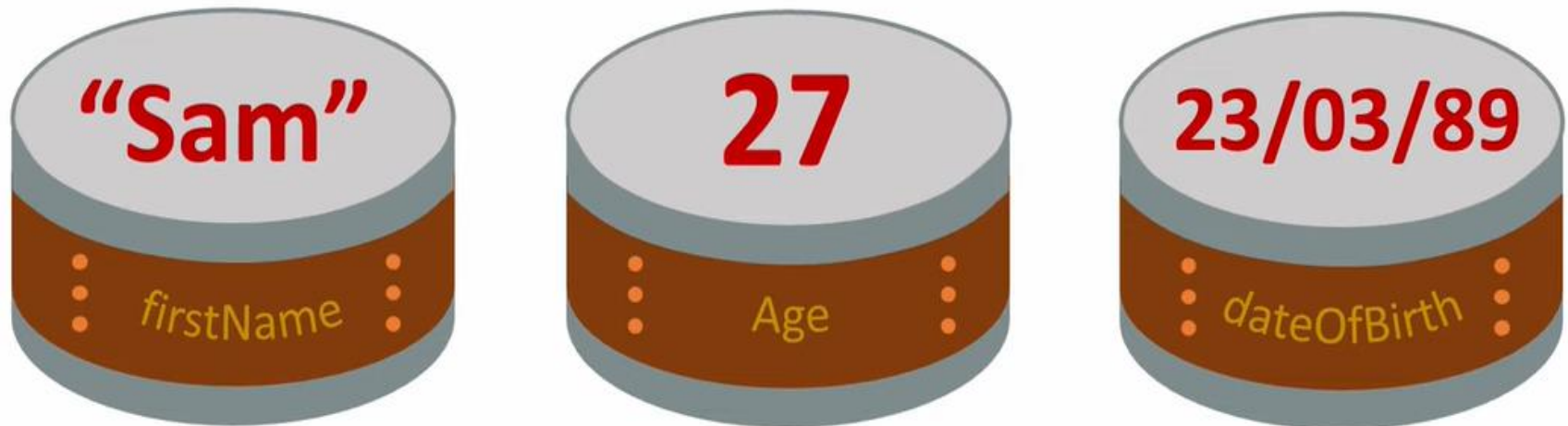


5:16
lave.c

8.1.1 Declare and use variables and constants

What is a variable?

Once you have **declared** a **variable**, you can then assign a value to it – this is like putting an item into the can.



The contents of the *firstName* **variable** holds the value "Sam".

The contents of the *Age* **variable** holds the value 27.

The contents of the *dateOfBirth* **variable** holds the value 23/03/89

8.1.1 Declare and use variables and constants

A Constants in a computer program is a named data store that contains a value that does not change during the execution of the program.

Constants differ from variables in that they cannot be changed once they are set.

Constants are still areas of memory that are used to store data, however they are used to store information that will never change e.g.



```
PI ← 3.141592654  
INPUT num  
product ← num * PI  
OUTPUT product
```

8.1.1 Declare and use variables and constants

USING VARIABLES

Declare side1
Declare side2
Declare Area

Output "Enter the length of side one:
side1 = INPUT()

Output "Enter the length of side two:
side2 = INPUT()

Area = side1 * side2

Output Area

The name of the variable is important as you will need it to use variable. The name can be anything, but you should keep it related to the data that it will store.

It is good practice to declare your variables at the start of the program. Why do you think that you should do this?



8.1.1 Declare and use variables and constants

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Variables

```
DECLARE <identifier> : <data type>
```

```
DECLARE Counter : INTEGER
```

```
DECLALRE TotalToPay: REAL
```

```
DECLARE GameOver : BOOLEAN
```

Constants

```
CONSTANT <identifier> ← <value>
```

```
CONSTANT HourlyRate ← 12.50
```

```
CONSTANT DefaultText ← "NA"
```

```
CONSTANT MaxScore ← 99999
```


8.1.1 Declare and use variables and constants



Variables and constants

Variable: A value stored in memory that can change while the program is running – can be an integer, character, string, real/float or Boolean value.

Constant: A value that is assigned when the program is first written and does not change while it is running.

- Constants make a program easier to read, as they are usually declared and assigned at the top of the program.
- They allow programmers to modify a program by changing one value rather than having to change every instance of a value throughout the program, reducing the possibility of errors.
- If constants are used instead of variables, a compiler can be used to optimise code, making the program run faster.

8.1.1 Declare and use variables and constants

 Declare and use variables.py - C:/Program Files/Python312/Declare and use variables.py (3.1... — □

File Edit Format Run Options Window Help

```
# Declare and use variables
name = "Noureddine"
age = 35
balance = 1000.50
```

```
# Declare constants
TAX_RATE = 0.1
MAX_SCORE = 100
```

```
# Example usage
print("Name:", name)
print("Age:", age)
print("Balance:", balance)
```

```
# Update variable
balance -= 200.75
print("Updated Balance:", balance)
```

```
# Use constants in a calculation
income = 1500
tax_amount = income * TAX_RATE
net_income = income - tax_amount
print("Net Income after Tax:", net_income)
```

```
# Constants are constants; attempting to change them will result in an error
# MAX_SCORE = 200 # Uncommenting this line will result in an error
```

```
===== RESTART: C:/Program Fi
Name: Noureddine
Age: 35
Balance: 1000.5
Updated Balance: 799.75
Net Income after Tax: 1350.0
|
```

8.1.2 Understand and use the basic data types: Including: integer, real, char, string . Boolean

Click the link below to watch the video for the use of basic data types

<https://www.youtube.com/watch?v=ugq7ig1qU8E>

8.1 Programming

The use of basic data types

CAMBRIDGE
IGCSE
(0478/0984)



5:27
ave.o

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

Data types

Variables and **constants** hold data. This data comes in a variety of forms, known as its **data type**.

You need to be aware of the following five **data types** for the exam.

Data type	Pseudocode	Description	Notes	Examples	
Integer	INTEGER	Whole number	Written as normal in denary.	7 -12	100 -3
Real	REAL	Number capable of containing a fraction part	Always has at least one digit on either side of the decimal point with zeroes added as necessary.	3.14 0.0 -3.99	-5.0 0.225 134.0098
Char	CHAR	Single character	Enclosed in single quotes.	'x' '7'	'C' '@'
String	STRING	Sequence of characters	Enclosed in double quotes. A string may contain no characters (i.e., an empty string).	"Craig" "This is a string!" "Your score is 9,312"	""
Boolean	BOOLEAN	Logical values		TRUE FALSE	

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Data types

INTEGER Whole number
REAL Number capable of containing a fractional part
CHAR Single character
STRING Sequence of characters
BOOLEAN Logical values

Declaring data types

```
DECLARE <identifier> : <data type>
```

```
DECLARE Counter : INTEGER  
DECLALRE TotalToPay: REAL  
DECLARE GameOver : BOOLEAN
```

8.1.2 Understand and use the basic data types: Including: integer, real, char, string . Boolean

Integer:

An Integer can be used to store a positive whole number (No decimals numbers). Integers will allow you to perform mathematical operators example : if you have the following code

```
num1 ← 5
num2 ← 10
sum ← num1 + num2
OUTPUT sum
```

```
File Edit Format Run Options Window Help
x = int(input("Enter an integer >>>"))
print ("You have entered", x)
```

```
...
RESTART: C:/Users/v90001010,
Enter an integer >>> 12
You have entered 12
>>> |
```

```
File Edit Format Run Options Window He
x = input("Enter an integer >>>")
y = int(x)
print("You have entered", y)
```

```
...
RESTART: C:/Users/v90001010,
Enter an integer >>> 12
You have entered 12
>>> |
```

```
File Edit Format Run Options Window Help
num = int(input("Enter an integer number: "))
print("num:", num)
```

```
...
RESTART: C:/Users/v90001010/App
Enter an integer number: 12
num: 12
>>> |
```


8.1.2 Understand and use the basic **data types**: Including: **integer, real, char, string . Boolean**

REAL NUMBER:

The Real data type will allow you to store decimal numbers (Positive or negative) example. Cost = 7.50

Real data can be used in mathematical operations

FirstReal = float(25) sets up a real variable with the value of 25.0 in Python

File Edit Format Run Options Window Help

```
FirstReal = float(input("Enter a real number: "))  
print("FirstReal:", FirstReal)
```

```
RESTART: C:/Users/v90001010/App  
Enter a real number: 25  
FirstReal: 25.0  
|
```

```
num1 ← 5.5  
num2 ← 10.4  
sum ← num1 + num2  
OUTPUT sum
```

CHAR:

The Char data type can be used to store **one single character**.

Example: Gender = "M" or Gender = "F"

```
letter ← 'n'  
gender ← "F"
```

BOOLEAN:

The Boolean data type can have only two possible values, these are TRUE or FALSE.

Example: Accepted = true or False

```
INPUT num1  
INPUT num2  
num1IsHigher ← num1 > num2  
IF num1IsHigher THEN  
    PRINT "Number 1 is higher than number 2"  
ELSE  
    PRINT "Number 2 is higher than or equal to number 2"  
ENDIF
```

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

STRING:

The String data type is used for data that can be used to store a number of letters, digits and symbols. The data can be a combination of these three items e.g. "123HAS@", or it can be just one type e.g. "Hello Mr Noureddine".

When assigning data to a string it should be placed within quotes e.g. name = "Noureddine"

```
name ← "Avi"  
email ← "avinator123@example.com"  
emptyString ← ""
```


8.1.2 Understand and use the basic **data types**: Including: **integer, real, char, string . Boolean**

 *datatype.py - C:/Program Files/Python312/datatype.py (3.12.0)*

— □

File Edit Format Run Options Window Help

```
# Integer
age = 35
print("Age:", age, "Type:", type(age))

# Real (Floating-point)
salary = 2500.75
print("Salary:", salary, "Type:", type(salary))

# Char (In Python, this is represented by a string with a single character)
gender = 'M'
print("Gender:", gender, "Type:", type(gender))

# String
name = "Nouredine"
print("Name:", name, "Type:", type(name))

# Boolean
is_teacher = True
print("Is Teacher?", is_teacher, "Type:", type(is_teacher))
```

```
= RESTART: C:/Program Files/Python312/datatype.py
Age: 35 Type: <class 'int'>
Salary: 2500.75 Type: <class 'float'>
Gender: M Type: <class 'str'>
Name: Nouredine Type: <class 'str'>
Is Teacher? True Type: <class 'bool'>
```

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean



Title

Integer

- Positive or negative whole number (e.g., 6)
- Use: $\text{total} = \text{total} + \text{score}$
- Never used for telephone numbers
- Used only when arithmetic needs to be performed on the data

Real

- Number with a decimal part (e.g., 6.5)
- Sometimes called float (floating point)
- Use: $\text{cost} = \text{total} + \text{vat}$

Char

- Single alphanumeric character (e.g., "1" or "a")
- Used for menu choices
- Use: $\text{choice} = \text{input}(\text{"Enter your choice:"})$

String

- Multiple alphanumeric characters (e.g., "Craig")
- Used for words and telephone numbers
- Use: $\text{forename} = \text{"Craig"}$

Boolean

- Variable that can be in one of two states
- True or False
- Use: $\text{playerAlive} = \text{True}$

Most languages support other data types such as dates and pictures.

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 1 :

- 4 Five data types and five data samples are shown below.

Draw a line to link each data type to the correct data sample.

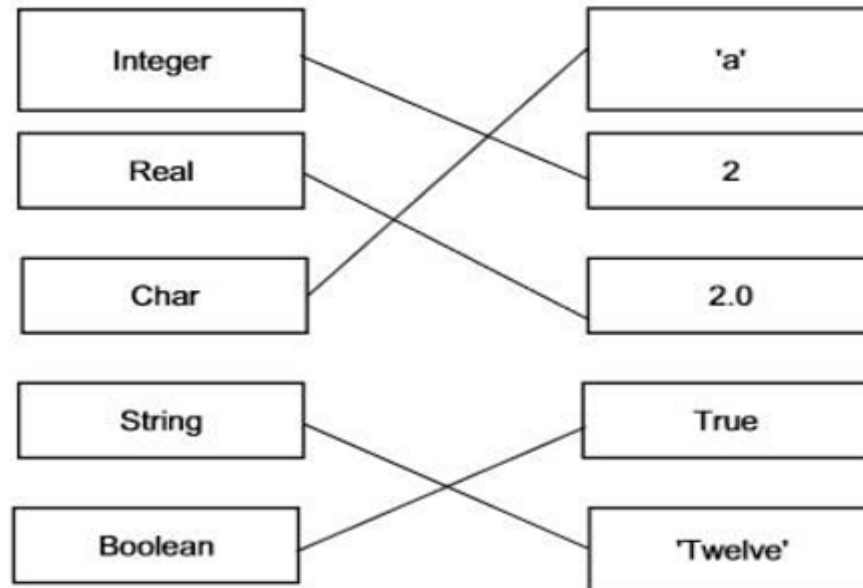
Data type	Data sample
Integer	'a'
Real	2
Char	2.0
String	True
Boolean	"Twelve"

[4]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 1 with Answer:

4 1 mark for each correct link, up to maximum of 4 marks



[4]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 2:

2 Describe, giving an example for each, the following data types used in programming.

Integer

Description

.....

Example

String

Description

.....

Example

[4]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 2 with Answer:

Question	Answer	Marks
Section B		
2	Integer – 1 mark for description 1 mark for example e.g. Any whole number for example a week number / 26 String – 1 mark for description 1 mark for example e.g. Any data item that contains letters and/or numbers and/or special characters ... for example someone's name / def7773@.	4

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 3 :

5 Explain the difference between a variable and a constant in a program.

.....

.....

.....

.....[2]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 3 with Answer:

- 5 Any **two** points from
- a variable is used to store data that can change during the running of a program
 - a constant is used to store data that will not be changed during the running of a program

[2]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 4 :

3 The following diagram shows **four** data structures and **four** descriptions.

Draw a line to connect each data structure to the correct description.

Data structure	Description
Constant	A collection of related data
Array	A value that can change whilst a program is running
Table	A value that never changes whilst a program is running
Variable	A series of elements of the same data type

[3]

8.1.2 Understand and use the basic **data types**: Including: integer, real, char, string . Boolean

IGCSE EXAM Question 4 with Answer :

Question	Answer	Marks
3	1 mark for each correct line, max 3 marks. Data Structure Constant Array Table Variable Description A collection of related data. A value that can change whilst a program is running. A value that never changes whilst a program is running. A series of elements of the same data type.	3

8.1.3 Understand and use **input** and **output**

[Click the link below to watch the video for ser input and display output](https://www.youtube.com/watch?v=atC6ZU0-vAk)

<https://www.youtube.com/watch?v=atC6ZU0-vAk>

8.1 Programming

User input and display output

CAMBRIDGE
IGCSE
(0478/0984)



3:47
ave.c

8.1.3 Understand and use **input** and **output**

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Input

```
INPUT <identifier>
```

```
INPUT Answer
```

Output

```
OUTPUT <value(s)>
```

```
OUTPUT Score
```

```
OUTPUT "You have ", Lives, " lives left"
```

8.1.3 Understand and use **input** and **output**



User input and display output

Input: A value read from an input device (e.g., keyboard).

Output: Data generated by the computer and displayed to the user.

8.1.3 Understand and use **input** and **output**

 input and output.py - C:/Program Files/Python312/input and output.py (3.12.0)

File Edit Format Run Options Window Help

```
# Input: Get user's name and age
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Process: Calculate the birth year
current_year = 2023
birth_year = current_year - age

# Output: Display a personalized message
print("\nHello, " + name + "!")
print("You are", age, "years old.")
print("Your birth year is approximately", birth_year)

# Output: Display a farewell message
print("\nThank you for using this program. Goodbye!")
```

Edit Shell Debug Options Window Help

```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2
AMD64)] on win32
Type "help", "copyright", "credits" or "lice

= RESTART: C:/Program Files/Python312/input
Enter your name: noureddine
Enter your age: 35

Hello, noureddine!
You are 35 years old.
Your birth year is approximately 1988

Thank you for using this program. Goodbye!
```

8.1.4 The use of the three basic programming **constructs**

[Click the link below to watch the video](#)

<https://www.youtube.com/watch?v=QgffZbGnBvg>

8.1 Programming

The use of the three basic
programming constructs

CAMBRIDGE
IGCSE
(0478/0984)



10:09
ave.or

8.1.4 The use of the three basic programming constructs



Sequence

Sequence means executing instructions in order, one after the other.

1 
2 
3 

#Import libraries

```
import random
```

#Constants

```
rolls_per_player = 2
```

#Initialise variables

```
total = 0
```

#Output the dice

```
print("dice 1:",dice1)
```

```
print("dice 2:",dice2)
```

#Cast the dice values from integers to strings ready for changing the order of the dice

```
dice1_string = str(dice1)
```

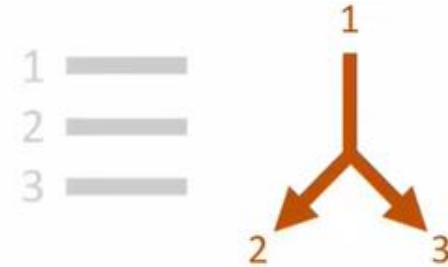
```
dice2_string = str(dice2)
```

8.1.4 The use of the three basic programming constructs



Selection (branching)

Selection means a program will branch depending on certain conditions.



#Change the order of the dice to the highest value first and join the values together

```
if dice1 > dice2:  
    roll_value = dice1_string + dice2_string  
else:  
    roll_value = dice2_string + dice1_string
```

#Output the value of the dice

```
if user input == roll_value:  
    print("You worked it out correctly.")  
else:  
    print("No, the value of the roll is:", roll_value)
```

8.1.4 The use of the three basic programming constructs



Selection (branching)

Selection means a program will branch depending on certain conditions.



#Ask the user for their membership type and set their club price

```
MemberType = input("What type of membership do you have?")
```

Match MemberType:

```
case 'Child':
```

```
    ClubPrice = 5.0
```

```
case 'Adult':
```

```
    ClubPrice = 10.0
```

```
case 'Senior':
```

```
    ClubPrice = 6.50
```

```
case _:
```

```
    print "Invalid membership type"
```

Match (CASE) statements allow one of several branches of code to be executed, depending on the value of the variable.



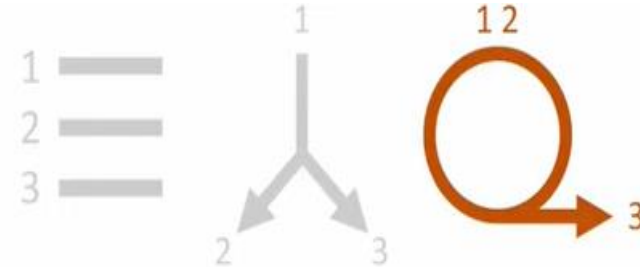
We are using Python's match statement in the above example. Other languages use similar concepts, but they are all implemented differently – the exam board refers to these as CASE statements. Make sure to check out the pseudocode example shown later in this video.

8.1.4 The use of the three basic programming constructs



Iteration (looping)

Iteration, sometimes called looping, means repeating sections of code.



#Roll the dice

```
for rolls in range(rolls_per_player):
```

```
    dice1 = 0
```

```
    dice2 = 0
```

```
    #Handle the possibility of a double throw
```

```
    while dice1 == dice2:
```

```
        dice1 = random.randint(1,6)
```

```
        dice2 = random.randint(1,6)
```

```
    #Output the dice
```

```
    print("dice 1:",dice1)
```

```
    print("dice 2:",dice2)
```

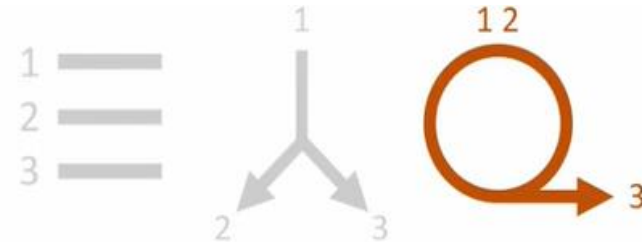
A **FOR** loop (also known as a **count-controlled loop**) is used when the required number of iterations is known ahead of execution.

8.1.4 The use of the three basic programming constructs



Iteration (looping)

Iteration, sometimes called looping, means repeating sections of code.



A **REPEAT loop** (also known as a **post-condition loop**) is used when the required number of iterations is not known ahead of execution because the variable used to determine when the iteration ends is changing within the iteration itself.

A **REPEAT loop** will always execute **at least once** as the check is performed at the end of the loop.

```
REPEAT
  OUTPUT "Please enter the password"
  INPUT Password
  UNTIL Password = "Secret"
```

Although the language you are learning may not utilise these commands, you still need to be aware of them for the exam.

A **WHILE loop** (also known as a **pre-condition loop**) is an alternative to REPEAT where the condition is checked **before** entering the loop, this means the loop might execute **zero** times if the condition is FALSE at the start of the first iteration.

```
WHILE Number > 9 DO
  Number ← Number - 1
ENDWHILE
```

8.1.4 The use of the three basic programming constructs

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

IF statements

```
IF <condition>  
  THEN  
    <statements>  
ENDIF
```

```
IF password = "secret"  
  THEN  
    OUTPUT "Password correct"  
ENDIF
```

IF...ELSE statements

```
IF <condition>  
  THEN  
    <statements>  
  ELSE  
    <statements>  
ENDIF
```

```
IF Age >= 18  
  THEN  
    OUTPUT "You are old enough to vote"  
  ELSE  
    OUTPUT "You are not of voting age"  
ENDIF
```

8.1.4 The use of the three basic programming constructs

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

CASE statements

```
CASE OF <identifier>
  <value 1> : <statement>
  <value 2> : <statement>
  ...
ENDCASE
```

An OTHERWISE clause can be the last case:

```
CASE OF <identifier>
  <value 1> : <statement>
  <value 2> : <statement>
  ...
  OTHERWISE <statement>
ENDCASE
```

CASE statements

INPUT Move

```
CASE OF Move
  'W' : Position ← Position - 10
  'E' : Position ← Position + 10
  'A' : Position ← Position - 1
  'D' : Position ← Position + 1
  OTHERWISE OUTPUT "Error"
ENDCASE
```

8.1.4 The use of the three basic programming constructs

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Count-controlled FOR loop

```
FOR <identifier> ← <value1> TO <value2>  
    <statements>  
NEXT <identifier>
```

```
FOR Counter ← 1 TO 10  
    OUTPUT Counter  
NEXT Counter
```

Output window

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```


8.1.4 The use of the three basic programming constructs

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Count-controlled FOR loop with a step increment

```
FOR <identifier> ← <value1> TO <value2> STEP <increment>  
    <statements>  
NEXT <identifier>
```

```
FOR Counter ← 1 TO 10 STEP 2  
    OUTPUT Counter  
NEXT Counter
```

```
FOR Counter ← 10 TO 1 STEP -2  
    OUTPUT Counter  
NEXT Counter
```

Output window

```
1  
3  
5  
7  
9
```

Output window

```
10  
8  
6  
4  
2
```

8.1.4 The use of the three basic programming constructs

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Post-condition REPEAT loop

```
REPEAT
    <statements>
UNTIL <condition>
```

```
REPEAT
    OUTPUT "Please enter the password"
    INPUT Password
UNTIL Password = "Secret"
```

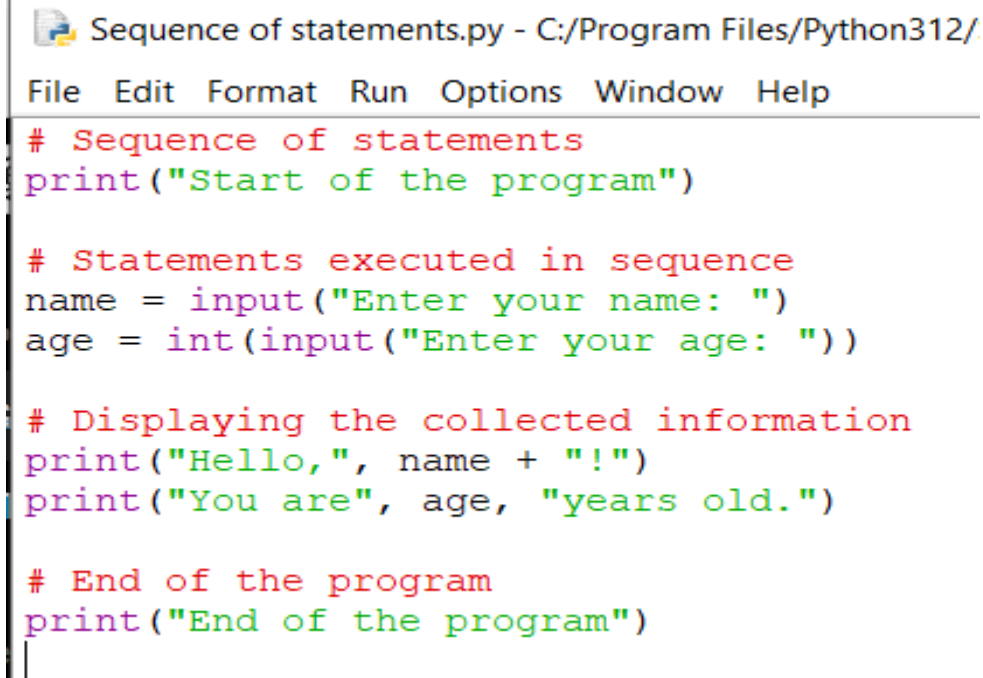
Pre-condition WHILE loop

```
WHILE <condition> DO
    <statements>
ENDWHILE
```

```
WHILE Number > 9 DO
    Number ← Number - 1
ENDWHILE
```

8.1.4 (a) Understand and use the concept of sequence

In programming, the concept of a sequence refers to an ordered set of instructions or statements that are **executed in a specific order**. It represents a flow of control where each statement is **executed one after the other**. The sequence is a fundamental programming concept and forms the basis of most programs.



```
Sequence of statements.py - C:/Program Files/Python312/
File Edit Format Run Options Window Help
# Sequence of statements
print("Start of the program")

# Statements executed in sequence
name = input("Enter your name: ")
age = int(input("Enter your age: "))

# Displaying the collected information
print("Hello,", name + "!")
print("You are", age, "years old.")

# End of the program
print("End of the program")
```

```
===== RESTART: C:/Program F
Start of the program
Enter your name: Nouredine
Enter your age: 35
Hello, Nouredine!
You are 35 years old.
End of the program
```

8.1.4 (b) Understand and use the concept of **selection**: Including: **IF** statements and **CASE** statements

The concept of selection involves **making decisions** in a program based on **certain conditions**. **Two** common **constructs** for implementing selection are **IF statements** and **CASE statements** (also known as **switch statements** in some programming languages).

Example using IF statements:

 IFStatement.py - C:/Program Files/Python312/IFStatement.py (3.12.0)

File Edit Format Run Options Window Help

```
# Get the user's age
age = int(input("Enter your age: "))

# Check if the person is eligible to vote
if age >= 18:
    print("You are eligible to vote.")
else:
    print("Sorry, you are not eligible to vote yet.")
```

```
===== RESTART: C:/Program Files/Python312/Python.exe
Enter your age: 35
You are eligible to vote.
```

```
===== RESTART: C:/Program Files/Python312/Python.exe
Enter your age: 13
Sorry, you are not eligible to vote yet.
```

Pseudocode iF Statements:

```
// Get the user's age
OUTPUT "Enter your age: "
INPUT age
// Check if the person is eligible to vote
IF age >= 18 THEN
    OUTPUT "You are eligible to vote."
ELSE
    OUTPUT "Sorry, you are not eligible to vote yet."
ENDIF
```

8.1.4 (b) Understand and use the concept of **selection**: Including: **IF** statements and **CASE** statements

Example using CASE statements: Python can emulate using **IF-ELIF-ELSE** structures.

CASEStatement.py - C:/Program Files/Python312/CASEStatement.py (3.12.0)

File Edit Format Run Options Window Help

```
# Get a numerical grade from the user
grade = int(input("Enter your numerical grade (0-100): "))
```

```
# Convert numerical grade to letter grade
```

```
if 90 <= grade <= 100:
```

```
    letter_grade = 'A'
```

```
elif 80 <= grade < 90:
```

```
    letter_grade = 'B'
```

```
elif 70 <= grade < 80:
```

```
    letter_grade = 'C'
```

```
elif 60 <= grade < 70:
```

```
    letter_grade = 'D'
```

```
else:
```

```
    letter_grade = 'F'
```

```
# Display the letter grade
```

```
print("Your letter grade is:", letter_grade)
```

```
= RESTART: C:/Program Files/Python312/CASE
```

```
Enter your numerical grade (0-100): 90
```

```
Your letter grade is: A
```

```
===== RESTART: C:/Program Files/Py
```

```
Enter your numerical grade (0-100): 40
```

```
Your letter grade is: F
```

Pseudocode

// Get a numerical grade from the user

OUTPUT "Enter your numerical grade (0-100): "

INPUT grade

// Convert numerical grade to letter grade using IF statements

IF grade >= 90 AND grade <= 100 THEN

letterGrade = 'A'

ELSE IF grade >= 80 AND grade < 90 THEN

letterGrade = 'B'

ELSE IF grade >= 70 AND grade < 80 THEN

letterGrade = 'C'

ELSE IF grade >= 60 AND grade < 70 THEN

letterGrade = 'D'

ELSE

letterGrade = 'F'

ENDIF

// Display the letter grade

OUTPUT "Your letter grade is: " + letterGrade

8.1.4 (b) Understand and use the concept of **selection**: Including: **IF** statements and **CASE** statements

Example using **CASE** statements:

caseStatement 1.py - C:/Program Files/Python312/caseStatement

File Edit Format Run Options Window Help

```
def get_grade_A():
    return 'A'

def get_grade_B():
    return 'B'

def get_grade_C():
    return 'C'

def get_grade_D():
    return 'D'

def get_grade_F():
    return 'F'

def default():
    return 'Invalid grade'

# Get a numerical grade from the user
grade = int(input("Enter your numerical grade (0-100): "))

# Use the match statement to find the appropriate grade function
match grade:
    case 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100:
        letter_grade = get_grade_A()
    case 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89:
        letter_grade = get_grade_B()
    case 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79:
        letter_grade = get_grade_C()
    case 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69:
        letter_grade = get_grade_D()
    case 59:
        letter_grade = get_grade_F()
    case _:
        letter_grade = default()

# Display the letter grade
print("Your letter grade is:", letter_grade)
```

```
===== RESTART: C:/Program Files/Py
Enter your numerical grade (0-100): 77
Your letter grade is: C
```

```
===== RESTART: C:/Program Files/Py
Enter your numerical grade (0-100): 60
Your letter grade is: D
```

```
===== RESTART: C:/Program Files/Py
Enter your numerical grade (0-100): 59
Your letter grade is: F
```

```
===== RESTART: C:/Program Files/Py
Enter your numerical grade (0-100): 10
Your letter grade is: Invalid grade
```

8.1.4 (b) Understand and use the concept of **selection**: Including: **IF** statements and **CASE** statements

Sequence within other structures:

Look at the code below, which lines here do you think would be classed as sequence?

The lines that would be classed as **sequence** are:

Lines 1-2

Lines 4 - 10

Line 12

Lines 3, 11 and 13 are classed as **selection**

```
1. OUTPUT "Please enter your age"
2. age = INPUT
3. IF age >=17 THEN
4.     OUTPUT "Welcome to the driving license application"
5.     OUTPUT "Please enter your name"
6.     name = INPUT
7.     OUTPUT "Please enter your address"
8.     address = INPUT
9.     OUTPUT "Please enter your phone number"
10.    telNo = INPUT
11. ELSE
12.    OUTPUT "Sorry, you are too young to drive"
13. END IF
```


8.1.4 (c) Understand and use the concept of **iteration**: Including: count-controlled loops, pre-condition loops, post-condition loops

1. Count-controlled loop (For loop):

Pseudocode:

// Count-controlled loop (For loop)

FOR i FROM 1 TO 5

OUTPUT i

END FOR

 Count-controlled loop.py - C:/Program Files/Python:

File Edit Format Run Options Window Help

```
# Count-controlled loop (For loop)
```

```
for i in range(1, 6):
```

```
    print(i)
```

= RESTART: C

1
2
3
4
5

8.1.4 (c) Understand and use the concept of **iteration**: Including: count-controlled loops, pre-condition loops, post-condition loops

2. Pre-condition loop (While loop):

Pseudocode:

// Pre-condition loop (While loop)

SET i = 1

WHILE i <= 5 **DO**

OUTPUT i

SET i = i + 1

END WHILE

 Pre-condition loop (While loop).py - C:/Program F

File Edit Format Run Options Window Help

Pre-condition loop (While loop)

i = 1

while i <= 5:

 print(i)

 i = i + 1

= RESTART: C:/Pr

1

2

3

4

5

8.1.4 (c) Understand and use the concept of **iteration**: Including: **count-controlled loops**, **pre-condition loops**, **post-condition loops**

3. Post-condition loop (REPEAT loop):

Pseudocode:

Example 1:

// Post-condition loop (REPEAT loop)

SET i = 1

REPEAT

OUTPUT i

SET i = i + 1

UNTIL i > 5

 Post-condition loop (REPEAT loop).py - C:/Progra

File Edit Format Run Options Window Help

```
# Post-condition loop (REPEAT loop)
```

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i = i + 1
```

```
= RESTART: C:/Progr
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
.
```

8.1.4 (c) Understand and use the concept of **iteration**: Including: count-controlled loops, pre-condition loops, post-condition loops

3. Post-condition loop (REPEAT loop):

Pseudocode:

Example 2:

// Post-condition loop (REPEAT loop)

SET total = 0

SET number = 0

REPEAT

// Input a number

INPUT "Enter a number: ", number

// Add the number to the total

SET total = total + number

UNTIL number = 0

// Output the total

OUTPUT "The sum of the numbers entered is: ", total

```
Post-condition loop 2 (REPEAT loop).py - C:/Program Files/Python312/Post-
File Edit Format Run Options Window Help
# Post-condition loop (REPEAT loop)
total = 0
number = 1 # Initialize number to a non-zero value

while number != 0:
    # Input a number
    number = int(input("Enter a number: "))
    # Add the number to the total
    total += number

# Output the total
print("The sum of the numbers entered is:", total)
```

```
== RESTART: C:/Program Files/Python312/
Enter a number: 5
Enter a number: 3
Enter a number: 1
Enter a number: 0
The sum of the numbers entered is: 9
```

8.1.4 (c) Understand and use the concept of **iteration**: Including: count-controlled loops, pre-condition loops, post-condition loops

Practical Activity:

Example 1:

File Edit Format Run Options Window Help

```
FirstNumber = int (0)
SecondNumber = int(0)
Sum = int(0)
FirstNumber = int( input( "Enter First Whole Number: "))
SecondNumber = int( input( "Enter Second Whole Number: "))
Sum = FirstNumber + SecondNumber
print ( " The sum equal" , Sum)
|
```

```
...
RESTART: C:/Users/v90001010/AppData/I
Enter First Whole Number: 20
Enter Second Whole Number: 20
The sum equal 40
>>> |
```

Example 2:

```
FirstNumber = int (0)
SecondNumber = int (0)
Sum = int (0)
FirstNumber = int(input( "Enter First Whole Number: "))
SecondNumber = int (input( "Enter Second Whole Number: "))
if FirstNumber > SecondNumber:
    print ("First Whole Number is largest ", FirstNumber)
else:
    print ("Second Whole Number is largest ", SecondNumber)
```

```
>>>
Enter First Whole Number: 7
Enter Second Whole Number: 3
First Whole Number is largest 7
>>>
```

8.1.4 (c) Understand and use the concept of **iteration**: Including: count-controlled loops, pre-condition loops, post-condition loops

Practical Activity:

Example 3:

```
Number = int (0)
Count = int (0)
Sum = int (0)
while Count < 5:
    Number = int(input( "Enter a Whole Number: "))
    Sum = Sum + Number
    Count = Count + 1
print ("Sum of five numbers is ", Sum)
```

```
>>>
Enter a Whole Number: 6
Enter a Whole Number: 7
Enter a Whole Number: 8
Enter a Whole Number: 9
Enter a Whole Number: 10
Sum of five numbers is 40
>>> |
```

8.1.4 (c) Understand and use the concept of **iteration**:

The term iteration means to repeat something over and over like a loop.

Iteration means to repeat. The ability to have a program repeat steps without having to duplicate actual lines of code is vital for efficiency.

File Edit Format Run Options Window

```
for Counter in range (0, 20):  
    print ('*', end = '')
```

Code With No Loop

```
1. Output 1  
2. Output 2  
3. Output 3  
4. Output 4  
5. Output 5  
6. Output 6  
7. Output 7  
8. Output 8  
9. Output 9  
10. Output 10  
11. Output 11  
12. Output 12
```

/Python code/Iteration

>>> |

Code With Loop

```
1. For x = 1 to 12  
2.   Output x  
3.   x = x + 1  
4. End For
```

Code With Loop (1 to 10 Million)

```
1. For x = 1 to 10000000  
2.   Output x  
3.   x = x + 1  
4. End For
```


8.1.4 (c) Understand and use the concept of **iteration**:

TYPES OF ITERATION:

- For Loop
- While loop
- Repeat Until Loop

```
FOR count ← 1 TO 10  
  OUTPUT "Count: ", count  
NEXT count
```

```
count ← 1  
WHILE count <= 10 DO  
  OUTPUT "Count: ", count  
  count ← count + 1  
ENDWHILE
```

```
count ← 1  
REPEAT  
  OUTPUT "Count: ", count  
  count ← count + 1  
UNTIL count = 11
```

Each loop works in a very different way, has different abilities and is best suited to certain situations.

FOR LOOPS : A For loop is used when code needs to be **repeated a specified number of times**. The number of times that it will need to be repeated could be determined by the programmer, the **program user** or a **calculation**.

WHILE LOOPS: While loops are also known as **pre-condition loops**. This is because a while loop will **repeat forever WHILE** a set condition is **true**. This condition is checked at the start of the loop hence the name "**pre-condition**".

REPEAT UNTIL LOOPS: A REPEAT UNTIL loop is also known as a "**post-condition loop**". This type of loop will **repeat forever UNTIL** a condition becomes **true**.

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

Sample question

3 Identify the errors in the following program code to print the average of 10 numbers input, and then rewrite the program correctly.

```
10 Total = 1
20 Count = 0
30 WHILE Count <= 10
40   INPUT Number
50   Total = Total - number
60   Count = Count + 1
70   PRINT Total/Count
80 ENDWHILE
```

Student answer

```
10 Total = 0
20 Count = 0
30 WHILE Count < 10
40   INPUT Number
50   Total = Total + number
60   Count = Count + 1
70 ENDWHILE
80 PRINT Total/10
```

Examiner's comments

The student has rewritten the program correctly but hasn't explicitly identified the errors. This would need to be done for full marks. For example, in line 10 the total is incorrectly initialised, in line 30 the test should be for less than 10 to stop the loop repeating 11 times, in line 50 the total is decreased rather than increased and in line 70 an attempt at an average will be printed every time the loop is repeated.

Examiner's tips

Always make sure all tasks are completed.

The programming statements on an examination paper will not be from any particular programming language.

The programming statements usually have line numbers to make it easier to identify errors.

Explain the difference between the programming concepts of **sequence** and **selection**. Include an example of a programming statement for each concept in your explanation.

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

IGCSE EXAM Question 1 with Answer:

Question	Answer	Marks
5	2 marks for appropriate explanation, 1 mark example programming statements showing sequence, 1 mark example programming statement(s) showing selection e.g. Sequence is the concept of one statement being executed after another(1) whereas selection decides which statement(s) are to be executed depending upon the result of a question (1) sequence example (1) <pre>PRINT X PRINT Y</pre> Selection example (1) <pre>IF X > Y THEN PRINT X ELSE PRINT Y</pre>	4

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 2 :

4 A programmer wants to test that the readings from 2000 electricity meters are greater than 400 units and less than 900 units. The programmer uses selection and repetition statements as part of the program.
Explain, using programming statements, how selection and repetition could be used in this program.

Selection

.....

.....

.....

.....

Repetition

.....

.....

.....

.....

.....

.....

.....

[4]

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 2 with Answer:

Question	Answer	Marks
4	Answers must be given in context. There are many possible answers. E.g.: Selection use of IF statement to check the values of the meter readings (1 mark) IF Reading > 400 and Reading < 900 THEN ... (1 mark) Repetition use of FOR loop to check all 2000 meter readings (1 mark) FOR Meter = 1 TO 2000 ... NEXT (1 mark)	4

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 3 :

- 4 (a) Four pseudocode descriptions and five pseudocode statements are shown. Draw one line to link each pseudocode description to the correct pseudocode statement. Not all pseudocode statements will be used.

Pseudocode description

A loop that will iterate at least once.

A conditional statement to deal with many possible outcomes.

A loop that will iterate a set number of times.

A conditional statement with different outcomes for true and false.

Pseudocode statement

FOR...TO...NEXT

IF...THEN...ELSE...ENDIF

WHILE...DO...ENDWHILE

CASE...OF...OTHERWISE...ENDCASE

REPEAT...UNTIL

[4]

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 3 with Answer:

Question	Answer	Marks
4(a)	1 mark for each correct line Pseudocode description A loop that will iterate at least once. A conditional statement to deal with many possible outcomes. A loop that will iterate a set number of times. A conditional statement with different outcomes for true and false. Pseudocode statement FOR...TO...NEXT IF...THEN...ELSE...ENDIF WHILE...DO...ENDWHILE CASE...OF...OTHERWISE...ENDCASE REPEAT...UNTIL	4

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 4:

4 IF ... THEN ... ELSE ... ENDIF is one type of conditional statement used when writing pseudocode.

Identify and describe **another** type of conditional statement that you could use when writing pseudocode. Give a reason why you would use this type of conditional statement.

Conditional statement

.....

.....

.....

Description

.....

.....

.....

Reason

.....

[4]

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

IGCSE EXAM Question 4 with Answer:

Question	Answer	Marks
4	2 marks for identification, 1 mark for description, 1 mark for reason. Identification: <pre>CASE OF ... OTHERWISE ... (ENDCASE) OF ... OF ... (OTHERWISE) ... ENDCASE</pre> Description: – a statement that allows for multiple selections // not any of the above Reason: – to simplify pseudocode/ make pseudocode more understandable etc.	4

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 5:

REPEAT ... UNTIL is one type of loop structure.

Identify and describe **two** other types of loop structure that you could use when writing pseudocode.

Loop structure 1.....

Description.....

.....

Loop structure 2.....

Description.....

.....[4]

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

IGCSE EXAM Question 5 with Answer:

This question tells you that REPEAT ... UNTIL is one loop structure and then asks you to identify and describe two other loop structures.

Answers:

FOR (...TO...NEXT)

- A set number of iterations

WHILE (...DO...END WHILE)

- Used where the loop may never be executed / Whilst a specific condition is true

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 6:

REPEAT ... UNTIL and WHILE ... DO ... ENDWHILE are two different loop structures you can use when writing pseudocode.

Explain, using examples, why you would choose to use each type of loop.

Example 1
.....
.....
.....

Reason for choice
.....
.....

Example 2
.....
.....
.....

Reason for choice
.....
.....

[6]

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

IGCSE EXAM Question 6 with Answer:

This question gives you the names of two different loop structures but this time it asks you to give two example uses of the loops and the reason why you would choose them

For each example **1 mark** for **correct structure**, **1 mark** for **appropriate content** inside loop and **1 mark** for **reason**. There are many correct answers these are only samples

```
REPEAT
  ..INPUT Number
  Total ← Total + Number
UNTIL Number = 0
  – at least one repeat is required
```

```
WHILE Number <> -1 DO
  ..INPUT Number
  Total ← Total + Number
ENDWHILE
  – the loop may never be executed
```

[6]

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 7 :

(a) Describe the purpose of each statement in this algorithm.

```
FOR I ← 1 TO 300
  INPUT Name[I]
NEXT I
```

.....

.....

.....

.....

.....[2]

(b) Identify, using pseudocode, another loop structure that the algorithm in **part (a)** could have used.

.....

.....[1]

(c) Write an algorithm, using pseudocode, to input a number between 0 and 100 inclusive. The algorithm should prompt for the input and output an error message if the number is outside this range.

.....

8.1.4 (a.b, c) Understand and use the concept of **sequence**, **selection** and **iteration**:

IGCSE EXAM Question 7 with Answer:

Question	Answer	Marks
5(a)	Any two from: - Loop with 300 repetitions (starting at 1) / Loops from 1 to 300 - Values input/stored (in consecutive/different locations) in an array (at position I) - Increases the loop counter/I value by 1 (and returns to the start of the loop)	2
5(b)	Any one from: REPEAT (... UNTIL) WHILE (... DO ... ENDWHILE)	1
5(c)	- Prompt and input number (1) - Checking the input number is between 0 and 100 - both limits (1) - Correct error message (1) Many correct algorithms. This is an example only. <pre>OUTPUT "Enter a number between 0 and 100 " INPUT Number IF Number < 0 OR Number > 100 THEN OUTPUT "The number you have entered is outside the specified range" ENENDIF</pre>	3

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 8 :

An algorithm has been written in pseudocode to input 100 numbers and print out the sum.
A REPEAT ... UNTIL loop has been used.

```
Count ← 0
Sum ← 0
REPEAT
    INPUT Number
    Sum ← Sum + Number
    Count ← Count + 1
UNTIL Count > 100
PRINT Sum
```

(a) Find the error in the pseudocode and suggest a correction.

Error.....

Correction

[2]

(b) Rewrite the correct algorithm using a more suitable loop structure.

.....

.....

.....

.....

.....

8.1.4 (a.b, c) Understand and use the concept of sequence, selection and iteration:

IGCSE EXAM Question 8 with Answer:

Question	Answer	Marks
4(a)	Error - Count $\leftarrow$ 0 Correction - Count $\leftarrow$ 1 or Error - UNTIL Count > 100 Correction - UNTIL Count >= 100 or UNTIL Count = 100 or UNTIL Count > 99	2
4(b)	- use of FOR with correct start and end values ... - ... use of NEXT - ... removal of increment for Count Sample algorithm Sum $\leftarrow$ 0 FOR Count $\leftarrow$ 1 TO 100 INPUT Number Sum $\leftarrow$ Sum + Number NEXT // NEXT Count PRINT Sum	3

8.1.4 (d) Understand and use the concepts of totalling and counting

[Click the link below to watch the video](#)

<https://www.youtube.com/watch?v=f8srsTZBWOM>

8.1 Programming

Totalling and counting

CAMBRIDGE
IGCSE
(0478/0984)



2:03
ave.c

8.1.4 (d) Understand and use the concepts of totalling and counting

Totalling <pre>TotalMark = TotalMark + CurrentQuestionMark</pre>	Python <pre>TotalMark = TotalMark + CurrentQuestionMark</pre> Visual Basic <pre>TotalMark = TotalMark + CurrentQuestionMark</pre>
 <pre>TotalMark = TotalMark + CurrentQuestionMark;</pre>	C# <pre>TotalMark = TotalMark + CurrentQuestionMark;</pre> Java <pre>TotalMark = TotalMark + CurrentQuestionMark;</pre>

8.1.4 (d) Understand and use the concepts of **totalling** and **counting**

Counting

Python

```
NumberOfStudents = NumberOfStudents +1
```

```
NumberOfStudents +=1
```

Visual Basic

```
NumberOfStudents = NumberOfStudents +1
```

```
NumberOfStudents +=1
```

C#

```
NumberOfStudents = NumberOfStudents +1;
```

```
NumberOfStudents += 1;
```

```
NumberOfStudents++;
```

Java

```
NumberOfStudents = NumberOfStudents +1;
```

```
NumberOfStudents ++;
```


8.1.4 (d) Understand and use the concepts of **totalling** and **counting**

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Totalling

```
Total ← 0
FOR Counter ← 1 TO MaxNoOfQuestionsInTest
    Total ← Total + StudentsTest[Counter]
NEXT counter
```

Counting

```
PassCount ← 0
FOR Counter ← 1 TO ClassSize
    INPUT StudentMark
    IF StudentMark > 50
        THEN
            PassCount ← PassCount + 1
        ENDIF
NEXT Counter
```

8.1.4 (d) Understand and use the concepts of **totalling** and **counting**

1. Totalling:

Totalling refers to the process of summing up or accumulating values. It often involves using a loop to iterate through a set of values and adding them together

 Totalling.py - C:/Program Files/Python312/Totalling.py (3.12)

File Edit Format Run Options Window Help

```
# Example: Totalling using a for loop
```

```
numbers = [1, 2, 3, 4, 5]
```

```
total = 0
```

```
for num in numbers:  
    total += num
```

```
print("The total is:", total)
```

```
===== RESTART  
The total is: 15
```

Totalling Pseudocode

```
total <- 0 # Initialize the total variable
```

```
// Loop through the list of numbers
```

```
for each number in list:
```

```
    total <- total + number # Add the number to the total
```

```
// Display the total
```

```
print total
```

8.1.4 (d) Understand and use the concepts of **totalling** and **counting**

2.Counting:

Counting involves keeping track of the number of occurrences of a particular event or element. It can be implemented using a loop or built-in functions.

 Counting.py - C:/Program Files/Python312/Counting.py (3.12.0)

File Edit Format Run Options Window Help

```
# Example: Counting the number of even numbers in a list
```

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
count_even = 0
```

```
for num in numbers:
```

```
    if num % 2 == 0:
```

```
        count_even += 1
```

```
print("The count of even numbers is:", count_even)
```

```
= RESTART: C:/Program Files/Python:
```

```
The count of even numbers is: 5
```

```
|
```

// Counting Pseudocode

count <- 0 # Initialize the count variable

// Loop through the collection

for each element in collection:

 if element satisfies condition:

 count <- count + 1 # Increment count

//Display the count

print count

8.1.4 (d) Understand and use the concepts of **totalling** and **counting**

3. Combining Totalling and Counting:

In many scenarios, you might need to both total and count elements.

For example, finding the average of a set of numbers involves totalling and counting.

 Combining Totalling and Counting.py - C:/Program Files/Python312/Comb

File Edit Format Run Options Window Help

```
# Example: Finding the average of a list of numbers
numbers = [10, 20, 30, 40, 50]
total = 0
count = 0

for num in numbers:
    total += num
    count += 1

average = total / count

print("The average is:", average)
```

```
= RESTART: C:/Program
The average is: 30.0
```

Pseudocode: Finding the average of a list of numbers
numbers <- [10, 20, 30, 40, 50]

total <- 0
count <- 0

// Loop through the list of numbers

for each num in numbers:

 total <- total + num **# Accumulate the sum of numbers**

 count <- count + 1 **# Increment count for each number**

average <- total / count **# Calculate the average**

// Display the average

print("The average is:", average)

8.1.4 (e) Understand and use the concept of string handling: Including: length, substring, upper, lower and the first character of the string can be position zero or one

Click the link below to watch the use of basic string handling

<https://www.youtube.com/watch?v=eAbXPmW2f04>

8.1 Programming

The use of basic string handling

CAMBRIDGE
IGCSE
(0478/0984)



3:50
ave.c

8.1.4 (e) Understand and use the concept of string handling: Including: length, substring, upper, lower and the first character of the string can be position zero or one

String manipulation

Example 1: An online form

Regardless of how the user inputs their first and last names, the program should store the names internally with the first letter as a capital and the rest in lower case.

So, "crAIG SARGEnt" would become "Craig Sargent".

This could be achieved by the following steps:

- **STEP 1:** Extract the first letter from the input and convert it to upper case.
- **STEP 2:** Extract all the remaining letters and convert them to lower case.
- **STEP 3:** Join the two parts back together.

Form Layout

* Emp First Name

Emp Middle Initial

* Emp Last Name

* Emp Part Or Full Time

Emp Salary

Emp Dept

Emp Hiredate 

Emp Manager

Emp Special Info

Emp Telecommute

* Rec Create Date 

Rec Update Date 

8.1.4 (e) Understand and use the concept of **string handling**: Including: **length**, **substring**, **upper**, **lower** and **the first character of the string can be position zero or one**

String manipulation

Example 2: Text parsing

You can input phrases into search engines like “6cm in inches”. The search engine then parses this text string and provides a sensible output.

- Extract the number 6.
- Extract the characters “cm”.
- Extract the characters “inches”.

The screenshot shows a Google search for "6cm in inches". The search bar contains the text "6cm in inches". Below the search bar, the results show "About 21,200,000 results (0.32 seconds)". A unit conversion widget is displayed, showing "Length" as the category. The input is "6" in "Centimetre", and the output is "2.3622" in "Inch". An orange dashed box highlights the input "6" and the output "2.3622". Below the widget, a formula is shown: "divide the length value by 2.54". Below the widget, there is a section titled "People also ask" with four questions: "What size is 6 cm in inches?", "What size is 20x30 cm in inches?", "Is 3 cm bigger than 1 inch?", and "How many cm exactly is an inch?".

8.1.4 (e) Understand and use the concept of **string handling**: Including: **length**, **substring**, **upper**, **lower** and **the first character of the string can be position zero or one**

String manipulation commands (Python)

To get the length of a string

Concept	Keyword/Symbol
String length	<code>len()</code>

Returns the length of a string.

`length = len(string)` e.g., if string was "Hello", length would be 5.

Converting cases

Concept	Keyword/Symbol
Uppercase	<code>.upper</code>
Lowercase	<code>.lower</code>

Returns the string in upper case or lower case.

`ustring = string.upper()` e.g., if string was "Hello", ustring would be "HELLO".

`lstring = string.lower()` e.g., if string was "Hello", lstring would be "hello".

To get a substring

Concept	Keyword/Symbol
Substrings	<code>string[x:x]</code>

Returns part of a string, starting at the character of the first parameter and counting up by the number in the second parameter.

`chars = string[2:3]` e.g., if string was "Hello", chars would be "l".

8.1.4 (e) Understand and use the concept of **string handling**: Including: **length**, **substring**, **upper**, **lower** and **the first character of the string can be position zero or one**



String manipulation commands

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

String operations

LENGTH (<identifier>)

Returns the integer value representing the length of string.
The identifier should be of data type string.

LCASE (<identifier>)

Returns the string/character with all characters in lower case.
The identifier should be of data type string or char.

UCASE (<identifier>)

Returns the string/character with all characters in upper case.
The identifier should be of data type string or char.

SUBSTRING (<identifier>, <start>, <length>)

Returns a string of length <length> starting at position <start>.
The identifier should be of data type string, length and start should be positive and data type integer.

String operations

LENGTH ("Happy Days")
will return 10

LCASE ('W')
will return 'w'

UCASE ("Happy")
will return "HAPPY"

SUBSTRING ("Happy Days", 1, 5)
will return "Happy"

8.1.4 (e) Understand and use the concept of **string handling**: Including: **length**, **substring**, **upper**, **lower** and **the first character of the string can be position zero or one**

Length of a String:

 Combining Totalling and Counting.py - C:/Program File

```
File Edit Format Run Options Window Help
text = "Hello, World!"
length = len(text)
print("Length of the string:", length)
```

```
= RESTART: C:/Program File
Length of the string: 13
|
```

Substring:

 Combining Totalling and Counting.py - C:/Program Files/Python312/Combining Totalling an...

```
File Edit Format Run Options Window Help
text = "Hello, Nouredine!"
substring = text[7:12] # Extracts characters from index 7 to 11
print("Substring:", substring)
```

```
==== RESTART: C:/P:
Substring: Noure
|
```

Pseudocode:

```
text <- "Hello, World!"
length <- calculate_length(text)
display "Length of the string:", length
```

Pseudocode:

```
text <- "Hello, Nouredine!"
substring <- extract_substring(text, 7, 12)
display "Substring:", substring
```

8.1.4 (e) Understand and use the concept of string handling: Including: length, substring, upper, lower and the first character of the string can be position zero or one

Uppercase:

```
File Edit Format Run Options Window Help
text = "Hello, Nouredine!"
uppercase_text = text.upper()
print("Uppercase:", uppercase_text)

==== RESTART: C:/Program File:
Uppercase: HELLO, NOUREDDINE!
|
```

Lowercase:

```
File Edit Format Run Options Window Help
text = "Hello, Nouredine!"
lowercase_text = text.lower()
print("Lowercase:", lowercase_text)

==== RESTART: C:/Program Files,
Lowercase: hello, nouredine!
|
```

Pseudocode:

```
text <- "Hello, Nouredine!"
uppercase_text <- convert_to_uppercase(text)
display "Uppercase:", uppercase_text
```

Pseudocode:

```
text <- "Hello, Nouredine!"
lowercase_text <- convert_to_lowercase(text)
display "Lowercase:", lowercase_text
```

8.1.4 (e) Understand and use the concept of string handling: Including: length, substring, upper, lower and the first character of the string can be position zero or one

Accessing Characters by Index:

File Edit Format Run Options Window Help

```
text = "Hello, Nouredine!"
first_character = text[0]
second_character = text[7]
print("First Character:", first_character)
print("Second Character:", second_character)
```

```
==== RESTART: C:/Prog
First Character: H
Second Character: N
```

Pseudocode:

```
text <- "Hello, Nouredine!"
first_character <- get_character_at_index(text, 0)
second_character <- get_character_at_index(text, 7)
display "First Character:", first_character
display "Second Character:", second_character
```

8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Click the link below to watch the video on the common arithmetic and comparison operators

<https://www.youtube.com/watch?v=glsljucGazo>

8.1 Programming

The common arithmetic
and comparison operators



CAMBRIDGE
IGCSE
(0478/0984)

5:05
ave

8.1.4 (f) Understand and use **arithmetic**, **logical** and **Boolean operators**



Common arithmetic operators

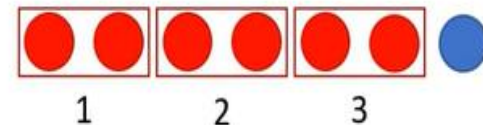
Operator	Example	Meaning	Result
+	$x = 7 + 2$	Addition	9
-	$x = 7 - 2$	Subtraction	5
*	$x = 7 * 2$	Multiplication	14
/	$x = 7 / 2$	Division	3.5
^	$x = 7 ^ 2$	Exponentiation	49
MOD	$x = 7 \text{ MOD } 2$	Modulus	1
DIV	$x = 7 \text{ DIV } 2$	Integer division	3

Arithmetic
Logic Unit



7 DIV 2

How many times does the number on the right fit into the number on the left?



8.1.4 (f) Understand and use **arithmetic**, **logical** and **Boolean operators**



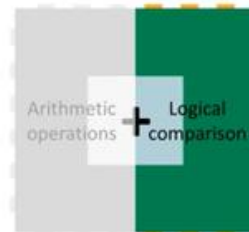
Common comparison operators

Operator	Example	Meaning
=	7 = 7	Equal to
<>	7 <> 5	Not equal to
<	5 < 7	Less than
<=	5 <= 5	Less than or equal to
>	7 > 5	Greater than
>=	7 >= 7	Greater than or equal to

Note: Different languages may use alternate symbols to represent these operators. For example, "not equal to" could be:

*!=
<>
≠*

Arithmetic
Logic Unit



8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Click the link below to watch the video on the common Boolean operators

https://www.youtube.com/watch?v=YN6IWw_O8pA

8.1 Programming

The common Boolean operators

CAMBRIDGE
IGCSE
(0478/0984)



4:01
dave.c

8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Boolean operators

We use **Boolean expressions** all the time in programming. Typical examples are IF statements and FOR or WHILE loops.

Boolean expressions typically make use of one or more **logical operators** and evaluate to either **TRUE** or **FALSE**.

You need to be aware of how the following **Boolean operators** function.

AND
OR
NOT

8.1.4 (f) Understand and use **arithmetic**, **logical** and **Boolean operators**

Boolean operators

```
IF <BOOLEAN EXPRESSION>
```

```
    THEN
```

```
    ...
```

```
ENDIF
```

```
WHILE <BOOLEAN EXPRESSION> DO
```

```
    ...
```

```
ENDWHILE
```

```
REPEAT
```

```
    ...
```

```
UNTIL <BOOLEAN EXPRESSION>
```

8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Boolean operators

NOT

```
end_of_file = False
```

```
WHILE not end_of_file
```

...is the same as...

```
while end_of_file == False:
```

AND

```
end_of_file = False
```

```
found = False
```

```
WHILE not end_of_file and not found
```

OR

```
IF x == 2 or x == 4
```

```
WHILE choice < 1 or choice > 3
```

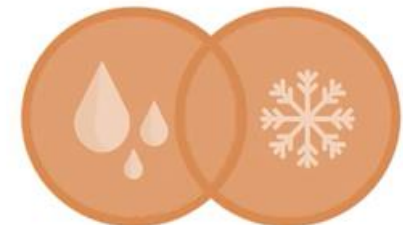
Use an umbrella if it's raining but NOT cold.



Wear a coat if it's raining AND cold.



Catch a bus if it's raining OR cold.



8.1.4 (f) Understand and use **arithmetic, logical** and **Boolean operators**

Boolean operators

NOT

```
end_of_file = False
```

```
WHILE not end_of_file
```

...is the same as...

```
while end_of_file == False:
```

AND

```
end_of_file = False
```

```
found = False
```

```
WHILE not end_of_file and not found
```

OR

```
IF x == 2 or x == 4
```

```
WHILE choice < 1 or choice > 3
```

Creating an infinite loop

```
WHILE True
```



8.1.4 (f) Understand and use **arithmetic, logical and Boolean operators**

Logical Operators:

Logical AND:

Pseudocode

x <- True

y <- False

result <- x AND y

DISPLAY "Result of Logical AND:", result

Logical OR:

Pseudocode

x <- True

y <- False

result <- x OR y

DISPLAY "Result of Logical OR:", result

Logical AND.py - C:/Program Files/Python312/Logical AND.py (3

File Edit Format Run Options Window Help

```
x = True
y = False

result = x and y
print("Result of Logical AND:", result)
```

```
===== RESTART: C:/Pr
Result of Logical AND: False
|
```

Logical AND.py - C:/Program Files/Python312/Logical A

File Edit Format Run Options Window Help

```
x = True
y = False

result = x or y
print("Result of Logical OR:", result)
```

```
===== RESTART: C:/I
Result of Logical OR: True
|
```


8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Logical Operators:

Logical NOT:

Pseudocode

```
x <- True
```

```
result <- NOT x
```

```
DISPLAY "Result of Logical NOT:", result
```

Logical OR:

Pseudocode

```
DECLARE age: INTEGER
```

```
// Get age input from the user
```

```
INPUT "Enter your age: ", age
```

```
// Checking if the person is eligible for a driving license
```

```
IF age >= 18 OR age >= 16 THEN
```

```
    OUTPUT "You are eligible for a driving license."
```

```
ELSE
```

```
    OUTPUT "You are not eligible for a driving license."
```

```
ENDIF
```

```
Logical AND.py - C:/Program Files/Python312/Logical AN
File Edit Format Run Options Window Help
x = True
result = not x
print("Result of Logical NOT:", result)
```

```
===== RESTART: C:/Pr
Result of Logical NOT: False
```

```
File Edit Format Run Options Window Help
```

```
# Logical OR in Python
```

```
age = int(input("Enter your age: "))
```

```
# Checking if the person is eligible for a driving license
```

```
if age >= 18 or age >= 16:
```

```
    print("You are eligible for a driving license.")
```

```
else:
```

```
    print("You are not eligible for a driving license.")
```

```
===== RESTART: C:/Program Files/Py
Enter your age: 17
You are eligible for a driving license.
```

```
===== RESTART: C:/Program Files/Py
Enter your age: 16
You are eligible for a driving license.
```

```
===== RESTART: C:/Program Files/Py 100
Enter your age: 12
You are not eligible for a driving license.
```

8.1.4 (f) Understand and use **arithmetic, logical and Boolean operators**

Boolean Operators:

If Statement :

Pseudocode

```
x <- 10
y <- 20

IF x > 5 AND y < 30 THEN
    DISPLAY "Both conditions are true."
ELSE
    DISPLAY "At least one condition is false."
ENDIF
```

while Statement : The while statement is used to execute a block of code repeatedly as **long as** a condition is **true**.

Pseudocode

```
count <- 0

WHILE count < 5 DO
    DISPLAY "Count:", count
    count <- count + 1
END WHILE
```

 Logical AND.py - C:/Program Files/Python312/Logical AND.py (3.

File Edit Format Run Options Window Help

```
x = 10
```

```
y = 20
```

```
if x > 5 and y < 30:
```

```
    print("Both conditions are true.")
```

```
else:
```

```
    print("At least one condition is false.")
```

```
===== RESTART: C:/
```

```
Both conditions are true.
```

```
|
```

 Logical AND.py - C:/Program Files/Pyth

File Edit Format Run Options Windc

```
count = 0
```

```
while count < 5:
```

```
    print("Count:", count)
```

```
    count += 1
```

```
=====
```

```
Count: 0
```

```
Count: 1
```

```
Count: 2
```

```
Count: 3
```

```
Count: 4
```

8.1.4 (f) Understand and use arithmetic, logical and Boolean operators

Boolean Operators:

REPEAT – UNTIL Statement : The REPEAT - UNTIL statement, on the other hand, executes a block of code repeatedly **until a condition becomes true**.

Pseudocode

count <- 0

REPEAT

 DISPLAY "Count:", count

 count <- count + 1

UNTIL count >= 10

 Logical AND.py - C:/Program Files/Pyt

File Edit Format Run Options Wind

```
count = 0
```

```
while True:
    print("Count:", count)
    count += 1
    if count >= 10:
        break
```

```
===== RESTART:
Count: 0
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
Count: 6
Count: 7
Count: 8
Count: 9
```

8.1.5 Understand and use **nested** statements

Click the link below to watch the video how to use nested statements

<https://www.youtube.com/watch?v=eZqFsqVw4yw>

8.1 Programming

Using nested statements

CAMBRIDGE
IGCSE
(0478/0984)



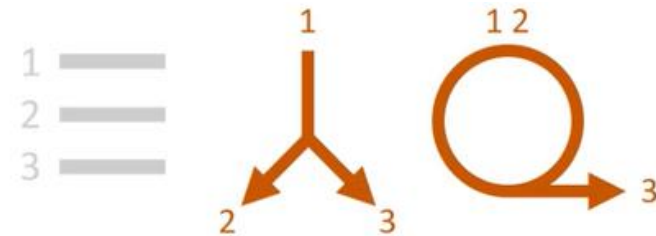
3:17
ave.c

8.1.5 Understand and use **nested** statements



Nested structures

Nesting means putting one statement inside another – this can be achieved with iteration and selection statements, or a mix of both.

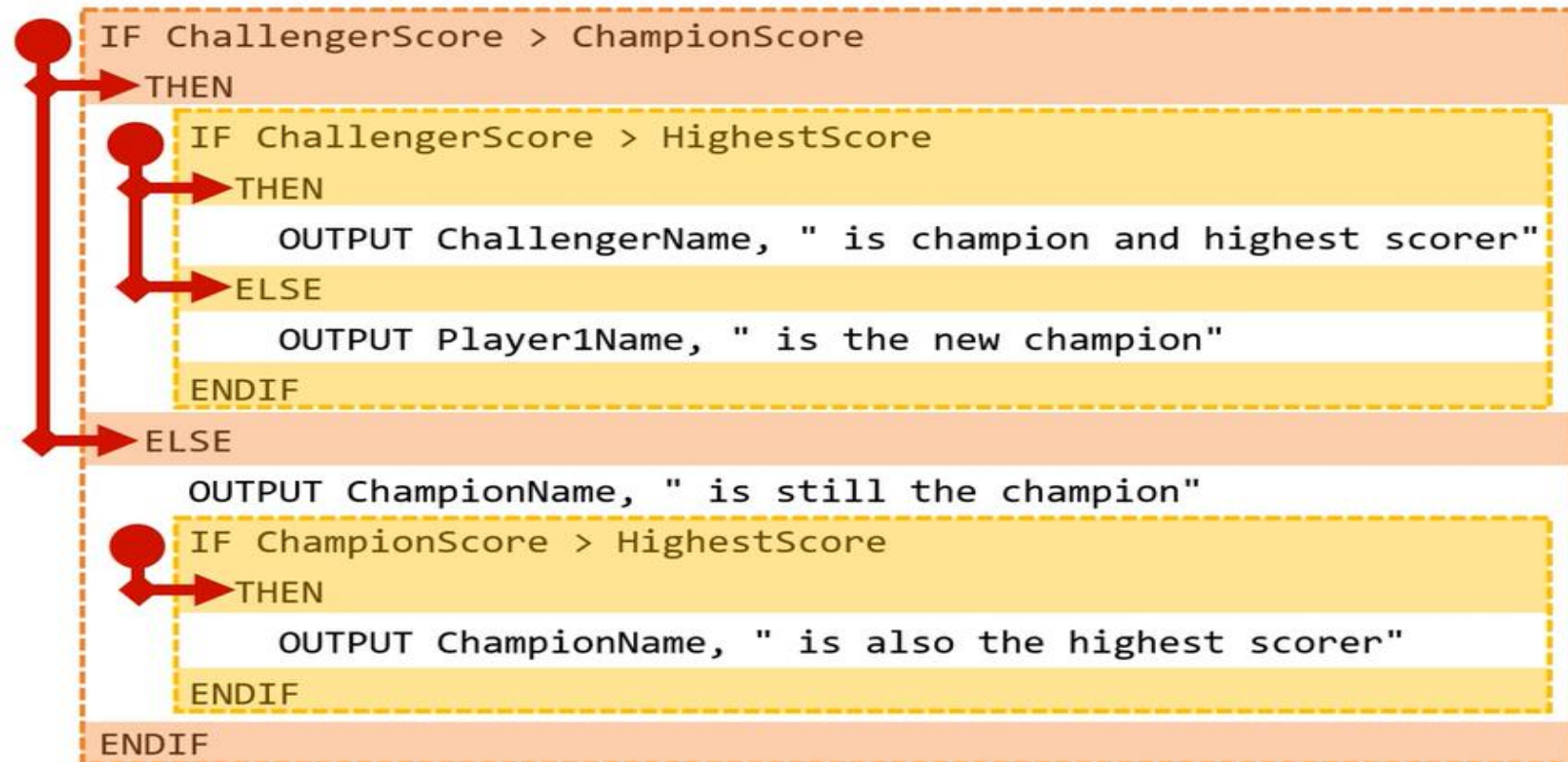


```
WHILE NotSolved
  ... instructions here...
  FOR i ← 1 TO 5
    ... instructions here...
  ENDFOR
  ... instructions here...
ENDWHILE
```

```
IF GameWon
  THEN
    ... instructions here...
    IF Score > HighScore
      THEN
        ... instructions here...
      ENDIF
    ... instructions here...
  ENDIF
ENDIF
```

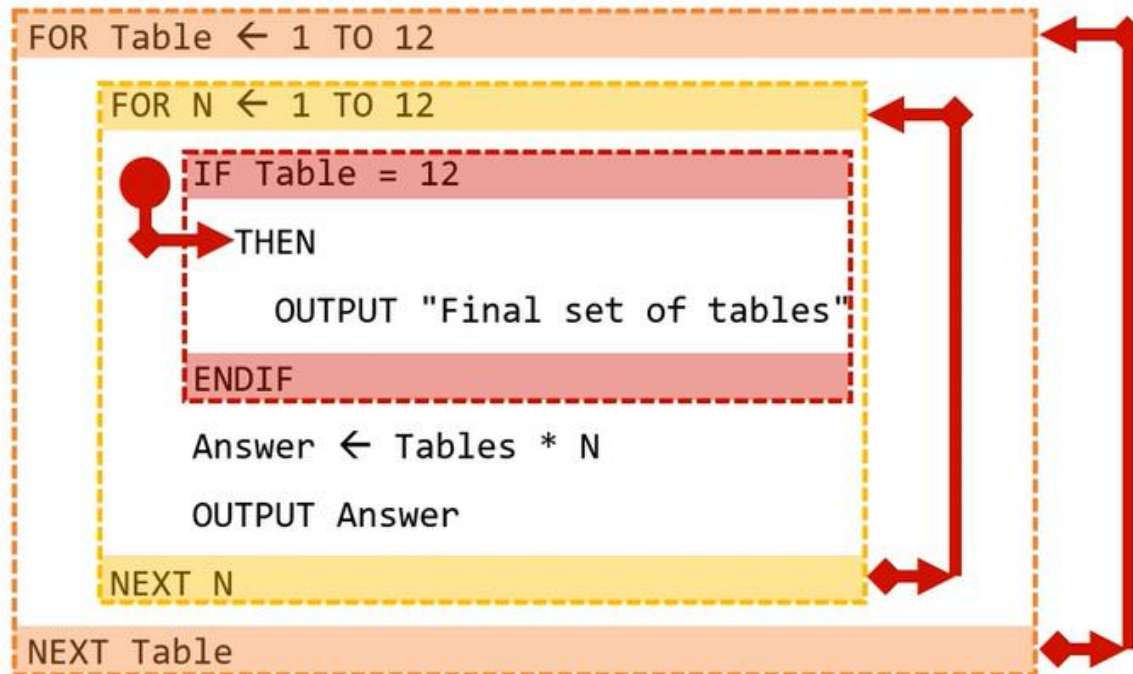
8.1.5 Understand and use **nested** statements

Nested statement examples using Cambridge IGCSE pseudocode



8.1.5 Understand and use **nested** statements

Although you can nest statements within statements to many levels, you will not be expected to write more than *three*. The most complex nest statement in the exam will be something like the following example:



8.1.5 Understand and use **nested** statements

1. Nested Selection (IF statements): nested selection using nested IF statements

Pseudocode

```
x <- 10
y <- 5

IF x > 0 THEN
    DISPLAY "x is positive"
    IF y > 0 THEN
        DISPLAY "y is also positive"
    ELSE
        DISPLAY "y is not positive"
    END IF
ELSE
    DISPLAY "x is not positive"
END IF
```

 Logical AND.py - C:/Program Files/Python312/Logic

File Edit Format Run Options Window Help

```
x = 10
```

```
y = 5
```

```
if x > 0:
    print("x is positive")
    if y > 0:
        print("y is also positive")
    else:
        print("y is not positive")
else:
    print("x is not positive")
```

```
= RESTART: C:/Program F
x is positive
y is also positive
|
```

8.1.5 Understand and use **nested** statements

2. **Nested Iteration (Nested Loops):**

Nested iteration using nested loops (a loop inside another loop)

Pseudocode

```
FOR i FROM 0 TO 2 DO
    FOR j FROM 0 TO 1 DO
        DISPLAY "(", i, ", ", ", j, ")"
    END FOR
END FOR
```

 Logical AND.py - C:/Program Files/Python312/Lc

File Edit Format Run Options Window Help

```
for i in range(3):
    for j in range(2):
        print(f"({i}, {j})")
```

```
===== ]
(0, 0)
(0, 1)
(1, 0)
(1, 1)
(2, 0)
(2, 1)
|
```

8.1.6 (a) Understand what is meant by procedures, functions and parameters
Click the link below to watch the video Procedures, functions and parameters
<https://www.youtube.com/watch?v=hiaejS7bgQQ>

8.1 Programming

Procedures, functions
and parameters

CAMBRIDGE
IGCSE
(0478/0984)



6:33
AVE C

8.1.6 (a) Understand what is meant by procedures, functions and parameters

Procedures and functions

```
def roll_a_dice():
    #Roll a dice
    return random.randint(1,6)

def order_dice(d1, d2):
    #Change the order of the dice to highest value first
    #and join the values together
    if d1 >= d2:
        return int(d1 + d2)
    else:
        return int(d2 + d1)

def output_throw():
    #Output the results of the two dice thrown
    dice1 = 0
    dice2 = 0
    while dice1 == dice2:
        dice1 = str(roll_a_dice())
        dice2 = str(roll_a_dice())
    print("Dice 1:", dice1)
    print("Dice 2:", dice2)
    roll = order_dice(dice1, dice2)
    return roll

#Main program
#This tool outputs the value of two dice
import random
print(output_throw())
```

Advantages of sub-programs

- Programs are easier to write and debug.
- Components can be easily reused.
- **Functions** can be stored in a library for easy reuse across multiple programs.
- For example, *import random* imports the *random* library of **functions** into a program.
- Programs are also easier to test.

There are two main types of sub-program.

Procedures

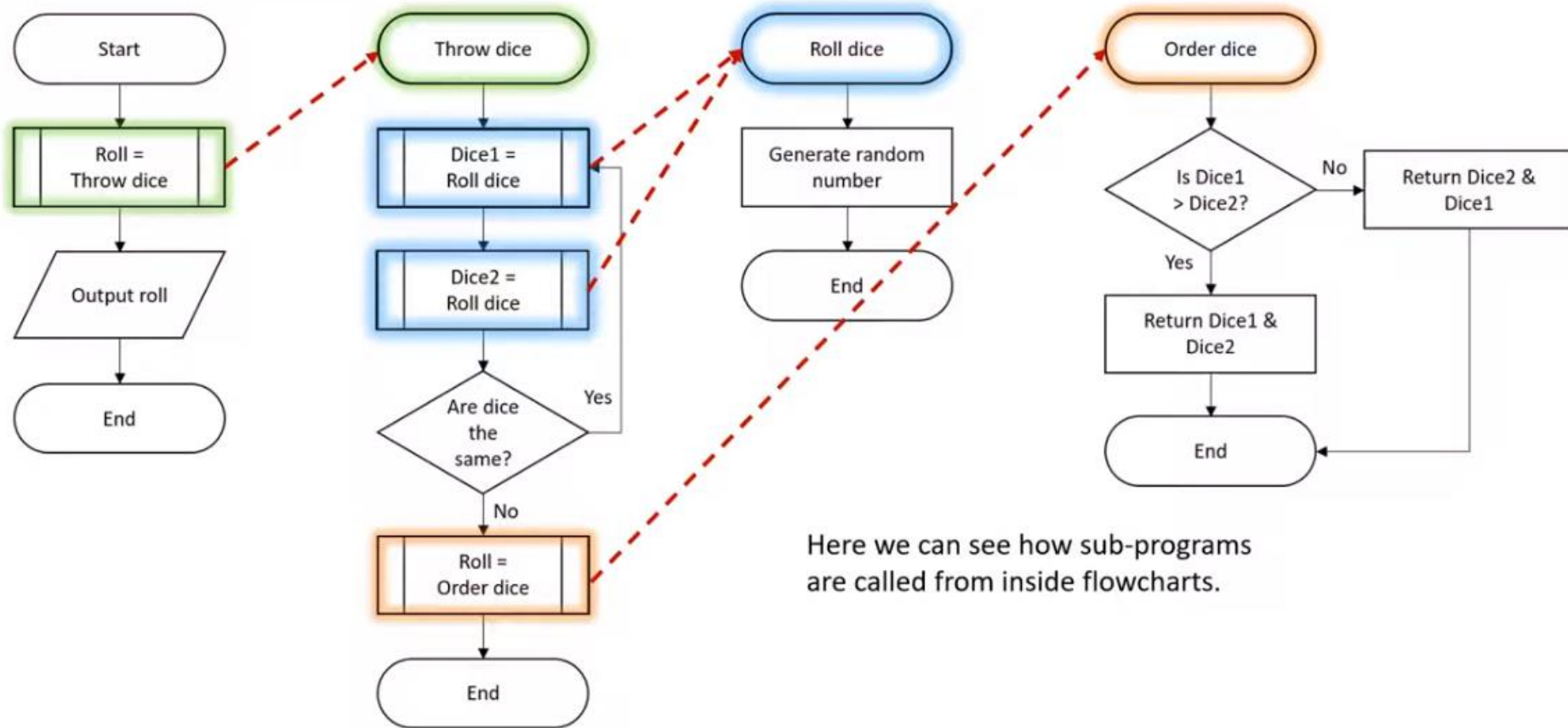
- Carry out a task.
- Provide structure to code.

Functions

- Carry out a task **and** return a value.
- Create reusable program components.

8.1.6 (a) Understand what is meant by procedures, functions and parameters

Procedures and functions



8.1.6 (a) Understand what is meant by procedures, functions and parameters

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Procedures

A procedure with no parameters:

```
PROCEDURE <identifier>  
    <statements>  
ENDPROCEDURE
```

A procedure **with** parameters:

```
PROCEDURE <identifier> (<param1>:<datatype>, <param2>:<datatype> ...)  
    <statements>  
ENDPROCEDURE
```

Procedures should be called as follows:

```
CALL <identifier>  
CALL <identifier>(Value1, Value2 ...)
```

8.1.6 (a) Understand what is meant by procedures, functions and parameters

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Procedures

```
PROCEDURE Default Line
    CALL LINE(60)
ENDPROCEDURE

PROCEDURE Line(Size : INTEGER)
    DECLARE Length : INTEGER
    FOR Length ← 1 TO Size
        OUTPUT '-'
    NEXT Length
ENDPROCEDURE

IF MySize = Default
    THEN
        CALL DefaultLine
    ELSE
        CALL Line(MySize)
ENDIF
```


8.1.6 (a) Understand what is meant by procedures, functions and parameters

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Functions

A function with no parameters:

```
FUNCTION <identifier> RETURNS <data type>
    <statements>
ENDFUNCTION
```

A function **with** parameters:

```
FUNCTION <identifier> (<param1>:<datatype>, <param2>:<datatype> ...) RETURNS <data type>
    <statements>
ENDFUNCTION
```

The keyword CALL should not be used when calling a function.

Functions should only be called as part of an expression.

When the RETURN statement is executed, the value returned replaces the function call in the expression and the expression is then evaluated.

8.1.6 (a) Understand what is meant by procedures, functions and parameters

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Functions

```
FUNCTION SumSquare(Number1:INTEGER, Number2:INTEGER) RETURNS INTEGER
    RETURN (Number1 * Number1) + (Number2 * Number2)
ENDFUNCTION
```

```
OUTPUT "Sum of squares = ", SumSquare(10, 20)
```

8.1.6 (a) Understand what is meant by procedures, functions and parameters



Procedures and functions

- Larger programs are developed as a set of subroutines or *sub-programs*.
- Structuring code using sub-programs makes it easier to read and debug.
- **Functions** return values and create reusable program components.
- **Procedures** structure a program in a modular way, making it easier to read.
- Each sub-program can be tested independently.
- Sub-programs can be saved in libraries and reused in other programs.

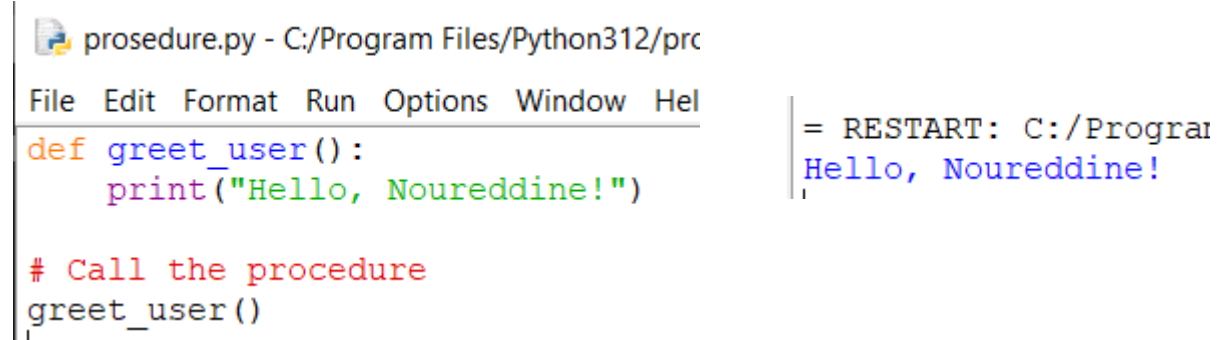
8.1.6 (a) Understand what is **meant** by **procedures**, **functions** and **parameters**

1. Procedures:

Procedures are sets of instructions that perform a specific task but **do not return a value**.

Pseudocode

```
PROCEDURE GreetUser():  
    DISPLAY "Hello, Noureddine!"  
END PROCEDURE
```



The screenshot shows a Python IDE window titled 'prosedure.py - C:/Program Files/Python312/prc'. The menu bar includes File, Edit, Format, Run, Options, Window, and Hel. The code defines a procedure named 'greet_user' which prints 'Hello, Noureddine!'. A comment indicates the procedure is called. The output window on the right shows the command prompt with the command 'python C:/Program Files/Python312/prc/procedure.py' and the output 'Hello, Noureddine!'.

```
prosedure.py - C:/Program Files/Python312/prc  
File Edit Format Run Options Window Hel  
def greet_user():  
    print("Hello, Noureddine!")  
  
# Call the procedure  
greet_user()
```

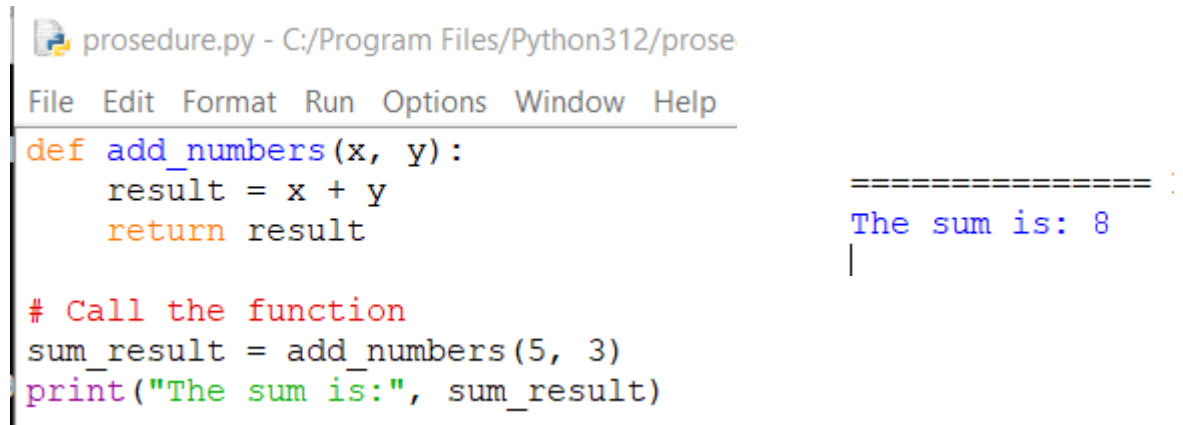
```
= RESTART: C:/Program Files/Python312/prc/procedure.py  
Hello, Noureddine!
```

2. Functions:

Functions are similar to procedures but **return a value** after performing a specific task.

Pseudocode

```
FUNCTION AddNumbers(x, y):  
    result <- x + y  
    RETURN result  
END FUNCTION
```



The screenshot shows a Python IDE window titled 'prosedure.py - C:/Program Files/Python312/prose'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code defines a function named 'add_numbers' which takes parameters 'x' and 'y', calculates their sum, and returns it. A comment indicates the function is called. The output window on the right shows the command prompt with the command 'python C:/Program Files/Python312/prose/procedure.py' and the output 'The sum is: 8'.

```
prosedure.py - C:/Program Files/Python312/prose  
File Edit Format Run Options Window Help  
def add_numbers(x, y):  
    result = x + y  
    return result  
  
# Call the function  
sum_result = add_numbers(5, 3)  
print("The sum is:", sum_result)
```

```
=====  
The sum is: 8
```

8.1.6 (a) Understand what is meant by procedures, functions and parameters

3.Parameters:

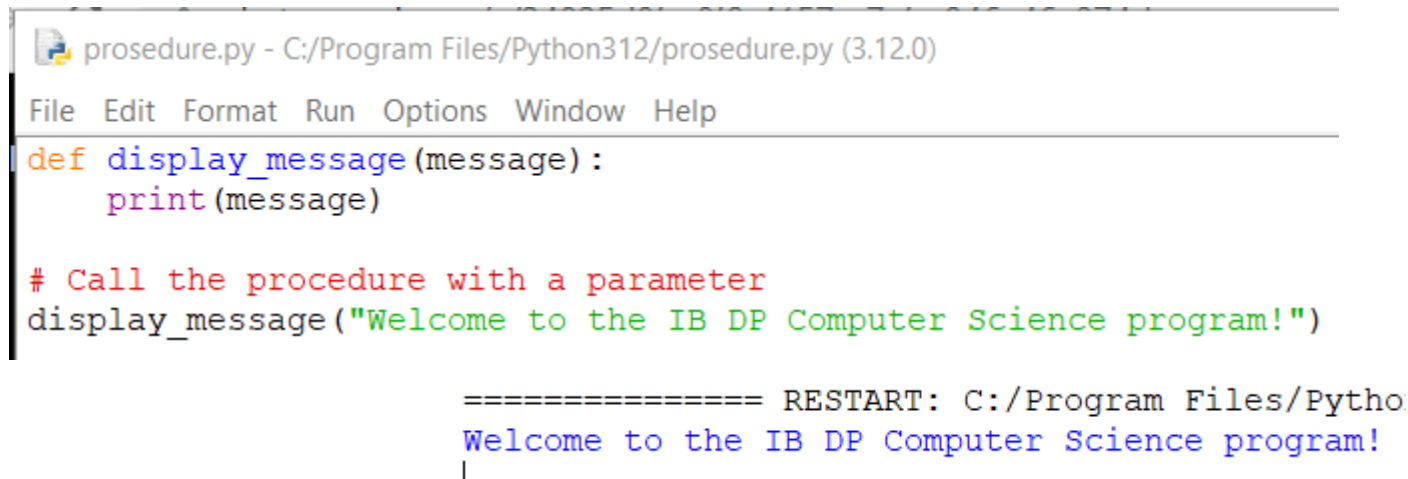
Parameters are **placeholders** in a procedure or function that receive values when the procedure or function is called.

Pseudocode

PROCEDURE DisplayMessage(message):

DISPLAY message

END PROCEDURE



```
prosedure.py - C:/Program Files/Python312/prosedure.py (3.12.0)
File Edit Format Run Options Window Help
def display_message(message) :
    print(message)

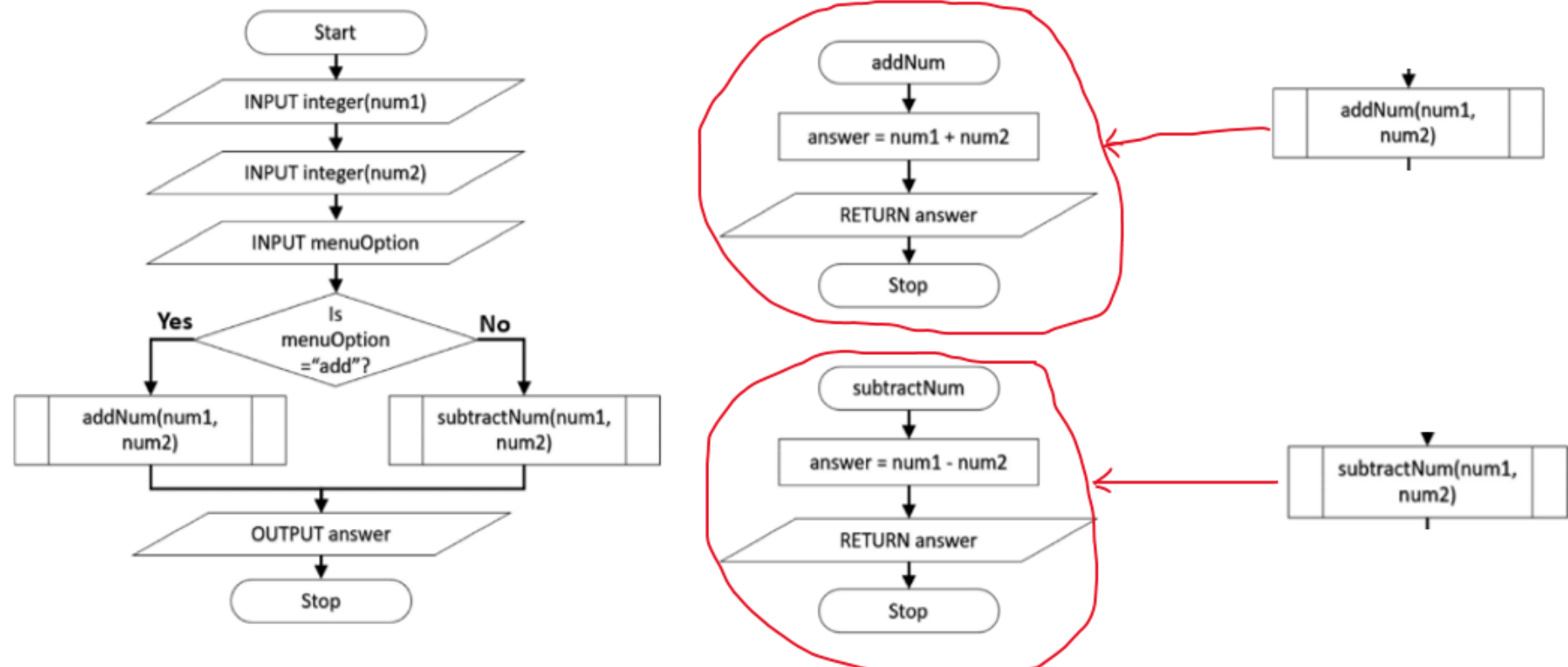
# Call the procedure with a parameter
display_message("Welcome to the IB DP Computer Science program!")

===== RESTART: C:/Program Files/Pytho
Welcome to the IB DP Computer Science program!
|
```

8.1.6 (a) Understand what is meant by procedures, functions and parameters

Sub-Programmes (Functions/Procedures) and collections within programmed solutions

Subroutine explained: Example 1



8.1.6 (a) Understand what is meant by procedures, functions and parameters

Sub-Programmes (Functions/Procedures) and collections within programmed solutions

Subroutine explained: Example 1

```
SUBROUTINE addition()  
  num1 <-- integer(USERINPUT)  
  num2 <-- integer(USERINPUT)  
  total <-- num1 + num2  
  returningValues = (num1,num2,total)  
  RETURN returningValues  
ENDSUBROUTINE
```

```
name <-- USER  
INPUT num1,num2,total <-- addition()  
OUTPUT "Hello" + name  
OUTPUT "You entered" + num1 + "and" + num2  
OUTPUT "The total is" + total
```

```
C:\Users\n.tadjerout\PycharmProj  
Enter your name: Nouredine  
Enter the first number: 20  
Enter the second number: 12  
Hello Nouredine  
You entered 20 and 12  
The total is 32
```

```
main.py x  
1  # Subroutine for addition  
  1 usage  
2  def addition():  
3      # Get user input  
4      num1 = int(input("Enter the first number: "))  
5      num2 = int(input("Enter the second number: "))  
6  
7      # Calculate the total  
8      total = num1 + num2  
9  
10     # Return values as a tuple  
11     return num1, num2, total  
12  
13  
14  # Main program  
15  name = input("Enter your name: ")  
16  num1, num2, total = addition()  
17  
18  # Output results  
19  print("Hello " + name)  
20  print("You entered", num1, "and", num2)  
21  print("The total is", total)  
22
```


8.1.6 (b) Define and use procedures and functions, with or without parameters

1. Procedure without Parameters:

Pseudocode:

PROCEDURE GreetUser():

DISPLAY "Hello, IGCSE CS Y11!"

END PROCEDURE

prosedure.py - C:/Program Files/Python312/prose

File Edit Format Run Options Window Help

```
def greet_user():  
    print("Hello, IGCSE CS Y11!")  
  
# Call the procedure  
greet_user()
```

```
===== RESTART: C:/Pr  
Hello, IGCSE CS Y11!  
|
```

2. Procedure with Parameters:

Pseudocode:

PROCEDURE DisplayMessage(message):

DISPLAY message

END PROCEDURE

prosedure.py - C:/Program Files/Python312/procedure.py (3.12.0)

File Edit Format Run Options Window Help

```
def display_message(message):  
    print(message)  
  
# Call the procedure with a parameter  
display_message("Welcome to IGCSE CS program!")
```

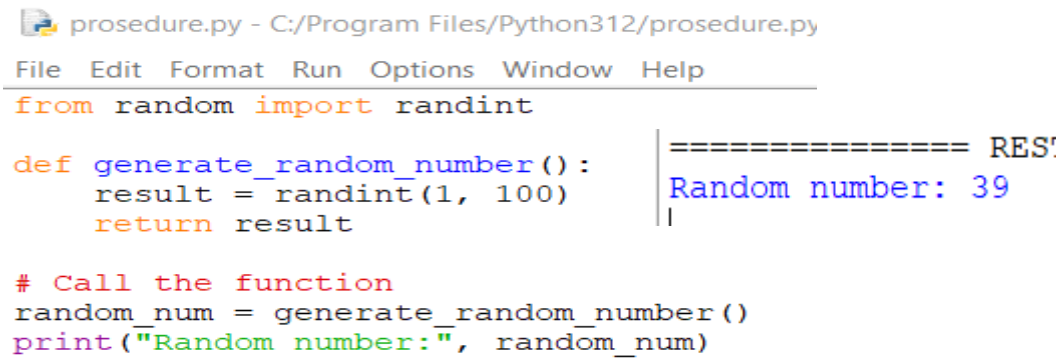
```
===== RESTART: C:/Pr  
Welcome to IGCSE CS program!  
|
```

8.1.6 (b) Define and use procedures and functions, with or without parameters

3. Function without Parameters:

Pseudocode:

```
FUNCTION GenerateRandomNumber():  
    result <- RANDOM(1, 100)  
    RETURN result  
END FUNCTION
```



The screenshot shows a Python IDE window titled 'prosedure.py - C:/Program Files/Python312/prosedure.py'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code defines a function 'generate_random_number()' that uses 'randint' from the 'random' module to generate a random integer between 1 and 100. It then calls this function and prints the result. The output in the console shows 'Random number: 39'.

```
from random import randint  
  
def generate_random_number():  
    result = randint(1, 100)  
    return result  
  
# Call the function  
random_num = generate_random_number()  
print("Random number:", random_num)
```

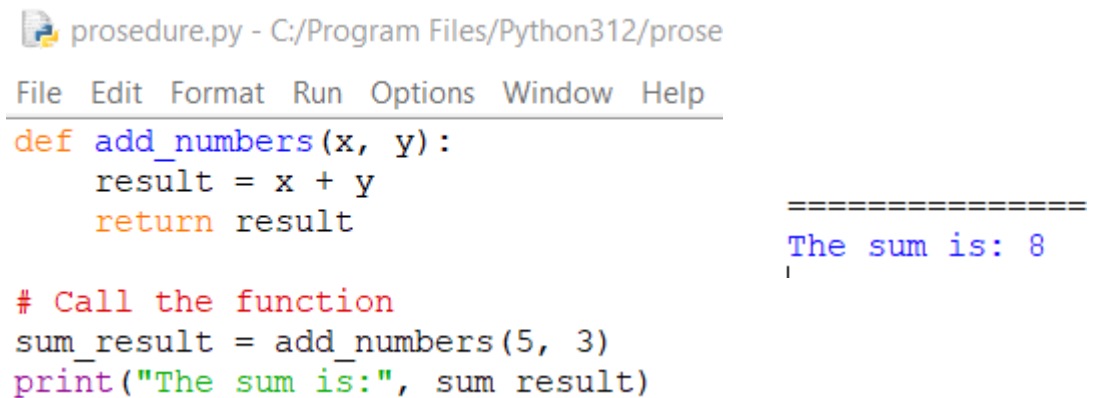
```
===== RES!  
Random number: 39  
|
```

Note: The **randint** Python function returns a randomly generated integer from the specified range, with both inclusive parameters

4. Function with Parameters::

Pseudocode:

```
FUNCTION AddNumbers(x, y):  
    result <- x + y  
    RETURN result  
END FUNCTION
```



The screenshot shows a Python IDE window titled 'prosedure.py - C:/Program Files/Python312/prose'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code defines a function 'add_numbers(x, y)' that takes two parameters and returns their sum. It then calls this function with arguments 5 and 3 and prints the result. The output in the console shows 'The sum is: 8'.

```
def add_numbers(x, y):  
    result = x + y  
    return result  
  
# Call the function  
sum_result = add_numbers(5, 3)  
print("The sum is:", sum_result)
```

```
=====  
The sum is: 8  
|
```

8.1.6 (c) Understand and use **local** and **global variables**

Click the link below to watch the video Local and global variables

https://www.youtube.com/watch?v=4Y-HQOR_DrM

8.1 Programming

Local and global variables

CAMBRIDGE
IGCSE
(0478/0984)



6:36
lave.c

8.1.6 (c) Understand and use **local** and **global variables**

The importance of local variables

```
def rollTwoDice():  
    x = 0  
    y = 0  
    while x == y:  
        x = random.randint(1,6)  
        y = random.randint(1,6)  
    if x > y:  
        return (x)  
    else:  
        return (y)
```

```
def displayScoreBonus():  
    x = 0  
    y = 0  
    while x == y:  
        x = random.randint(1,6)  
        y = random.randint(1,6)  
    if x > y:  
        bonus = 100  
    else:  
        bonus = 200  
    print("Bonus score is: " + str(bonus))
```

Here we see two separate subroutines written in Python:

- rollTwoDice()
- displayScoreBonus()

They both declare their own **local variables** called x and y.

8.1.6 (c) Understand and use **local** and **global variables**



The importance of local variables

Subroutines (**procedures** and **functions**) have their own **local variables**.

Local variables only exist while the subroutine is running.

Local variables are only accessible from inside the subroutine they are assigned to.

Benefits of local variables:

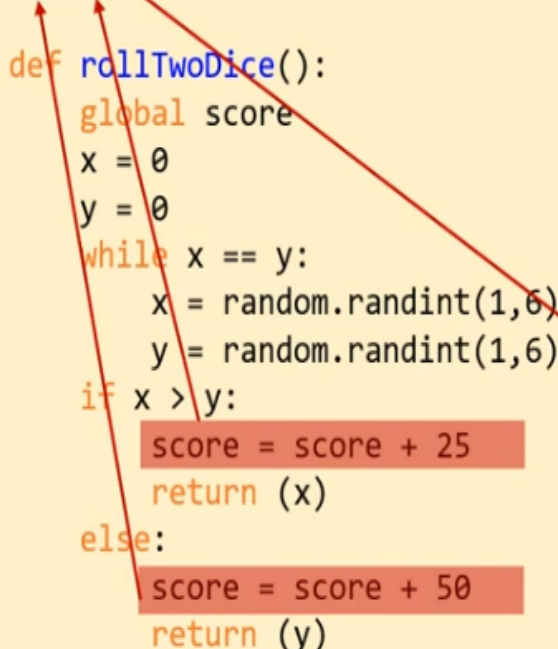
- Help to keep the subroutine self-contained.
- Subroutines can be used as part of a bigger program without **local variables** conflicting with other **variables** of the same name located in other parts of the program.
- Memory-efficient because the **variables** only exist (and take up space in memory) while the subroutine is in use.
- Program will be easier to maintain and debug.

8.1.6 (c) Understand and use **local** and **global variables**

The use of global variables

```
import random
score = 0

def rollTwoDice():
    global score
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        score = score + 25
    return (x)
    else:
        score = score + 50
    return (y)
```



```
def displayScoreBonus():
    global score
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    score = score + bonus

#Main program starts here
print("Higher dice is:", rollTwoDice())
displayScoreBonus()
print("You current score is: " + str(score))
```

We have updated these two subroutines to use a **global variable** called **score** – this **variable** has been declared and initialised to zero outside of these subroutines in the main program.

8.1.6 (c) Understand and use **local** and **global variables**

The use of global variables

```
import random
score = 0

def rollTwoDice():
    global score
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        score = score + 25
        return (x)
    else:
        score = score + 50
        return (y)
```

```
def displayScoreBonus():
    global score
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    score = score + bonus

#Main program starts here
print("Higher dice is:", rollTwoDice())
displayScoreBonus()
print("You current score is: " + str(score))
```

We have updated these two subroutines to use a **global variable** called **score** – this **variable** has been declared and initialised to zero outside of these subroutines in the main program.

Adding the word **global** before the **variable** inside each subroutine allows the subroutine to access this **global variable**. When the code in each subroutine updates the contents of **score**, it is updating the same, single **variable**. You have essentially given access to a **variable** outside of the subroutine, which it would never normally see or know about.

8.1.6 (c) Understand and use local and global variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

We have updated several lines of code and added some new ones to make this a better solution without using the global keyword.

Let's work through it now so you understand what is going on.

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0
```

```
def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

The program starts here. We can see it calls the **function** rollTwoDice and passes in a copy of the **global variable** score when it does this.

The **global variable** score has been copied and can now be used internally by rollTwoDice as a **local** version.

8.1.6 (c) Understand and use local and global variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0
```

```
def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus
```

```
#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

The **function** executes as normal but is now only updating **local** versions of all its **variables**. These include:

- x
- y
- score

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0
```

```
def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

When we hit the **return** lines, we now **return** two sets of values.

We **return** the **local** copy of score, and it ends up back in the main program **variable** retScore.

8.1.6 (c) Understand and use local and global variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0
```

```
def rollTwoDice(score):
```

```
    x = 0
```

```
    y = 0
```

```
    while x == y:
```

```
        x = random.randint(1,6)
```

```
        y = random.randint(1,6)
```

```
    if x > y:
```

```
        return (score + 25), x
```

```
    else:
```

```
        return (score + 50), y
```

```
def displayScoreBonus(score):
```

```
    x = 0
```

```
    y = 0
```

```
    while x == y:
```

```
        x = random.randint(1,6)
```

```
        y = random.randint(1,6)
```

```
    if x > y:
```

```
        bonus = 100
```

```
    else:
```

```
        bonus = 200
```

```
    print("Bonus score is: " + str(bonus))
```

```
    return bonus
```

#Main program starts here

```
retScore, highDice = rollTwoDice(score)
```

```
print("Higher dice is:", highDice)
```

```
score = retScore + (displayScoreBonus(score))
```

```
print("You current score is: " + str(score))
```

When we hit the **return** lines, we now **return** two sets of values.

We **return** the **local** copy of score, and it ends up back in the main program **variable** retScore.

Then we **return** the **local** copy of either x or y, and it ends up back in the main program **variable** highDice.

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

We now print out which of the dice was higher.

The value came from the `rollTwoDice()` function and one of its **local** `x` or `y` variables, but we are not using those anymore.

Instead, it was passed back out to the **global** variable `highDice`, and we are using that version.

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

We now update the **global** copy of the score **variable** to become equal to the current value of retScore plus the result, which will be returned from the **function** displayScoreBonus.

As we call displayScoreBonus, we again pass in a copy of the **global** variable score.

Remember, the **global** variable score has been copied and can now be used internally by displayScoreBonus as a **local** version.

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

The **function** executes as normal, but it is now only updating **local** versions of all its **variables**. These include:

- x
- y
- bonus

8.1.6 (c) Understand and use local and global variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

When we hit the **return** line, we now **return** the **local** copy of bonus. It ends up back in the main program and is used in the calculation to update the **global** version of the score **variable**.

8.1.6 (c) Understand and use **local** and **global** variables

The use of global variables

```
import random
score = 0
highDice = 0
retScore = 0

def rollTwoDice(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        return (score + 25), x
    else:
        return (score + 50), y
```

```
def displayScoreBonus(score):
    x = 0
    y = 0
    while x == y:
        x = random.randint(1,6)
        y = random.randint(1,6)
    if x > y:
        bonus = 100
    else:
        bonus = 200
    print("Bonus score is: " + str(bonus))
    return bonus

#Main program starts here
retScore, highDice = rollTwoDice(score)
print("Higher dice is:", highDice)
score = retScore + (displayScoreBonus(score))
print("You current score is: " + str(score))
```

The final line of the program prints out the current score by accessing the contents of the **global** score variable and casting it to a string data type.

8.1.6 (c) Understand and use **local** and **global variables**



About global variables

- **Variables** declared outside of subroutines (**procedures** and **functions**) are **global variables**.
- **Global variables** exist in memory for the entire time the program is running.
- **Global variables** are accessible from any subroutine, but this is considered bad practice and should be avoided.
- Benefits of **global variables**:
 - Memory usage is consistent and known in advance.
 - No additional execution time for reserving and returning memory.
- Drawbacks of **global variables**:
 - Conflicts with **variables** that share the same name in other parts of the program makes writing programs in teams and testing more difficult.
 - Memory-inefficient because the **variables** exist (and take up space in memory) while the program is in use.
- It's better to pass in copies of **global variables** to use as **parameters** when calling subroutines.

8.1.6 (c) Understand and use **local** and **global variables**

1. Local Variable: Example:

Pseudocode

```
PROCEDURE CalculateArea():  
  DECLARE length, width, area : INTEGER  
  length <- 10  
  width <- 5  
  area <- length * width  
  DISPLAY "Area:", area  
END PROCEDURE
```

Note: In this example, **length**, **width**, and **area** are **local variables** within the **CalculateArea procedure**. They are only accessible within the scope of that **procedure**.

2. Global Variable: Example:

Pseudocode

```
DECLARE total, count : INTEGER  
PROCEDURE UpdateTotal():  
  total <- total + 5  
PROCEDURE DisplayTotal():  
  DISPLAY "Total:", total  
END PROCEDURE
```

Note: In this example, **total** is a **global variable** declared **outside of any procedure**. Both **UpdateTotal** and **DisplayTotal procedures** can access and modify this **global variable**.

prosedure.py - C:/Program Files/Python312/pr

File Edit Format Run Options Window He

```
def calculate_area():  
    length = 10  
    width = 5  
    area = length * width  
    print("Area:", area)
```

```
# Call the procedure  
calculate_area()
```

```
= RESTART:  
Area: 50  
,
```

prosedure.py - C:/Program Files/Python312/pr

File Edit Format Run Options Window Help

```
total = 0 # Global variable
```

```
def update_total():  
    global total # Access the global variable  
    total += 5
```

```
def display_total():  
    print("Total:", total)
```

```
# Call procedures  
update_total()  
display_total()
```

```
=====  
Total: 5  
,
```

8.1.7 Understand and use library routines

Click the link below to watch the video for Library routines

<https://www.youtube.com/watch?v=1EHZ1B59T7s>

8.1 Programming

Library routines

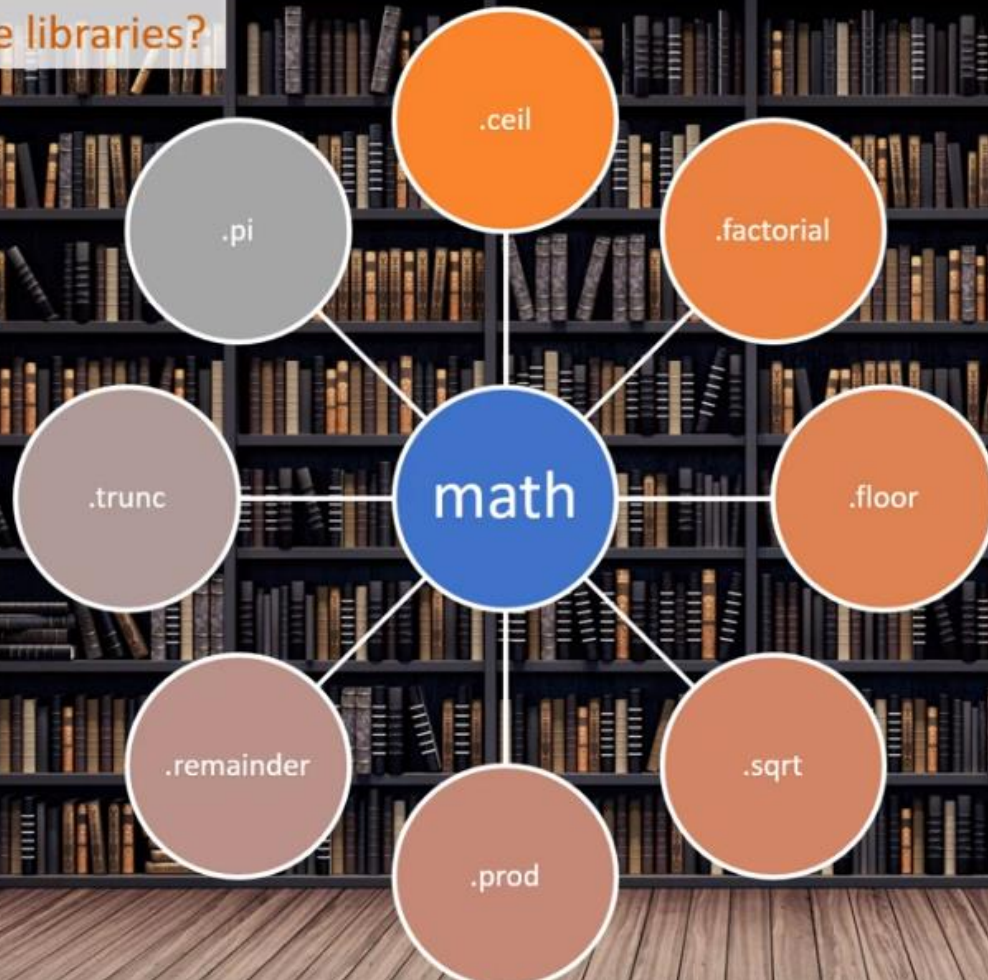
CAMBRIDGE
IGCSE
(0478/0984)



4:25
ave.o

8.1.7 Understand and use library routines

What are libraries?



Routines: Ready-compiled and tested programs that can be run when needed.

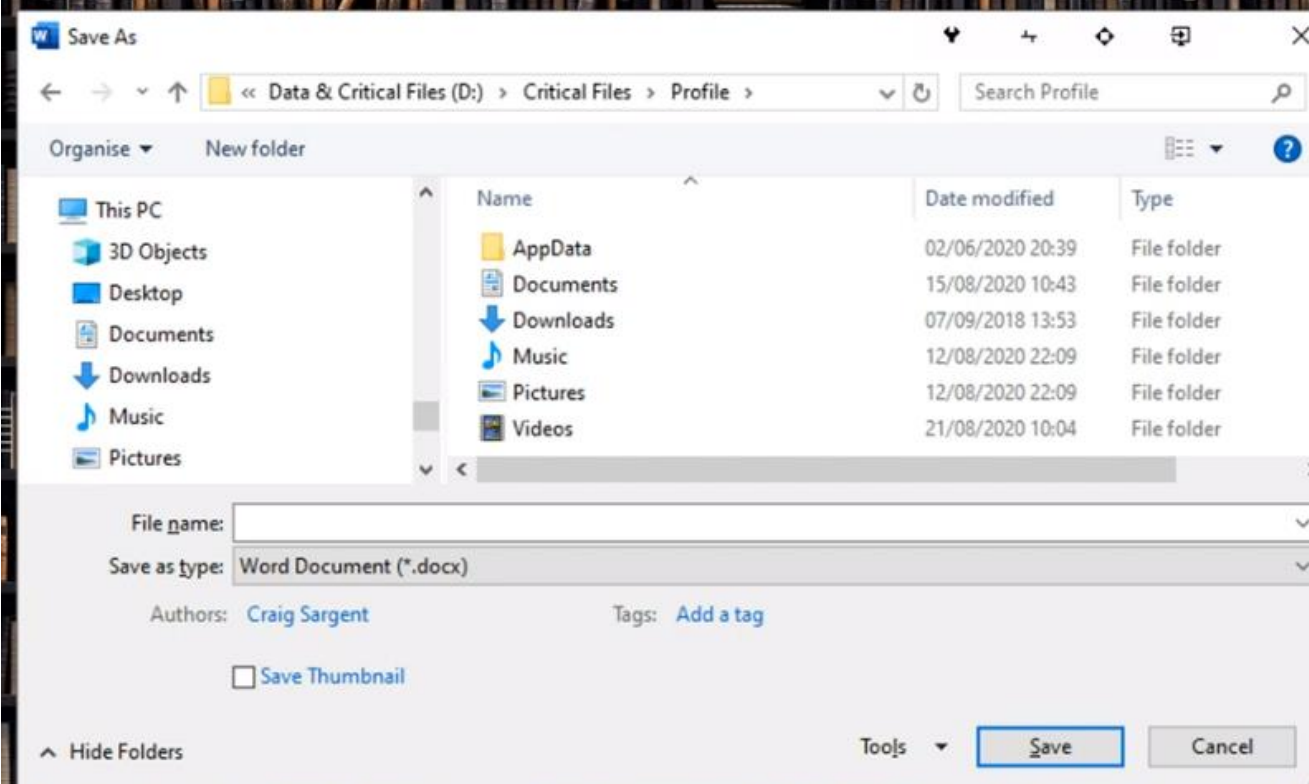
They are typically grouped together into software **libraries**.

Most programming languages have extensive libraries of pre-built **functions**.

For example, the *math* **library** in Python provides common solutions to many problems encountered by programmers who are dealing with numbers.

8.1.7 Understand and use library routines

What are libraries?



Someone programming in the Windows operating system can call **Dynamic Link Libraries (DLL)**.

These libraries contain **subroutines** written to carry out a common task on the Windows operating system – e.g., Save As, where the user needs to save their work as a file.

All the programmer needs to do is call the appropriate DLL **subroutine** with the correct parameters, and the Save As dialogue box will appear.

8.1.7 Understand and use **library routines**

Benefits and drawbacks of using library routines



Quick and easy to use and hook into your own code.



Pre-tested, so you can be relatively sure they are already free from errors.



Pre-compiled, so they are typically optimised to run quickly.



Adding functionality or making specific tweaks can be difficult – or impossible.



Sometimes you are *black-boxed* from the actual implementation.



You have to trust that the developers will continue to maintain the library.

8.1.7 Understand and use **library routines**

Library routines

Studying IGCSE Computer Science, you will need to use the following library routines:

MOD

Returns the remainder of an integer division

DIV

Returns the quotient (the whole number part) of an integer division

ROUND

Returns a value rounded to a specified number of decimal places

RANDOM

Returns a random number

8.1.7 Understand and use **library routines**

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

MOD

```
Value ← MOD(10,3)
```

Returns the remainder of 10 divided by 3.

Value = 1

DIV

```
Value ← DIV(10,3)
```

Returns the quotient of 10 divided by 3.

Value = 3

ROUND

```
Value ← ROUND(3.143278, 2)
```

Returns the value rounded to 2 decimal places.

Value = 3.14

RANDOM

```
Value ← RANDOM()
```

Returns a random number between 0 and 1.

Value = 0.3

8.1.7 Understand and use library routines

Coded examples in Python, Visual Basic and Java

	Examples of MOD, DIV, ROUND, RANDOM	Notes
Python	<pre>Value = 10 % 3 Value = 10 // 3 Value = divmod(10, 3) Value = round(3.143278, 2) import random Value = random()</pre>	Python's MOD uses the % operator. Python's DIV uses the // operator. The function divmod(x,y) provides both answers, the first answer is DIV and the second is MOD. Random needs to be imported from the random library routine.
Visual Basic	<pre>Value = 10 Mod 3 Value = 10 \ 3 Value = Math.Round(3.143278, 2) Value = Rnd()</pre>	Visual Basic's DIV uses the \ operator.
Java	<pre>import java.lang.Math; Value = 10 % 3; Value = 10 / 3; Value = Math.round(3.143278 * 100) / 100.0; import java.util.Random; Random rand = new Random(); double Value = rand.nextDouble();</pre>	Java's MOD uses the % operator. Java's DIV uses the normal division operator. If the numbers being divided are both integers then integer division is automatically performed. Java imports the library routine Math. Math.round only rounds to whole numbers. Random needs to be imported from the random library routine.

8.1.7 Understand and use **library routines**



Library routines

Routines are ready-compiled and tested programs that can be run when needed.

They are typically grouped together into software **libraries**.

Most programming languages have extensive libraries of pre-built **functions**.

Studying IGCSE Computer Science, you will need to use the following library routines:

- **MOD** – Returns the remainder of an integer division.
- **DIV** – Returns the quotient (the whole number part) of an integer division.
- **ROUND** – Returns a value rounded to a specified number of decimal places.
- **RANDOM** – Returns a random number.

8.1.7 Understand and use library routines

Library Routines: Example:

Pseudocode

```
DECLARE number1, number2, result : INTEGER
```

```
number1 <- 15
```

```
number2 <- 7
```

```
result <- MOD(number1, number2)
```

```
DISPLAY "MOD Result:", result
```

```
result <- DIV(number1, number2)
```

```
DISPLAY "DIV Result:", result
```

```
DECLARE decimalNumber : REAL
```

```
decimalNumber <- 7.8
```

```
result <- ROUND(decimalNumber)
```

```
DISPLAY "ROUND Result:", result
```

```
result <- RANDOM(10)
```

Generate a random number between 0 and 10

```
DISPLAY "RANDOM Result:", result
```

- MOD: Computes the remainder of the division of two numbers.
- DIV: Computes the quotient of the division of two numbers.
- ROUND: Rounds a decimal number to the nearest whole number.
- RANDOM: Generates a random integer within a specified range.

8.1.7 Understand and use library routines

Library Routines: Example:

prosedure.py - C:/Program Files/Python312/prosedure.py (3.12.0)

File Edit Format Run Options Window Help

```
number1 = 15
number2 = 7

mod_result = number1 % number2
print("MOD Result:", mod_result)

div_result = number1 // number2
print("DIV Result:", div_result)

decimal_number = 7.8
round_result = round(decimal_number)
print("ROUND Result:", round_result)

import random

random_result = random.randint(0, 10) # Generate a random number between 0 and 10
print("RANDOM Result:", random_result)
```

===== RES
MOD Result: 1
DIV Result: 2
ROUND Result: 8
RANDOM Result: 1
|

- **MOD**: Computes the remainder of the division of two numbers.
- **DIV**: Computes the quotient of the division of two numbers.
- **ROUND**: Rounds a decimal number to the nearest whole number.
- **RANDOM**: Generates a random integer within a specified range.

8.1.7 Understand how to **create a maintainable program**

Click the link below to watch the video for Maintainability

<https://www.youtube.com/watch?v=QGH4N3VfZpA>

8.1 Programming

Maintainability

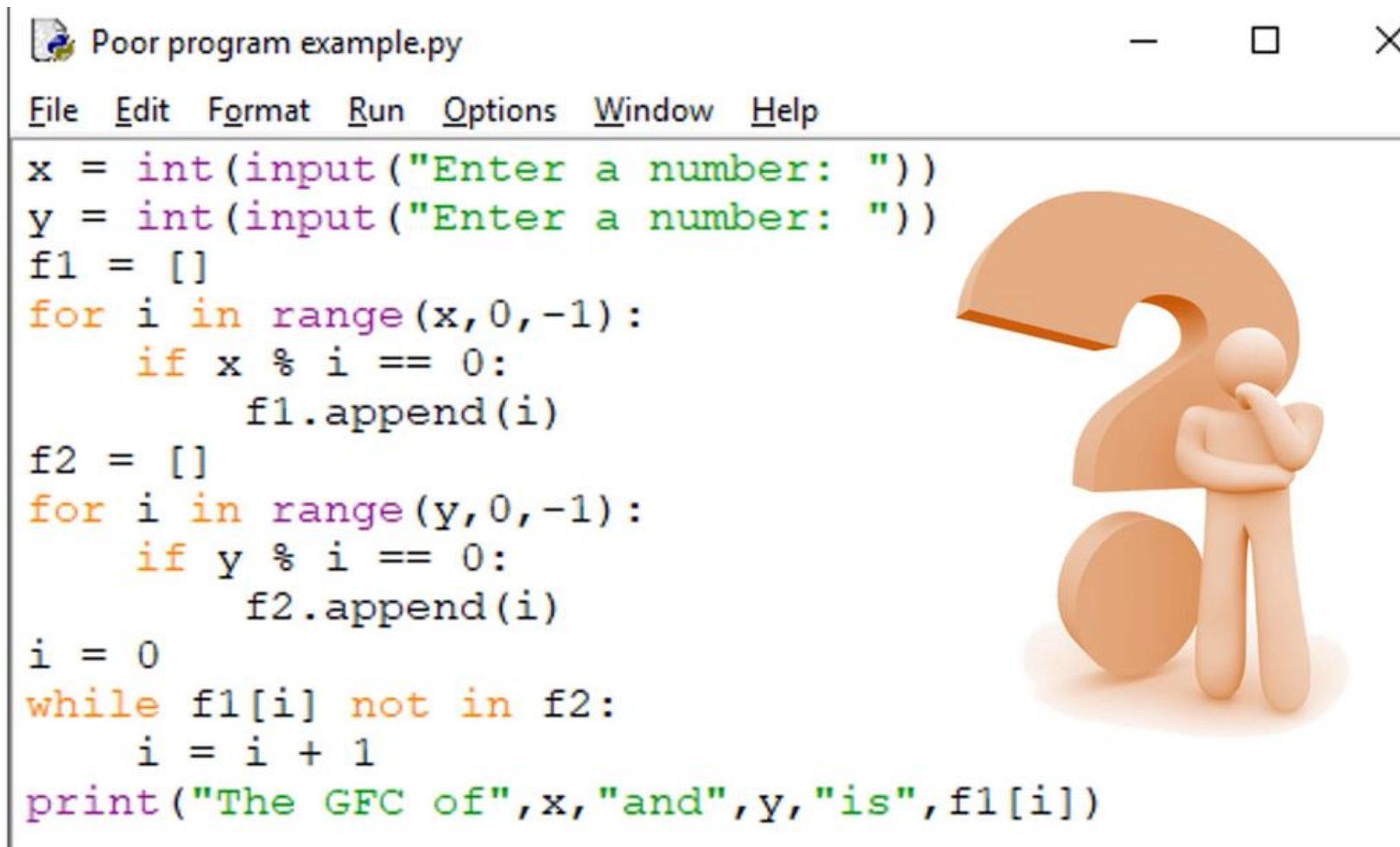
CAMBRIDGE
IGCSE
(0478/0984)



3:46
ave.c

8.1.7 Understand how to create a maintainable program

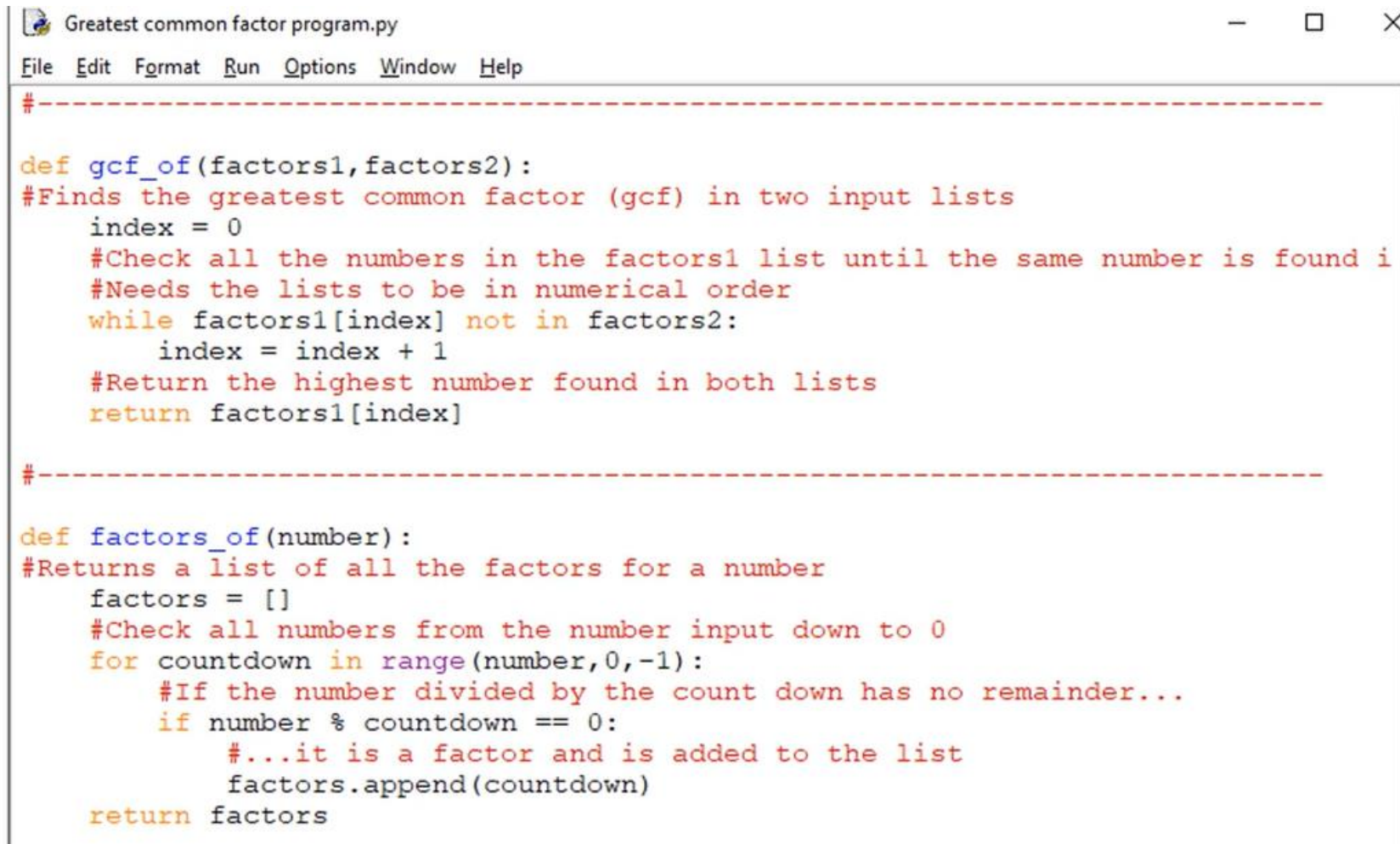
A program
that is not
well-
maintained



```
File Edit Format Run Options Window Help
x = int(input("Enter a number: "))
y = int(input("Enter a number: "))
f1 = []
for i in range(x, 0, -1):
    if x % i == 0:
        f1.append(i)
f2 = []
for i in range(y, 0, -1):
    if y % i == 0:
        f2.append(i)
i = 0
while f1[i] not in f2:
    i = i + 1
print("The GFC of", x, "and", y, "is", f1[i])
```

8.1.7 Understand how to create a maintainable program

A well-maintained program



```
Greatest common factor program.py
File Edit Format Run Options Window Help

#-----

def gcf_of(factors1,factors2):
    #Finds the greatest common factor (gcf) in two input lists
    index = 0
    #Check all the numbers in the factors1 list until the same number is found i
    #Needs the lists to be in numerical order
    while factors1[index] not in factors2:
        index = index + 1
    #Return the highest number found in both lists
    return factors1[index]

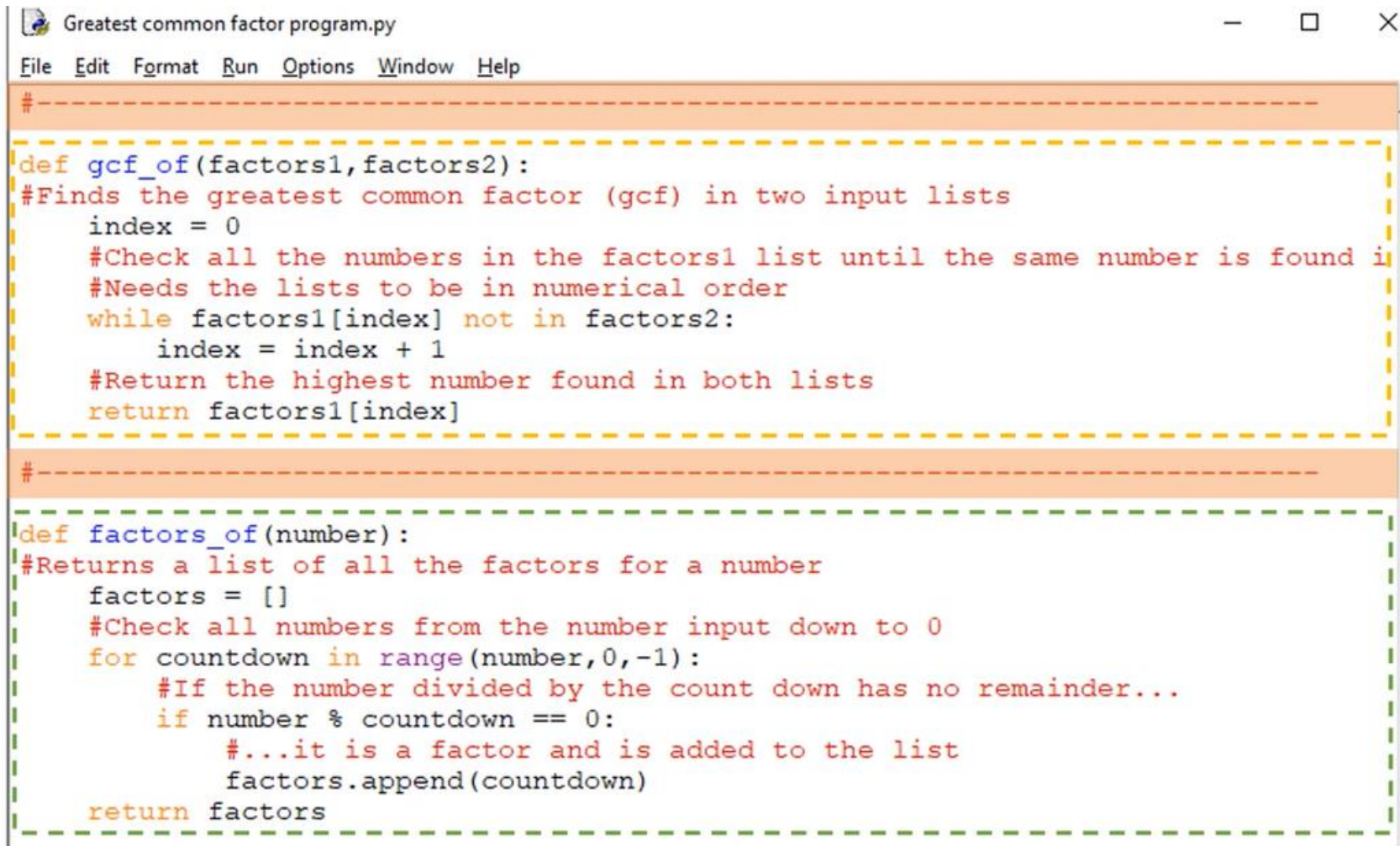
#-----

def factors_of(number):
    #Returns a list of all the factors for a number
    factors = []
    #Check all numbers from the number input down to 0
    for countdown in range(number,0,-1):
        #If the number divided by the count down has no remainder...
        if number % countdown == 0:
            #...it is a factor and is added to the list
            factors.append(countdown)
    return factors
```

8.1.7 Understand how to create a maintainable program

A well-maintained program

Using comments to divide the program into distinct sections.



```

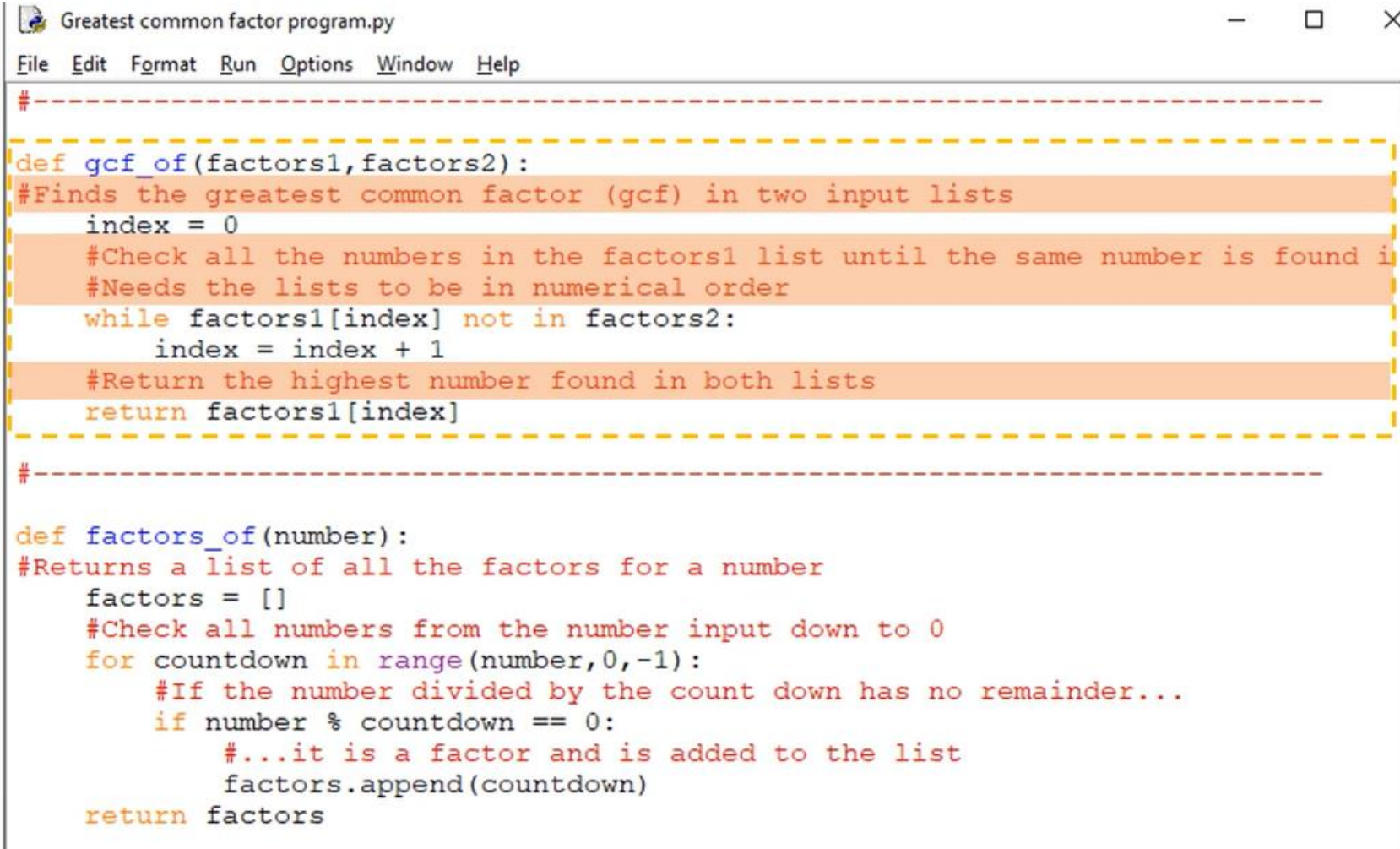
Greatest common factor program.py
File Edit Format Run Options Window Help
#-----
def gcf_of(factors1,factors2):
    #Finds the greatest common factor (gcf) in two input lists
    index = 0
    #Check all the numbers in the factors1 list until the same number is found in factors2
    #Needs the lists to be in numerical order
    while factors1[index] not in factors2:
        index = index + 1
    #Return the highest number found in both lists
    return factors1[index]
#-----
def factors_of(number):
    #Returns a list of all the factors for a number
    factors = []
    #Check all numbers from the number input down to 0
    for countdown in range(number,0,-1):
        #If the number divided by the count down has no remainder...
        if number % countdown == 0:
            #...it is a factor and is added to the list
            factors.append(countdown)
    return factors

```


8.1.7 Understand how to create a maintainable program

A well-maintained program

Using comments to explain what various parts of the program are designed to do.



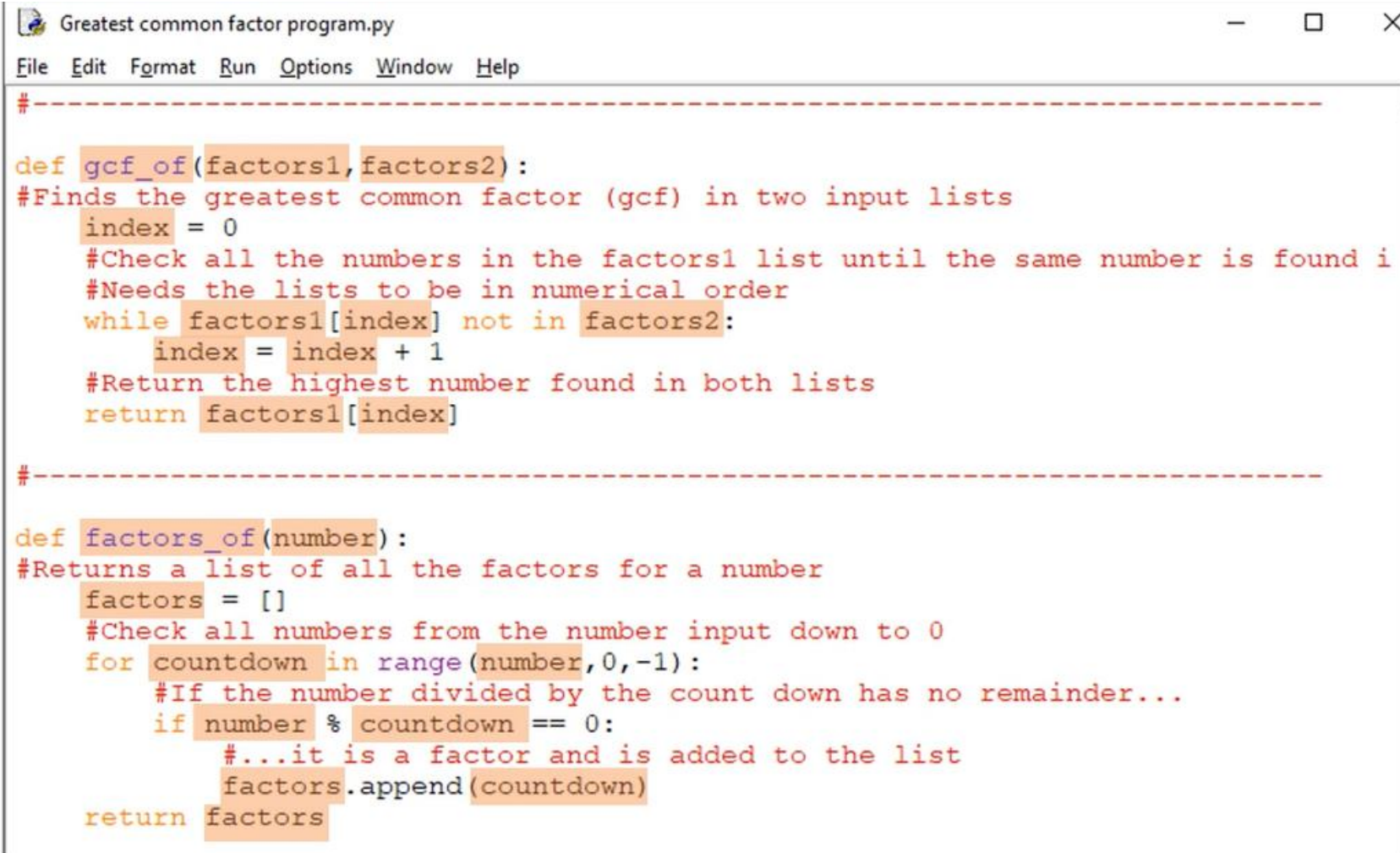
```
File Edit Format Run Options Window Help
#-----
def gcf_of(factors1,factors2):
    #Finds the greatest common factor (gcf) in two input lists
    index = 0
    #Check all the numbers in the factors1 list until the same number is found i
    #Needs the lists to be in numerical order
    while factors1[index] not in factors2:
        index = index + 1
    #Return the highest number found in both lists
    return factors1[index]
#-----

def factors_of(number):
    #Returns a list of all the factors for a number
    factors = []
    #Check all numbers from the number input down to 0
    for countdown in range(number,0,-1):
        #If the number divided by the count down has no remainder...
        if number % countdown == 0:
            #...it is a factor and is added to the list
            factors.append(countdown)
    return factors
```


8.1.7 Understand how to create a maintainable program

A well-maintained program

Using sensible, descriptive identifier names.



```
-----
def gcf_of(factors1, factors2):
    #Finds the greatest common factor (gcf) in two input lists
    index = 0
    #Check all the numbers in the factors1 list until the same number is found i
    #Needs the lists to be in numerical order
    while factors1[index] not in factors2:
        index = index + 1
    #Return the highest number found in both lists
    return factors1[index]

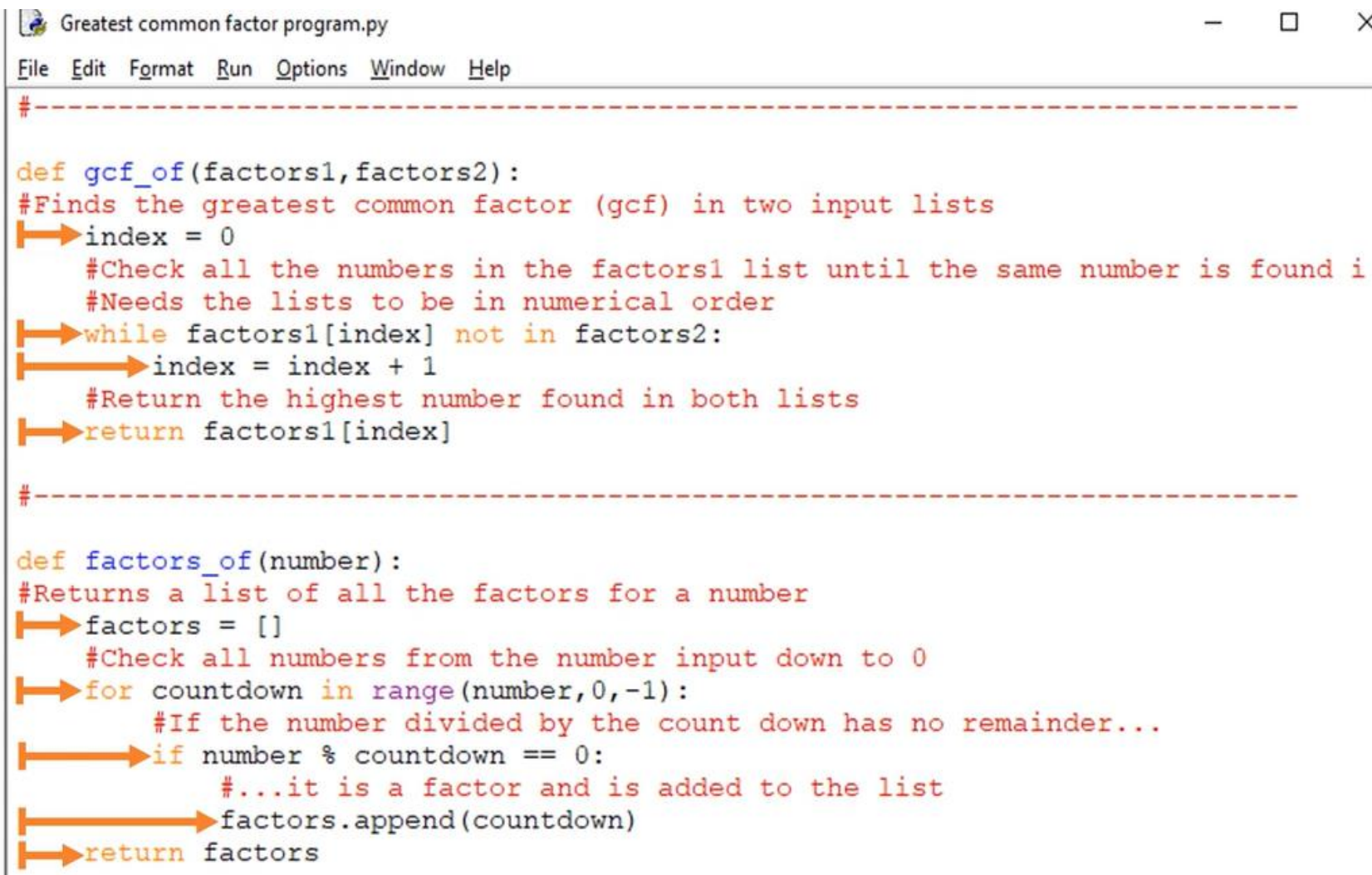
-----

def factors_of(number):
    #Returns a list of all the factors for a number
    factors = []
    #Check all numbers from the number input down to 0
    for countdown in range(number, 0, -1):
        #If the number divided by the count down has no remainder...
        if number % countdown == 0:
            #...it is a factor and is added to the list
            factors.append(countdown)
    return factors
```

8.1.7 Understand how to create a maintainable program

A well-maintained program

Using indentation.



```
-----
def gcf_of(factors1,factors2):
#Finds the greatest common factor (gcf) in two input lists
    index = 0
    #Check all the numbers in the factors1 list until the same number is found i
    #Needs the lists to be in numerical order
    while factors1[index] not in factors2:
        index = index + 1
    #Return the highest number found in both lists
    return factors1[index]

-----

def factors_of(number):
#Returns a list of all the factors for a number
    factors = []
    #Check all numbers from the number input down to 0
    for countdown in range(number,0,-1):
        #If the number divided by the count down has no remainder...
        if number % countdown == 0:
            #...it is a factor and is added to the list
            factors.append(countdown)
    return factors
```

8.1.7 Understand how to **create a maintainable program**



Writing maintainable code

To make your code as easy to read and maintain as possible, make sure you use:

- Comments to divide the program into sections and explain:
 - The program's purpose.
 - Sections of code – typically, selections, iterations and procedures.
 - Any unusual approaches taken.
- White space to make program sections easier to see.
- Indentation for every selection and iteration branch.
- Descriptive variable names, explaining each variable's purpose with a comment when it is declared.
- Procedures and/or functions to:
 - Structure code.
 - Eliminate duplicate code.
- Constants – declared at the top of the program.

8.1.7 Understand how to create a maintainable program

Example of : Maintainable Program

Pseudocode:

```
ARRAY numbers[MAX_COUNT]
PROCEDURE initializeNumbers()
    FOR i FROM 1 TO MAX_COUNT
        numbers[i] <- 0
    END FOR
END PROCEDURE
FUNCTION calculateSum(startIndex, endIndex)
    DECLARE sum : INTEGER
    sum <- 0
    FOR i FROM startIndex TO endIndex
        sum <- sum + numbers[i]
    END FOR
    RETURN sum
END FUNCTION
```

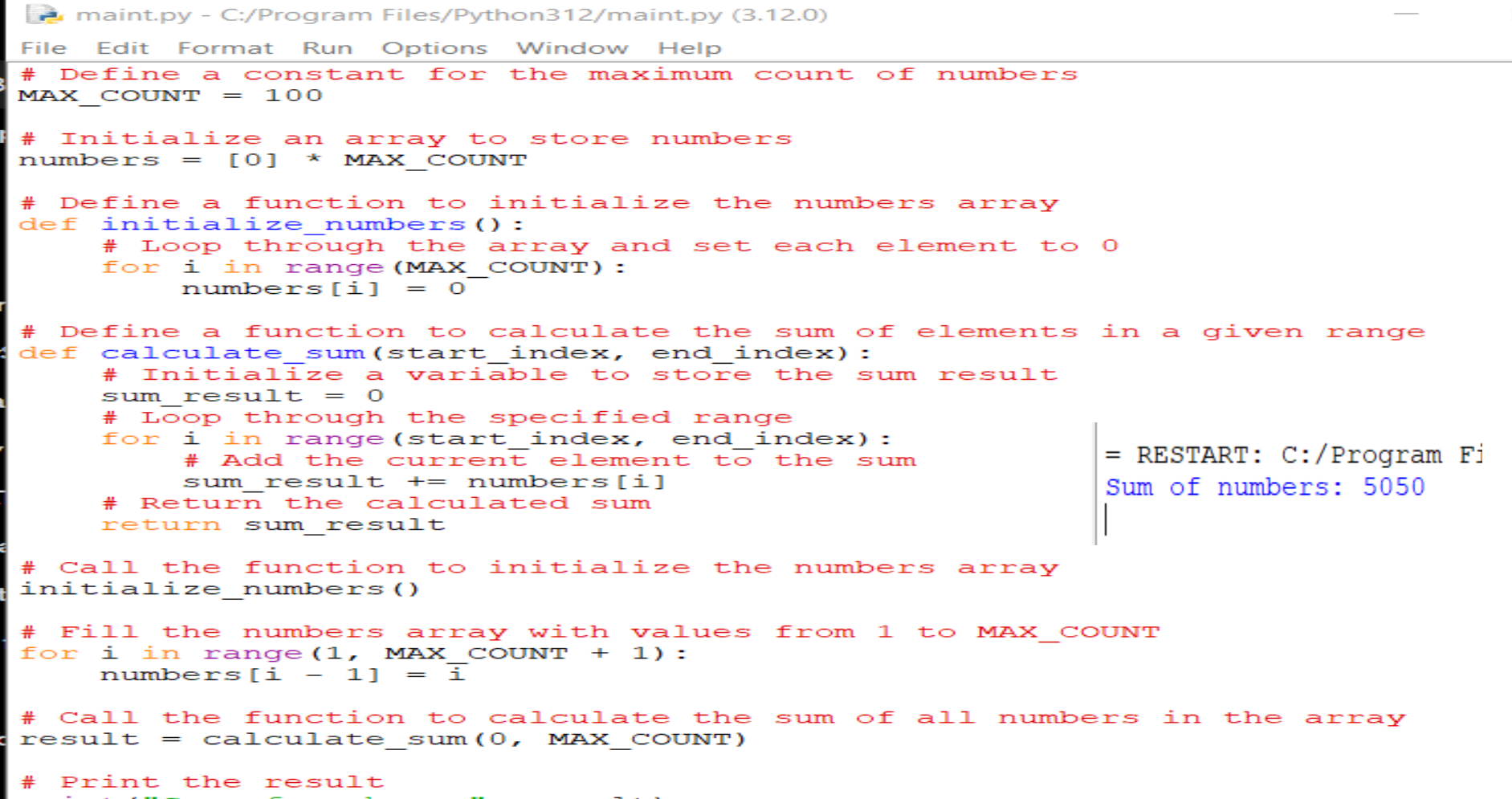
COMMENT This program initializes an array and calculates the sum of its elements.

```
initializeNumbers()
FOR i FROM 1 TO MAX_COUNT
    numbers[i] <- i
END FOR
DECLARE result : INTEGER
result <- calculateSum(1, MAX_COUNT)
OUTPUT "Sum of numbers:", result
```

8.1.7 Understand how to create a maintainable program

Example of : Maintainable Program

Pseudocode:



```
maint.py - C:/Program Files/Python312/maint.py (3.12.0)
File Edit Format Run Options Window Help
# Define a constant for the maximum count of numbers
MAX_COUNT = 100

# Initialize an array to store numbers
numbers = [0] * MAX_COUNT

# Define a function to initialize the numbers array
def initialize_numbers():
    # Loop through the array and set each element to 0
    for i in range(MAX_COUNT):
        numbers[i] = 0

# Define a function to calculate the sum of elements in a given range
def calculate_sum(start_index, end_index):
    # Initialize a variable to store the sum result
    sum_result = 0
    # Loop through the specified range
    for i in range(start_index, end_index):
        # Add the current element to the sum
        sum_result += numbers[i]
    # Return the calculated sum
    return sum_result

# Call the function to initialize the numbers array
initialize_numbers()

# Fill the numbers array with values from 1 to MAX_COUNT
for i in range(1, MAX_COUNT + 1):
    numbers[i - 1] = i

# Call the function to calculate the sum of all numbers in the array
result = calculate_sum(0, MAX_COUNT)

# Print the result
print("Sum of numbers:", result)
```

= RESTART: C:/Program Fi
Sum of numbers: 5050

8.1 Programming concepts

Past Paper 2 exams Question and Answers:

Question 1:

Explain the purpose of the library routines `DIV` and `ROUND`

`DIV`
.....
.....
.....

`ROUND`
.....
.....
.....

[4]

8.1 Programming concepts

Answer 1:

PUBLISHED

Question	Answer	Marks
4	One mark per mark point, max four DIV, max two • To perform integer division • Meaning only the whole number part of the answer is retained • Example of DIV For example <code>DIV(9,4) = 2</code> ROUND, max two • To return a value rounded to a specified number of digits / decimal places • The result will either be rounded to the next highest or the next lowest value • ... depending on whether the value of the preceding digit is ≥ 5 or < 5 • Example of ROUND for example, <code>ROUND(4.56, 1) = 4.6</code>	4

8.1 Programming concepts

Question 2:

An algorithm has been written in pseudocode to calculate a check digit for a four-digit number. The algorithm then outputs the five-digit number including the check digit. The algorithm stops when -1 is input as the fourth digit.

```
01 Flag ← FALSE
02 REPEAT
03     Total ← 0
04     FOR Counter ← 1 TO 4
05         OUTPUT "Enter a digit ", Counter
06         INPUT Number[Counter]
07         Total ← Total + Number * Counter
08         IF Number[Counter] = 0
09             THEN
10                 Flag ← TRUE
11             ENDIF
12     NEXT Counter
13     IF NOT Flag
14         THEN
15             Number[5] ← MOD(Total, 10)
16             FOR Counter ← 0 TO 5
17                 OUTPUT Number[Counter]
18             NEXT
19         ENDIF
20 UNTIL Flag
```

(a) Give the line number(s) for the statements showing:

Totalling

Count-controlled loop

Post-condition loop

[3]

8.1 Programming concepts

Question 2:

(b) Identify the **three** errors in the pseudocode and suggest a correction for each error.

Error 1

Correction

.....

Error 2

Correction

.....

Error 3

Correction

.....

[3]

8.1 Programming concepts

Question 2:

- (c) The algorithm does **not** check that each input is a single digit.
Identify the place in the algorithm where this check should occur.
Write pseudocode for this check.
Your pseudocode must make sure that the input is a single digit and checks for –1

Place in algorithm

Pseudocode

.....

.....

.....

.....

.....

[4]

8.1 Programming concepts

Answer 2:

Question	Answer	Marks
7(a)	• 07 • 04/12 or 16/18 • 02/20	3
7(b)	One mark for each error identified and correction • Line 07 <code>Total ← Total + Number * Counter</code> should be <code>Total ← Total + Number[Counter] * Counter</code> • Line 08 <code>IF Number[Counter] = 0</code> should be <code>IF Number[Counter] = -1 // should be IF Number[Counter] < 0</code> • Line 16 <code>FOR Counter ← 0 TO 5</code> should be <code>FOR Counter ← 1 TO 5</code>	3
7(c)	One mark for place in algorithm (max one) • around lines 05 and 06 • line 07 • (immediately) after the input of the number Three marks pseudocode One mark for each point (max three) • Use of <code>REPEAT ... UNTIL</code> // any working loop structure • check for <code>>0 // >=0</code> • check for <code><10 // >9</code> • check for whole number • check for -1 • check for length of digit <code><> 1</code> Example <pre>REPEAT OUTPUT "Enter a digit " INPUT Number[Counter] UNTIL Number[Counter] = Round(Number[Counter],0) AND ((Number[Counter] = -1) OR (Number[Counter] > 0 AND Number[Counter] < 10))</pre>	4

8.1 Programming concepts

Question 3:

11 The variables `P` and `Q` are used to store data in a program. `P` stores a string. `Q` stores a character.

(a) Write pseudocode statements to declare the variables `P` and `Q`, store "The world" in `P` and store 'W' in `Q`

.....

.....

.....

..... [2]

8.1 Programming concepts

Question 3:

- (b) Write a pseudocode algorithm to:
- convert `P` to upper case
 - find the position of `Q` in the string `P` (the first character in this string is in position 1)
 - store the position of `Q` in the variable `Position`
- [4]
- (c) Give the value of `Position` after the algorithm has been executed with the data in question 11(a).
- [1]

8.1 Programming concepts

Answer 3:

Question	Answer	Marks
11(a)	One mark for any two correct lines <pre>DECLARE P : STRING P ← "The world" DECLARE Q : CHAR Q ← 'W'</pre>	2
11(b)	One mark for each point (max four) • converting P to upper case • finding the length of P • using a loop to check for position of Q • using the string operation substring • storing the loop counter in Position if the value is found For example: <pre>P ← UCASE(P) Counter ← 1 Position ← 0 REPEAT IF SUBSTRING(P, Counter, 1) = Q THEN Position ← Counter ENDIF Counter ← Counter + 1 UNTIL Position <> 0 OR Counter = LENGTH(P)</pre>	4
11(c)	5	1

8.1 Programming concepts

Question 4:

1 Tick (✓) **one** box to complete the sentence.

A constant

- A stores a value that can change at any time during the execution of a program.
- B stores a value that cannot change during the execution of a program.
- C stores values of multiple data types.
- D stores values that must be of the same data type.

☐

☐

☐

☐

[1]

2 Explain the purpose of the library routines MOD and RANDOM

MOD

.....

.....

.....

.....

RANDOM

.....

.....

.....

[4]

8.1 Programming concepts

Answer 4:

Question	Answer	Marks
1	B	1

Question	Answer	Marks
2	One mark per mark point, max four MOD, max two • To perform (integer) division when one number is divided by another • ... and find the remainder • Allow example e.g. $7 \text{ MOD } 2 = 1$ RANDOM, max two • To generate (pseudo) random numbers • ...(usually) within a specified range • Allow example e.g. <code>RANDOM() * 10</code> returns a random number between 0 and 10	4

8.1 Programming concepts

Question 5:

3 Describe what happens when a function is called during the execution of a program.

.....

.....

.....

.....

.....

..... [3]

8.1 Programming concepts

Answer 5:

Question	Answer	Marks
3	One mark per mark point, max three MP1 A call statement is used in order to make use of a function // the function is called using its identifier MP2 Parameters are / may be passed (from the main program) to the function (to be used within the function) MP3 The function performs its task ... MP4 ... and returns a value / values to the main program	3

8.1 Programming concepts

Question 6:

- 2 Tick (✓) **one** box in each row to identify the most appropriate data type for each description. Only **one** tick (✓) per column.

Description	Data type				
	Boolean	Char	Integer	Real	String
a single character from the keyboard					
multiple characters from the keyboard					
only one of two possible values					
only whole numbers					
any number					

[4]

8.1 Programming concepts

Answer 6:

2

Four marks for five correct rows
Three marks for four correct rows
Two marks for three correct rows
One marks for two correct rows

Description	Data type				
	Boolean	Char	Integer	Real	String
a single character from the keyboard		✓			
multiple characters from the keyboard					✓
only one of two possible values	✓				
only whole numbers			✓		
any number				✓	

4

8.1 Programming concepts

Question 7:

2 Tick (✓) **one** or more boxes in each row to match the type(s) of test data to each description.

Description	Types of test data			
	Boundary	Erroneous / Abnormal	Extreme	Normal
test data that is always on the limit of acceptability				
test data that is either on the limit of acceptability or test data that is just outside the limit of acceptability				
test data that will always be rejected				
test data that is within the limits of acceptability				

[4]

8.1 Programming concepts

Answer 7:

2

One mark per row, max four

4

Description	Types of test data			
	Boundary	Erroneous / Abnormal	Extreme	Normal
test data that is always on the limit of acceptability			✓	
test data that is either on the limit of acceptability or test data that is just outside the limit of acceptability	✓			
test data that will always be rejected		✓		
test data that is within the limits of acceptability			✓	✓

8.1 Programming concepts

Question 8:

- 3 Describe what is meant by the terms variable and constant and give an example of each in your answer.

Variable

.....

.....

.....

Constant

.....

.....

.....

[4]

8.1 Programming concepts

Answer 8:

Question	Answer	Marks
3	One mark per mark point, max four • variables are used to represent values that can change during the execution of a program // variables can be used to store the results of calculations / counting / totalling // can store values entered by the user • variable example – any data that is input into a program such as a date • constants represent values that must stay the same throughout the execution of a program • constant example – any value that does not change, such as Pi in mathematical formulae	4

8.1 Programming concepts

Question 9:

2 Draw a line to connect each programming concept to the most appropriate description.

Programming concept

counting

repetition

selection

sequence

totalling

Description

carrying out an action multiple times within a loop structure

adding together the numbers in a list of numbers

tracking the number of iterations a program has performed in a loop

branching off to take a course of action depending on the answer to a question

a set of statements to be executed in order

[4]

8.1 Programming concepts

Answer 9:

QUESTIONED		
Question	Answer	Marks
Section B		
2	One mark for each correct line, max four Programming concept Description counting repetition selection sequence totalling carrying out an action multiple times within a loop structure adding together the numbers in a list of numbers tracking the number of iterations a program has performed in a loop branching off to take a course of action depending on the answer to a question a set of statements to be executed in order	4

8.1 Programming concepts

Question 10:

State **three** different features of a high-level programming language that a programmer could use to make sure that their program will be easier to understand by another programmer.
Give an example for each feature.

Feature 1
.....

Example
.....

Feature 2
.....

Example
.....

Feature 3
.....

Example
.....

[6]

8.1 Programming concepts

Answer 10:

PUBLISHED		
Question	Answer	Marks
6	One mark for each feature and one mark for corresponding example (max six) • ensuring that all identifiers have meaningful names ... • ... example using <code>Total</code> to store a running total • using comments to explain how the program works ... • ... example <code>// all values are zeroed before the next calculation</code> • using procedures and functions for the tasks within a program ... • ... example <code>CalculateInterest(Deposit, Rate)</code>	6

8.1 Programming concepts

Question 11:

An algorithm has been written in pseudocode to calculate a check digit for a four-digit number. The algorithm then outputs the five-digit number including the check digit. The algorithm stops when -1 is input as the fourth digit.

```
01 Flag ← FALSE
02 REPEAT
03     Total ← 0
04     FOR Counter ← 1 TO 4
05         OUTPUT "Enter a digit ", Counter
06         INPUT Number[Counter]
07         Total ← Total + Number * Counter
08         IF Number[Counter] = 0
09             THEN
10                 Flag ← TRUE
11         ENDIF
12     NEXT Counter
13     IF NOT Flag
14         THEN
15             Number[5] ← MOD(Total, 10)
16             FOR Counter ← 0 TO 5
17                 OUTPUT Number[Counter]
18             NEXT
19         ENDIF
20 UNTIL Flag
```

(a) Give the line number(s) for the statements showing:

Totalling

Count-controlled loop

Post-condition loop

[3]

8.1 Programming concepts

Question 11:

(b) Identify the **three** errors in the pseudocode and suggest a correction for each error.

Error 1

Correction

.....

Error 2

Correction

.....

Error 3

Correction

.....

[3]

8.1 Programming concepts

Question 11:

- (c) The algorithm does **not** check that each input is a single digit.
Identify the place in the algorithm where this check should occur.
Write pseudocode for this check.
Your pseudocode must make sure that the input is a single digit and checks for –1

Place in algorithm

Pseudocode

.....

.....

.....

.....

.....

[4]

8.1 Programming concepts

Answer 11:

Question	Answer	Marks
7(a)	• 07 • 04/12 or 16/18 • 02/20	3
7(b)	One mark for each error identified and correction • Line 07 <code>Total ← Total + Number * Counter</code> should be <code>Total ← Total + Number[Counter] * Counter</code> • Line 08 <code>IF Number[Counter] = 0</code> should be <code>IF Number[Counter] = -1 // should be IF Number[Counter] < 0</code> • Line 16 <code>FOR Counter ← 0 TO 5</code> should be <code>FOR Counter ← 1 TO 5</code>	3

Question	Answer	Marks
7(c)	One mark for place in algorithm (max one) • around lines 05 and 06 • line 07 • (immediately) after the input of the number Three marks pseudocode One mark for each point (max three) • Use of <code>REPEAT ... UNTIL</code> // any working loop structure • check for <code>>0 // >=0</code> • check for <code><10 // >9</code> • check for whole number • check for -1 • check for length of digit <code><> 1</code> Example <pre>REPEAT OUTPUT "Enter a digit " INPUT Number[Counter] UNTIL Number[Counter] = Round(Number[Counter],0) AND ((Number[Counter] = -1) OR (Number[Counter] > 0 AND Number[Counter] < 10))</pre>	4

8.1 Programming concepts

Question 12:

11 The variables `P` and `Q` are used to store data in a program. `P` stores a string. `Q` stores a character.

(a) Write pseudocode statements to declare the variables `P` and `Q`, store "The world" in `P` and store 'w' in `Q`

.....

.....

.....

..... [2]

(b) Write a pseudocode algorithm to:

- convert `P` to upper case
- find the position of `Q` in the string `P` (the first character in this string is in position 1)
- store the position of `Q` in the variable `Position`

.....

.....

.....

(c) Give the value of `Position` after the algorithm has been executed with the data in question 11(a).

.....

..... [1]

8.1 Programming concepts

Answer 12:

Question	Answer	Marks
11(a)	One mark for any two correct lines <pre> DECLARE P : STRING P ← "The world" DECLARE Q : CHAR Q ← 'W' </pre>	2
PUBLISHED		
Question	Answer	Marks
11(b)	One mark for each point (max four) • converting P to upper case • finding the length of P • using a loop to check for position of Q • using the string operation substring • storing the loop counter in Position if the value is found For example: <pre> P ← UCASE(P) Counter ← 1 Position ← 0 REPEAT IF SUBSTRING(P, Counter, 1) = Q THEN Position ← Counter ENDIF Counter ← Counter + 1 UNTIL Position <> 0 OR Counter = LENGTH(P) </pre>	4
11(c)	5	1

8.1 Programming concepts

Question 13:

9 The variable `Saying` is used to store string data in a program.

(a) Write the pseudocode statement to declare the variable `Saying`

.....
..... [1]

(b) Write the pseudocode statements to:

- allow a string to be input to the variable `Saying`
- store the content of the variable `Saying` in a text file named `"Quotations.txt"`
- make sure the text file is closed at the end of the algorithm.

.....
.....
.....
..... [5]

8.1 Programming concepts

Answer 13:

Question	Answer	Marks
9(a)	<code>DECLARE Saying : STRING</code>	1

Question	Answer	Marks
9(b)	One mark per mark point, max five MP1 input a string into <code>Saying</code> MP2 correct use of <code>OPENFILE</code> to write data MP3 correct use of <code>WRITEFILE</code> to write <code>Saying</code> MP4 correct use of <code>CLOSEFILE</code> MP5 correct use of filename <code>Quotations.txt</code> throughout For example: <pre>INPUT Saying OPENFILE "Quotations.txt" FOR WRITE WRITEFILE "Quotations.txt", Saying CLOSEFILE "Quotations.txt"</pre>	5

8.1 Programming concepts

Question 14:

6 State **two** features that should be included to create a maintainable program.

Give a reason why each feature should be used.

1

.....

.....

.....

2

.....

.....

.....

[4]

8.1 Programming concepts

Answer 14:

Question	Answer	Marks
6	One mark for each correct feature, max two One mark for each correct accompanying reason, max two For example: Meaningful identifiers – to enable the programmer (or future programmers) to easily recognize the purpose of a variable / array / constant // to enable easy tracking of a variable / constant / array through the program Use of comments – to annotate each section of a program so that a programmer can find specific sections / so that the programmer knows the purpose of that section of code Procedures and functions – to make programs modular and easier to update / add functionality	4

8.2 Arrays

Candidates should be able to:

- 1 Declare and use one-dimensional (1D) and two-dimensional (2D) arrays
- 2 Understand the use of arrays
- 3 Write values into and read values from an array using iteration

Notes and guidance

- Including the use of variables as indexes in arrays
- The first index can be zero or one
- Including nested iteration

8.2 Arrays

Click the link below to watch the video for the use of arrays

<https://www.youtube.com/watch?v=avBbuh1RiZw>

8.2 Arrays

The use of arrays

CAMBRIDGE
IGCSE
(0478/0984)



8.2 Arrays

What is an array?

An array can be thought of as a variable that can contain more than one data item – e.g., storing a list of names.

We can do that by allocating a contiguous part of memory for the purpose of storing that data.

Contiguous means all the data is stored together, one element after the other.

Note: Lists are different to arrays because lists are not contiguous.

This program will know the position in memory where the array starts – in this case, address 05.

The program uses an index relative to this start point to allow us to easily access the array's contents.

So, "Jane" is at index 2.

RANDOM ACCESS MEMORY	ADDRESS	INSTRUCTION/DATA	
	01		
	02		
	03		
	04		index
	05	Boris	[0]
	06	Marek	[1]
	07	Jane	[2]
	08	Ryan	[3]
	09		
	10		
	11		
	12		

8.2 Arrays

One-dimensional arrays (Python)

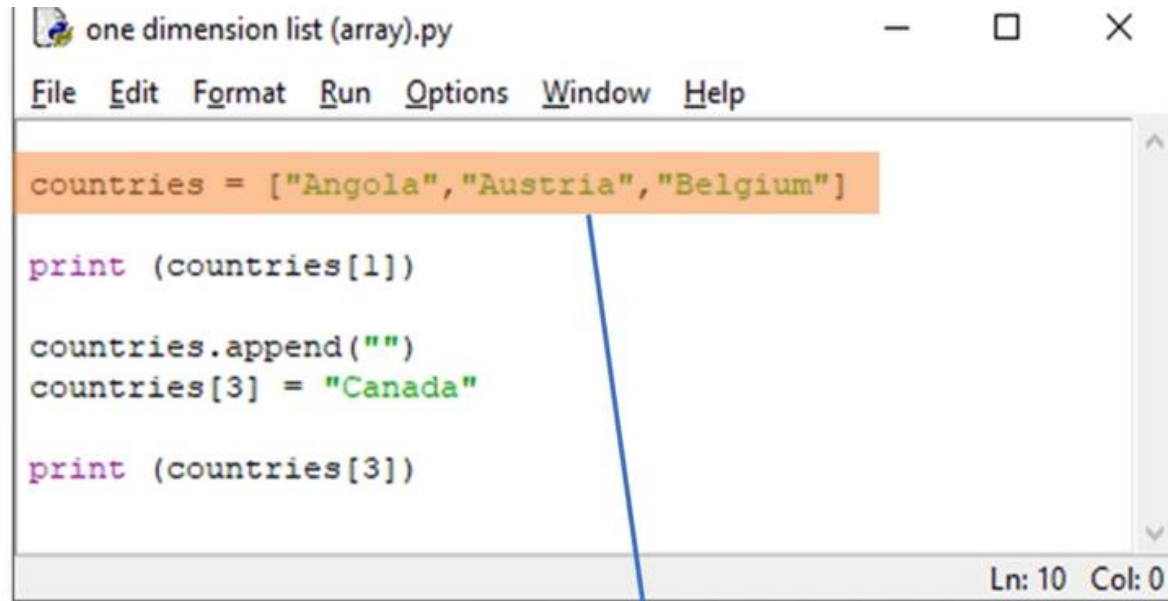
This line sets up a one-dimensional array of strings called “countries”.

It assigns the initial values of:

- “Angola” at index 0
- “Austria” at index 1
- “Belgium” at index 2

Note how indexes start at 0 and not 1.

Python actually uses arrays but allows programmers to use them as lists. That’s why most people think Python only supports lists.



```
one dimension list (array).py
File Edit Format Run Options Window Help

countries = ["Angola", "Austria", "Belgium"]

print (countries[1])

countries.append("")
countries[3] = "Canada"

print (countries[3])

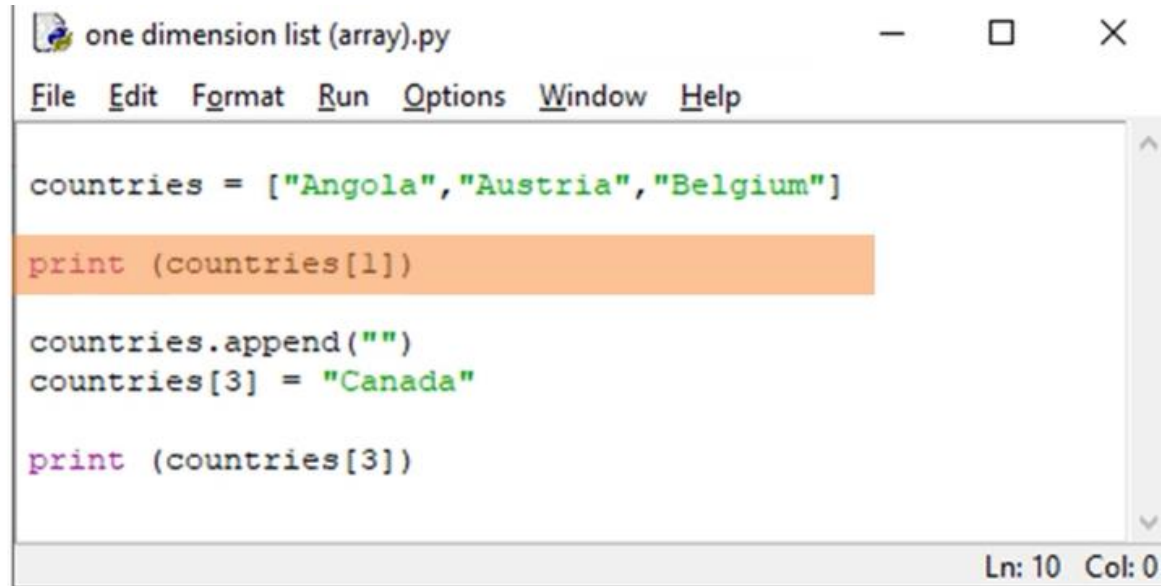
Ln: 10 Col: 0
```

countries	
Index	Contents
0	Angola
1	Austria
2	Belgium

8.2 Arrays

One-dimensional arrays (Python)

To output "Austria" from the array, we would need to use index 1, as index 1 represents the second item in the "countries" array.



```
one dimension list (array).py
File Edit Format Run Options Window Help

countries = ["Angola", "Austria", "Belgium"]
print (countries[1])

countries.append("")
countries[3] = "Canada"

print (countries[3])

Ln: 10 Col: 0
```

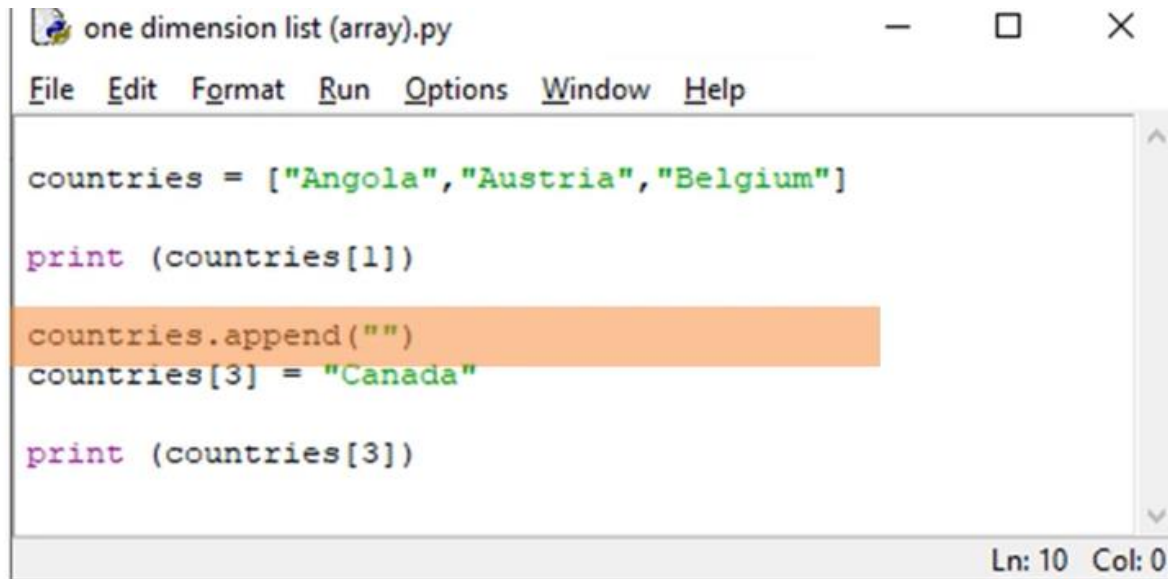
countries	
Index	Contents
0	Angola
1	Austria
2	Belgium

8.2 Arrays

One-dimensional arrays (Python)

Arrays are a static data structure – that means you can't change the size of an array once you have set it up.

This is a good example of Python allowing an array to behave like a list. Although you can't tell (because it is abstracted from you), Python is actually moving the entire data structure to a new part of the memory to ensure it is contiguous if it needs to be.



```
countries = ["Angola", "Austria", "Belgium"]

print (countries[1])

countries.append("")
countries[3] = "Canada"

print (countries[3])
```

Ln: 10 Col: 0

countries	
Index	Contents
0	Angola
1	Austria
2	Belgium
3	

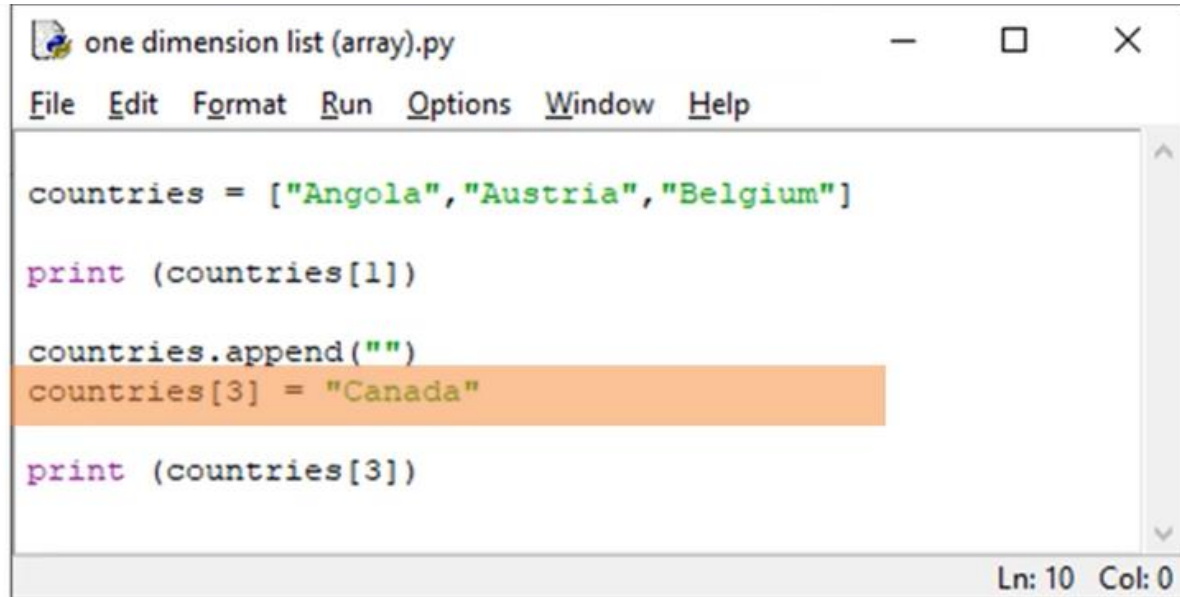
8.2 Arrays

One-dimensional arrays (Python)

Arrays are a static data structure – that means you can't change the size of an array once you have set it up.

This is a good example of Python allowing an array to behave like a list. Although you can't tell (because it is abstracted from you), Python is actually moving the entire data structure to a new part of the memory to ensure it is contiguous if it needs to be.

We can then insert an item into the new location at the end of the array using its index.



```
one dimension list (array).py
File Edit Format Run Options Window Help

countries = ["Angola", "Austria", "Belgium"]

print (countries[1])

countries.append("")
countries[3] = "Canada"

print (countries[3])

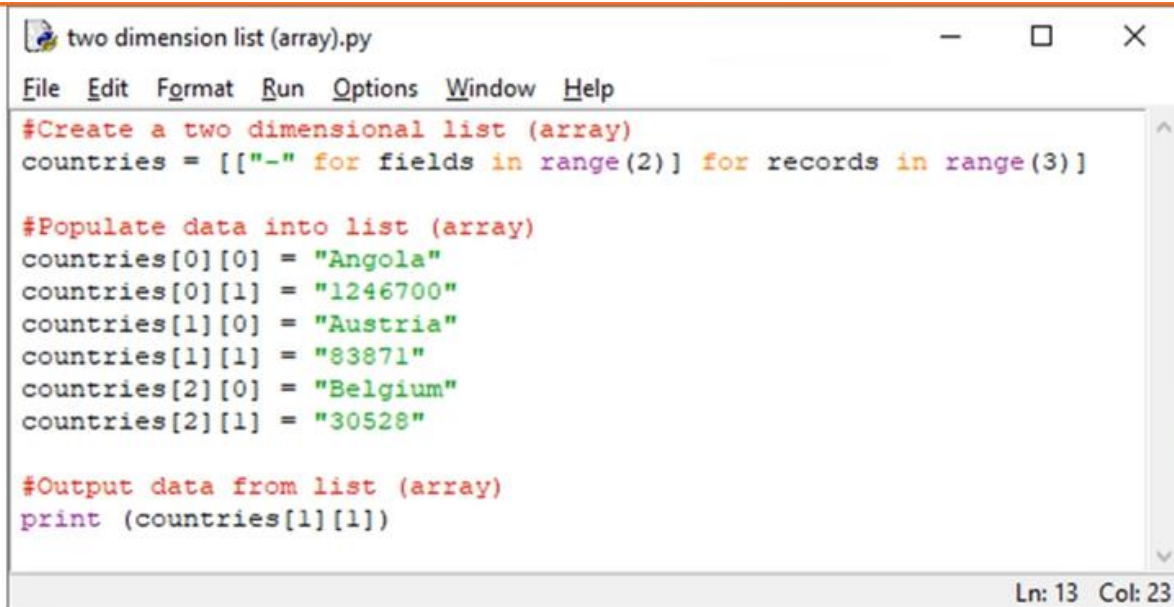
Ln: 10 Col: 0
```

countries	
Index	Contents
0	Angola
1	Austria
2	Belgium
3	Canada

8.2.1 Declare and use **one-dimensional (1D)** and **two-dimensional (2D) arrays**

Two-dimensional arrays (Python)

You can visualise a two-dimensional array as a table with two sets of indexes – one for the rows and another for the columns.



```
#Create a two dimensional list (array)
countries = [{"-" for fields in range(2)] for records in range(3)]

#Populate data into list (array)
countries[0][0] = "Angola"
countries[0][1] = "1246700"
countries[1][0] = "Austria"
countries[1][1] = "83871"
countries[2][0] = "Belgium"
countries[2][1] = "30528"

#Output data from list (array)
print (countries[1][1])
```

Ln: 13 Col: 23

countries		
Index	0	1
0	Angola	1246700
1	Austria	83871
2	Belgium	30528

8.2 Arrays

Two-dimensional arrays (Python)

You can visualise a two-dimensional array as a table with two sets of indexes – one for the rows and another for the columns.

"countries[0][0]" contains "Angola".

"countries[0][1]" contains "1246700".

"countries[1][1]" contains "83871".

```
#Create a two dimensional list (array)
countries = [ ["-" for fields in range(2)] for records in range(3)]

#Populate data into list (array)
countries[0][0] = "Angola"
countries[0][1] = "1246700"
countries[1][0] = "Austria"
countries[1][1] = "83871"
countries[2][0] = "Belgium"
countries[2][1] = "30528"

#Output data from list (array)
print (countries[1][1])
```

Ln: 13 Col: 23

countries		
Index	0	1
0	Angola	1246700
1	Austria	83871
2	Belgium	30528

8.2 Arrays

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Arrays

1D and 2D arrays are declared as follows (where l, l1, l2 are lower bounds and u, u1, u2 are upper bounds):

```
DECLARE <identifier> : ARRAY[<l>:<u>] OF <data type>
```

```
DECLARE <identifier> : ARRAY[<l1>:<u1>, <l2>:<u2>] OF <data type>
```

```
DECLARE StudentNames : ARRAY[1:30] OF STRING
```

```
DECLARE NoughtsAndCrosses : ARRAY[1:3, 1:3] OF CHAR
```

```
StudentNames[1] ← "Ali"
```

```
NoughtsAndCrosses[2,3] ← 'X'
```

```
StudentNames[n + 1] ← StudentNames[n]
```

An appropriate loop structure is used to assign the elements individually:

```
FOR Index ← 1 TO 30
```

```
    StudentNames[Index] ← ""
```

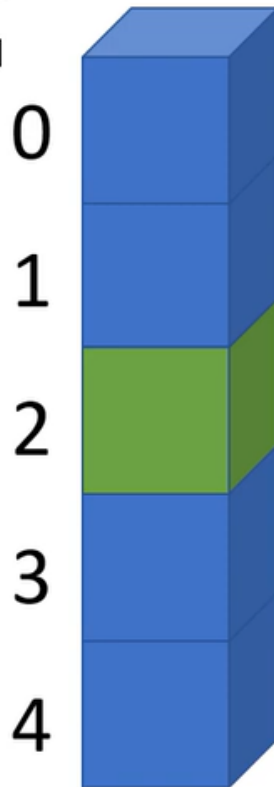
```
NEXT Index
```

8.2 Arrays

Going beyond a two-dimensional array

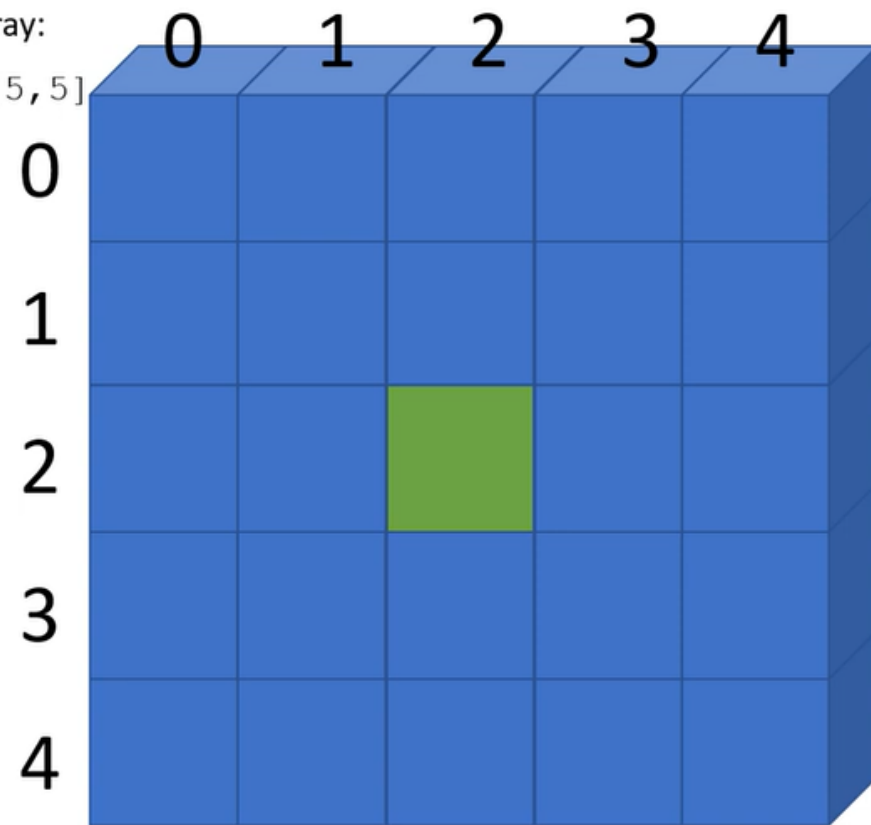
One-dimensional array:

```
array myArray[5]  
myArray[2]
```



Two-dimensional array:

```
array myArray[5,5]  
myArray[2,2]
```



8.2 Arrays



Arrays

- A static number of related data items stored in sequence within the same memory space.
- Each data item has the same (homogeneous) data type.
- A particular data item (element) is found using its index.
- Indexes usually start at 0 for the first data item (known as *zero-indexed*) but can also start from 1.
- Arrays may be single- or multi-dimensional.
- Visualise dimensions as a column (one-dimensional) or table (two-dimensional).

Names (1D array)	
Index	Name
0	Craig
1	Dave
2	Sam
3	Carol

names[2] = "Sam"

Names and scores (2D array)		
	Name	Score
Index	0	1
0	Craig	112
1	Dave	75
2	Sam	103
3	Carol	97

names[1][0] = "Dave"

Note: [1][0] could also be 112 depending on how you choose to store the data. It doesn't matter whether you visualise rows or columns first, providing you are consistent in your program. Read the question carefully in an exam so you can be sure which way around the array is being stored.

8.2.1 Declare and use **one-dimensional (1D)** and **two-dimensional (2D)** arrays

In simple terms:

- A **one-dimensional array** is like a **single row** of containers, each containing **one item**.
- A **two-dimensional array** is like a **table with rows and columns**, where each **cell holds an item**.

One-Dimensional (1D) Array: Example 1:

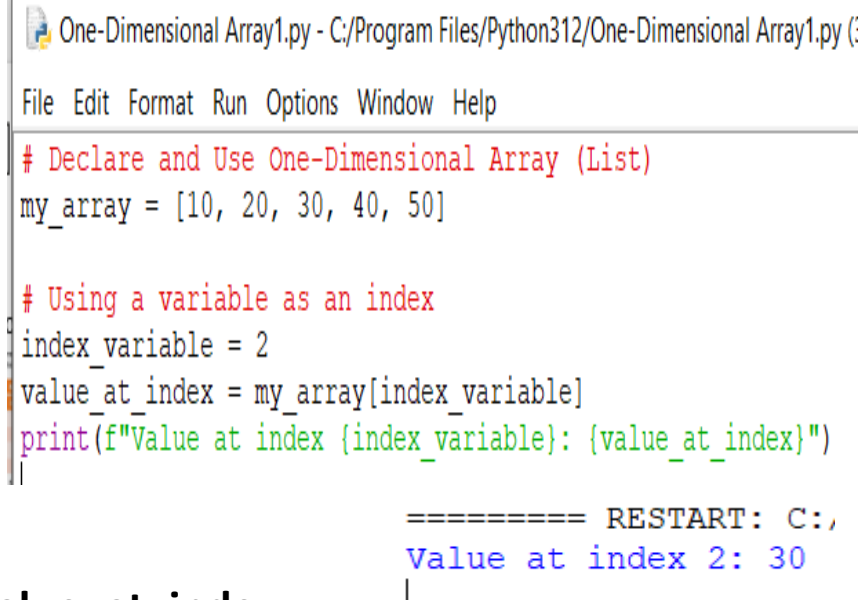
In Python, you can use a list to represent a one-dimensional array.

// Declare and Use One-Dimensional Array (List)

```
my_array[1] ← 10
my_array[2] ← 20
my_array[3] ← 30
my_array[4] ← 40
my_array[5] ← 50
```

// Using a variable as an index

```
index_variable ← 2
value_at_index ← my_array[index_variable]
OUTPUT "Value at index " + index_variable + ": " + value_at_index
```



The screenshot shows a Python IDE window titled "One-Dimensional Array1.py". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
# Declare and Use One-Dimensional Array (List)
my_array = [10, 20, 30, 40, 50]

# Using a variable as an index
index_variable = 2
value_at_index = my_array[index_variable]
print(f"Value at index {index_variable}: {value_at_index}")
```

The output of the program is displayed at the bottom right of the window:

```
===== RESTART: C:,
Value at index 2: 30
```


8.2.1 Declare and use **one-dimensional (1D)** and **two-dimensional (2D)** arrays

In simple terms:

One-Dimensional (1D) Array: Example 2

 1D array.py - C:/Program Files/Python312/1D array.py (3.12.0)

File Edit Format Run Options Window Help

```
# Declare a 1D array
one_dimensional_array = [1, 2, 3, 4, 5]

# Accessing elements
print("First element:", one_dimensional_array[0])
print("Second element:", one_dimensional_array[1])

# Modifying an element
one_dimensional_array[2] = 10

# Display the modified array
print("Modified array:", one_dimensional_array)

===== RESTART: C:/Progra
First element: 1
Second element: 2
Modified array: [1, 2, 10, 4, 5]
```

```
//algorithm declare_and_use_1d_array
// Declare a 1D array
one_dimensional_array ← [1, 2, 3, 4, 5]
```

```
// Accessing elements
output "First element:",
one_dimensional_array[0]
output "Second element:",
one_dimensional_array[1]
```

```
// Modifying an element
one_dimensional_array[2] ← 10
```

```
// Display the modified array
output "Modified array:",
one_dimensional_array
end algorithm
```


8.2.1 Declare and use **one-dimensional (1D)** and **two-dimensional (2D)** arrays

Two-Dimensional (2D) Array:

```
1D array.py - C:/Program Files/Python312/1D array.py (3.12.0)
File Edit Format Run Options Window Help
# Declare a 2D array
two_dimensional_array = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Accessing elements
print("Element at row 2, column 1:", two_dimensional_array[1][0])

# Modifying an element
two_dimensional_array[0][2] = 20

# Display the modified array
print("Modified array:")
for row in two_dimensional_array:
    print(row)

===== RESTART: C:/Pr
Element at row 2, column 1: 4
Modified array:
[1, 2, 20]
[4, 5, 6]
[7, 8, 9]
```

Note: outer array (**two_dimensional_array**) holds three inner arrays, each corresponding to a row in the 2D array. Each inner array contains elements representing the values in that specific row

```
//algorithm declare_and_use_2d_array
```

```
// Declare a 2D array
```

```
two_dimensional_array ← [
```

```
    [1, 2, 3], # Row 1
```

```
    [4, 5, 6], # Row 2
```

```
    [7, 8, 9] # Row 3
```

```
]
```

```
// Accessing elements
```

```
output "Element at row 2, column 1:",
two_dimensional_array[1][0]
```

```
// Modifying an element
```

```
two_dimensional_array[0][2] ← 20
```

```
// Display the modified array
```

```
output "Modified array:"
```

```
for each row in two_dimensional_array
```

```
    output row
```

```
end for
```

```
end algorithm
```

8.2.2 Understand the use of arrays

Use of Arrays in Python:

In Python, arrays are represented using lists. Lists are able to adapt and can hold various data types.

File Edit Format Run Options Window Help

```
# Declare an array (list)
my_array = [10, 20, 30, 40, 50]

# Access elements using indexes
first_element = my_array[0]
second_element = my_array[1]

# Modify an element
my_array[2] = 300

# Print the modified array
print("Modified array:", my_array)
```

```
===== RESTART: C:/Program Fi
Modified array: [10, 20, 300, 40, 50]
```

Declare and Use One-Dimensional Array (List)

```
ARRAY myArray[5] # Declaring an array of size 5
```

Initialize the array

```
myArray[0] <- 10
myArray[1] <- 20
myArray[2] <- 30
myArray[3] <- 40
myArray[4] <- 50
```

Access elements using variables as indexes

```
INDEX_VARIABLE <- 2
VALUE_AT_INDEX <- myArray[INDEX_VARIABLE]
```

Output the value at a specific index

```
OUTPUT "Value at index " + INDEX_VARIABLE + ": " +
VALUE_AT_INDEX
```

Modify an element

```
myArray[2] <- 300
```

Output the modified array

```
FOR i FROM 0 TO 4
    OUTPUT "Element at index " + i + ": " + myArray[i]
END FOR
```

8.2.2 Understand the use of arrays

Use of Variables as Indexes:

1D array.py - C:/Program Files/Python312/1D array.py (3.12.0)

File Edit Format Run Options Window Help

```
# Declare an array (list)
my_array = [10, 20, 30, 40, 50]
```

```
# Access elements using indexes
first_element = my_array[0]
second_element = my_array[1]
```

```
# Modify an element
my_array[2] = 300
```

```
# Print the modified array
print("Modified array:", my_array)
```

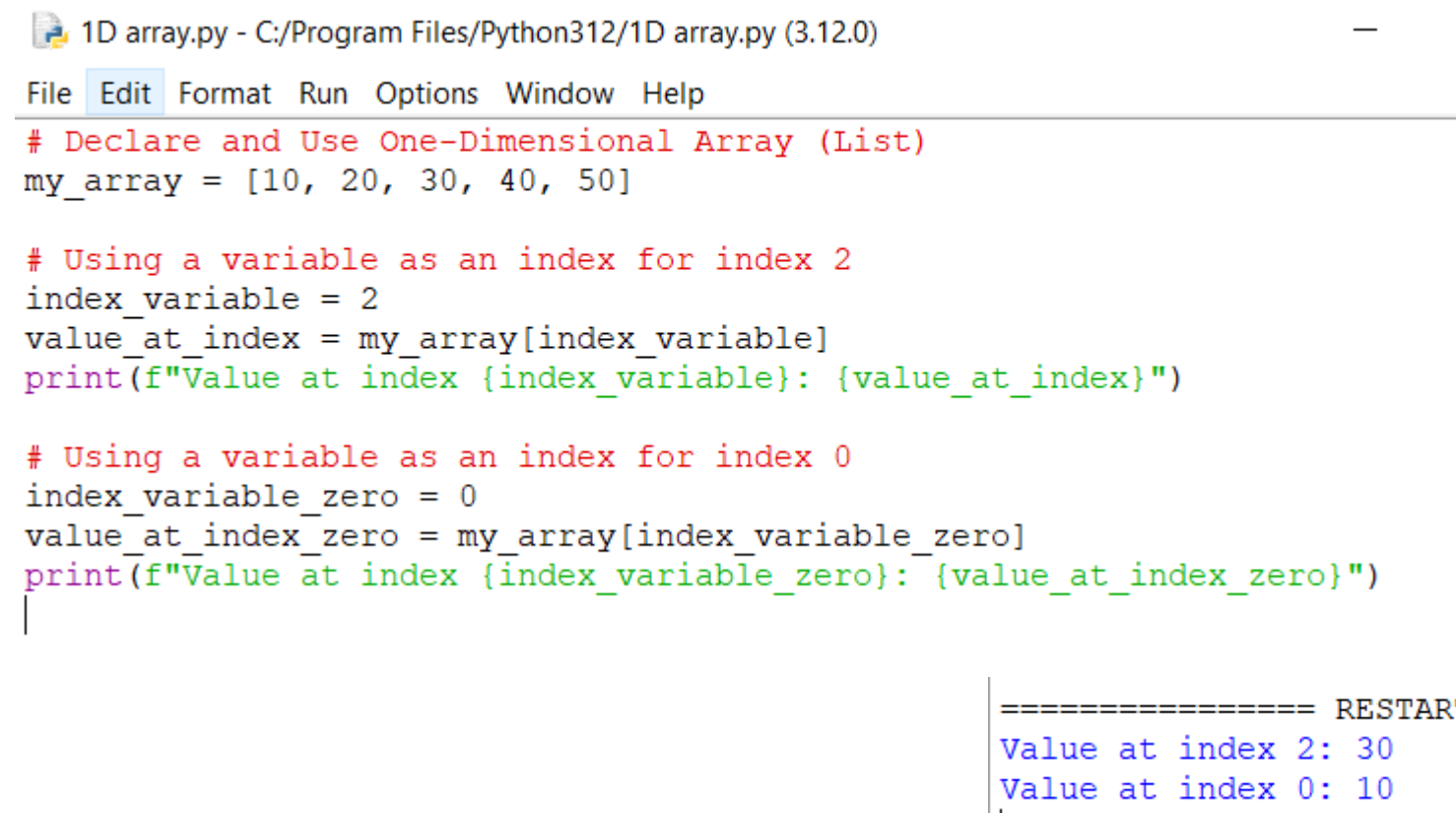
```
# Using a variable as an index
index_variable = 2
value_at_index = my_array[index_variable]
print(f"Value at index {index_variable}: {value_at_index}")
```

```
===== RESTART: C:/Program F
Modified array: [10, 20, 300, 40, 50]
Value at index 2: 300
```

8.2.2 Understand the use of arrays

Starting Index at Zero or One:

In most programming languages, including Python, arrays typically start with an index of zero. However, some languages, like MATLAB, use one-based indexing.



The screenshot shows a Python IDE window titled "1D array.py - C:/Program Files/Python312/1D array.py (3.12.0)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code in the editor is as follows:

```
# Declare and Use One-Dimensional Array (List)
my_array = [10, 20, 30, 40, 50]

# Using a variable as an index for index 2
index_variable = 2
value_at_index = my_array[index_variable]
print(f"Value at index {index_variable}: {value_at_index}")

# Using a variable as an index for index 0
index_variable_zero = 0
value_at_index_zero = my_array[index_variable_zero]
print(f"Value at index {index_variable_zero}: {value_at_index_zero}")
```

The output of the program is displayed in the bottom right corner:

```
===== RESTART: C:
Value at index 2: 30
Value at index 0: 10
```

8.2.3 Write **values** into and **read** values from an **array** using **iteration**

In this example, values are written into the array (myArray) using iteration and then read from the array using another iteration. The values are calculated based on the index.

```
DECLARE myArray[5] INTEGER
DECLARE index INTEGER
```

```
FOR index FROM 1 TO 5
    // Writing values into the array
    myArray[index] = index * 10
END FOR
```

```
FOR index FROM 1 TO 5
    // Reading values from the array
    OUTPUT "Value at index " + index + ": " + myArray[index]
END FOR
```

 1D array.py - C:/Program Files/Python312/1D array.py (3.12.0)

```
File Edit Format Run Options Window Help
# Declare and Use One-Dimensional Array (List)
my_array = [0] * 5 # Initialize array with zeros
index = 0

# Writing values into the array using iteration
for index in range(5):
    my_array[index] = (index + 1) * 10

# Reading values from the array using iteration
for index in range(5):
    print(f"Value at index {index + 1}: {my_array[index]}")
```

```
===== RESTART
Value at index 1: 10
Value at index 2: 20
Value at index 3: 30
Value at index 4: 40
Value at index 5: 50
|
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Question 1:

- 2 Tick (✓) **one** box to show the name of the data structure used to store a collection of data of the same data type.

A Array

☐

B Constant

☐

C Function

☐

D Variable

☐

[1]

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 1:

PUBLISHED

Question	Answer	Marks
2	A	1

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Question 2:

- 11** A one-dimensional (1D) array `Days[]` contains the names of the days of the week. A two-dimensional (2D) array `Readings[]` is used to store 24 temperature readings, taken once an hour, for each of the seven days of the week. A 1D array `AverageTemp[]` is used to store the average temperature for each day of the week.

The position of any day's data is the same in all three arrays. For example, if Wednesday is in index 4 of `Days[]`, Wednesday's temperature readings are in index 4 of `Readings[]` and Wednesday's average temperature is in index 4 of `AverageTemp[]`.

The temperature readings are in Celsius to one decimal place. Temperatures can only be from -20.0°C to $+50.0^{\circ}\text{C}$ inclusive.

Write a program that meets the following requirements:

- input and validate the hourly temperatures for one week
- calculate and store the average temperature for each day of the week
- calculate the average temperature for the whole week
- convert all the average temperatures from Celsius to Fahrenheit by using the formula $\text{Fahrenheit} = \text{Celsius} * 9/5 + 32$
- output the average temperature in Celsius and in Fahrenheit for each day
- output the overall average temperature in Celsius and in Fahrenheit for the whole week.

You must use pseudocode or program code **and** add comments to explain how your code works.

You do **not** need to declare any arrays, variables or constants; you may assume that this has already been done.

All inputs and outputs must contain suitable messages.

All data output must be rounded to one decimal place.

You will need to initialise and populate the array `Days[]` at the start of the program.

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 2:

Question	Answer	Marks
11	Read the whole answer: Check if each requirement listed below has been met. Requirements may be met using a suitable built-in function from the programming language used (Python, VB.NET or Java). On place a SEEN mark if requirement met, cross if no attempt seen, omission mark and/or comment if partially met (see marked scripts). Use the tables for AO2 and AO3 below to award a mark in a suitable band using a best fit approach, then add up the total: • AO2 (maximum 9 marks) • AO3 (maximum 6 marks) Data structures required: The names underlined must match those given in the scenario: Arrays or lists <u>Days []</u>, <u>Readings []</u>, <u>AverageTemp []</u> Variables <u>WeekLoop</u>, <u>DayLoop</u>, <u>InTemp</u>, <u>TotalDayTemp</u>, <u>TotalWeekTemp</u>, <u>AverageWeekTemp</u> Requirements (techniques): R1 Input and store hourly temperatures and validation of input temperatures for each day (with prompts, range check and (nested)iteration) R2 Calculate, round to one decimal place and store daily average temperatures and calculate the weekly average temperature rounded to one decimal place (iteration, totalling and rounding) R3 Convert all average temperatures to Fahrenheit (to one decimal place) and output the average temperatures in both Celsius and Fahrenheit. Output with appropriate messages. (output and rounding)	15

Marking Instructions in italics			
AO2: Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems			
0	1–3	4–6	7–9
No creditable response.	At least one programming technique has been used. <i>Any use of selection, iteration, counting, totalling, input and output.</i>	Some programming techniques used are appropriate to the problem. <i>More than one technique seen applied to the scenario, check the list of techniques needed.</i>	The range of programming techniques used is appropriate to the problem. <i>All criteria stated for the scenario have been covered by the use of appropriate programming techniques, check the list of techniques needed.</i>
	Some data has been stored but not appropriately. <i>Any use of variables or arrays or other language dependent data structures e.g. Python lists.</i>	Some of the data structures chosen are appropriate and store some of the data required. <i>More than one data structure used to store data required by the scenario.</i>	The data structures chosen are appropriate and store all the data required. <i>The data structures used store all the data required by the scenario.</i>

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 2:

Question	Answer	Marks
11	Example 15 mark answer in pseudocode <pre>// meaningful identifiers and appropriate data structures for // all data required DECLARE Days : ARRAY[1:7] OF STRING DECLARE Readings : ARRAY[1:7, 1:24] OF REAL DECLARE AverageTemp : ARRAY[1:7] OF REAL DECLARE WeekLoop : INTEGER DECLARE DayLoop : INTEGER DECLARE InTemp : REAL DECLARE TotalDayTemp : REAL DECLARE TotalWeekTemp : REAL DECLARE AverageWeekTemp : REAL // initial population of Days[] array // input and a loop are also acceptable Days[1] ← "Sunday" Days[2] ← "Monday" Days[3] ← "Tuesday" Days[4] ← "Wednesday" Days[5] ← "Thursday" Days[6] ← "Friday" Days[7] ← "Saturday" // input temperatures inside nested loop FOR WeekLoop ← 1 TO 7 TotalDayTemp ← 0 FOR DayLoop ← 1 TO 24 OUTPUT "Enter temperature ", DayLoop, " for ", Days[WeekLoop]</pre>	

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 2:

Question	Answer	Marks
11	<pre>INPUT InTemp // validation of input for between -20 and +50 inclusive WHILE InTemp < -20.0 OR InTemp > 50.0 DO OUTPUT "Your temperature must be between -20.0 and +50.0 inclusive. Please try again" INPUT InTemp ENDWHILE Readings[WeekLoop, DayLoop] ← InTemp // totalling of temperatures during the day TotalDayTemp ← TotalDayTemp + ROUND(InTemp, 1) NEXT DayLoop // average temperature for the day AverageTemp[WeekLoop] ← ROUND(TotalDayTemp / 24,1) NEXT WeekLoop // calculate the average temperature for the week TotalWeekTemp ← 0 FOR WeekLoop ← 1 TO 7 TotalWeekTemp ← TotalWeekTemp + AverageTemp[WeekLoop] NEXT WeekLoop AverageWeekTemp ← ROUND(TotalWeekTemp / 7,1) // outputs in Celsius and Fahrenheit FOR WeekLoop ← 1 TO 7 OUTPUT "The average temperature on ", Days[WeekLoop], " was ", AverageTemp[WeekLoop], " Celsius and ", ROUND(AverageWeekTemp * 9 / 5 + 32), 1, " Fahrenheit" NEXT WeekLoop OUTPUT "The average temperature for the week was ", AverageWeekTemp, " Celsius and ", ROUND(AverageWeekTemp * 9 / 5 + 32, 1), " Fahrenheit"</pre>	


```
# Meaningful identifiers and appropriate data structures for all data required
days = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]
readings = [[0.0] * 24 for _ in range(7)]
average_temp = [0.0] * 7

# Initial population of Days[] array
# Input and a loop are also acceptable
# (In this Python code, the array is already initialized with day names)

# Input temperatures inside nested loop
for week_loop in range(7):
    total_day_temp = 0
    for day_loop in range(1):
        # Input temperatures inside nested loop
        in_temp = float(input(f"Enter temperature {day_loop + 1} for {days[week_loop]}: "))

        # Validation of input for between -20 and +50 inclusive
        while not (-20.0 <= in_temp <= 50.0):
            print("Your temperature must be between -20.0 and +50.0 inclusive. Please try again")
            in_temp = float(input(f"Enter temperature {day_loop + 1} for {days[week_loop]}: "))

        readings[week_loop][day_loop] = in_temp
        # Totalling of temperatures during the day
        total_day_temp += round(in_temp, 1)

    # Average temperature for the day
    average_temp[week_loop] = round(total_day_temp / 24, 1)

# Calculate the average temperature for the week
total_week_temp = sum(average_temp)
average_week_temp = round(total_week_temp / 7, 1)

# Outputs in Celsius and Fahrenheit
for week_loop in range(7):
    print(f"The average temperature on {days[week_loop]} was {average_temp[week_loop]} Celsius and "
          f"{round(average_temp[week_loop] * 9 / 5 + 32, 1)} Fahrenheit")

print(f"The average temperature for the week was {average_week_temp} Celsius and "
      f"{round(average_week_temp * 9 / 5 + 32, 1)} Fahrenheit")
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 2:

```
= RESTART: C:/Program Files/Python312/Logical OR.py
Enter temperature 1 for Sunday: 10
Enter temperature 1 for Monday: 12
Enter temperature 1 for Tuesday: 14
Enter temperature 1 for Wednesday: 15
Enter temperature 1 for Thursday: 24
Enter temperature 1 for Friday: 27
Enter temperature 1 for Saturday: 33
The average temperature on Sunday was 0.4 Celsius and 32.7 Fahrenheit
The average temperature on Monday was 0.5 Celsius and 32.9 Fahrenheit
The average temperature on Tuesday was 0.6 Celsius and 33.1 Fahrenheit
The average temperature on Wednesday was 0.6 Celsius and 33.1 Fahrenheit
The average temperature on Thursday was 1.0 Celsius and 33.8 Fahrenheit
The average temperature on Friday was 1.1 Celsius and 34.0 Fahrenheit
The average temperature on Saturday was 1.4 Celsius and 34.5 Fahrenheit
The average temperature for the week was 0.8 Celsius and 33.4 Fahrenheit
|
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Question 3:

- 12** A two-dimensional (2D) array `Account[]` contains account holders' names and passwords for a banking program.

A 2D array `AccDetails[]` has three columns containing the following details:

- column one stores the balance – the amount of money in the account, for example 250.00
- column two stores the overdraft limit – the maximum total amount an account holder can borrow from the bank after the account balance reaches 0.00, for example 100.00
- column three stores the withdrawal limit – the amount of money that can be withdrawn at one time, for example 200.00

The amount of money in a bank account can be negative (overdrawn) but **not** by more than the overdraft limit.

For example, an account with an overdraft limit of 100.00 must have a balance that is greater than or equal to –100.00

Suitable error messages must be displayed if a withdrawal cannot take place, for example if the overdraft limit or the size of withdrawal is exceeded.

The bank account ID gives the index of each account holder's data held in the two arrays. For example, account ID 20's details would be held in:

`Account[20,1]` and `Account[20,2]`

`AccDetails[20,1]` `AccDetails[20,2]` and `AccDetails[20,3]`

The variable `Size` contains the number of accounts.

The arrays and variable `Size` have already been set up and the data stored.

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Question 3:

Write a program that meets the following requirements:

- checks the account ID exists and the name and password entered by the account holder match the name and password stored in `Account[]` before any action can take place
- displays a menu showing the four actions available for the account holder to choose from:
 1. display balance
 2. withdraw money
 3. deposit money
 4. exit
- allows an action to be chosen and completed. Each action is completed by a procedure with a parameter of the account ID.

You must use pseudocode or program code **and** add comments to explain how your code works. All inputs and outputs must contain suitable messages.

You only need to declare any local arrays and local variables that you use.

You do **not** need to declare and initialise the data in the global arrays `Account[]` and `AccDetails[]` and the variable `Size`

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 3:

Question	Answer	Marks
12	Read the whole answer: Check if each requirement listed below has been met. Requirements may be met using a suitable built-in function from the programming language used (Python, VB.NET or Java) Mark SEEN on script if requirement met, cross if no attempt seen, NE if partially met (see marked scripts). Use the tables for A02 and A03 below to award a mark in a suitable band using a best fit approach Then add up the total. Marks are available for: • AO2 (maximum 9 marks) • AO3 (maximum 6 marks) Data Structures required names shown underlined must be used as given in the scenario Arrays or lists <u>Account</u>, <u>AccDetails</u> Variable <u>Size</u>, <u>AccountNumber</u> Requirements (techniques) R1 Check account number and password (iteration and validation, selection, input, output) R2 Display menu and make a selection (output, input and selection) R3 Perform actions selected (use of arrays and procedures with parameters) Example 15 mark answer in pseudocode // Procedures to be called PROCEDURE CheckDetails(AccID : INTEGER) DECLARE Name, Password : STRING // local variables Valid ← FALSE IF AccID <0 OR AccID > Size THEN OUTPUT "Invalid Account Number" ELSE OUTPUT "Please Enter Name " INPUT Name OUTPUT "Please Enter Password " INPUT Password IF Name <> Account[AccID,1] OR Password <> Account[AccID,2] THEN OUTPUT "Invalid name or password" ELSE	15

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 3:

```
        Valid ← True
    ENDIF
ENDIF
ENDPROCEDURE

PROCEDURE Balance(AccID : INTEGER)
    OUTPUT "Your balance is ", AccDetails[AccID,1]
ENDPROCEDURE

PROCEDURE Withdrawal(AccID : INTEGER)
    DECLARE Amount : REAL // local variable
    REPEAT
        OUTPUT "Please enter amount to withdraw "
        INPUT Amount
        IF Amount > AccDetails[AccID,3]
            THEN
                OUTPUT "Amount greater than withdrawal limit"
            ENDIF
        IF Amount > AccDetails[AccID,2] + AccDetails[AccID,1]
            THEN
                OUTPUT "Amount greater than cash available"
            ENDIF
        IF Amount <= AccDetails[AccID,3] AND Amount < AccDetails[AccID,2] +
            AccDetails[AccID,1]
            THEN
                AccDetails[AccID,1] ← AccDetails[AccID,1] - Amount
            ENDIF
        UNTIL Amount <= AccDetails[AccID,3] AND Amount > AccDetails[AccID,2] +
            AccDetails[AccID,1] AND Amount > 0
    ENDPROCEDURE

PROCEDURE Deposit(AccID : INTEGER)
    DECLARE Amount : REAL // local variable
    REPEAT
        OUTPUT "Please enter a positive amount to deposit "
        INPUT Amount
    UNTIL Amount > 0
    AccDetails[AccID,1] ← AccDetails[AccID,1] + Amount
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 3:

```
ENDPROCEDURE

// Declarations of global variables for information - not required in candidate responses
DECLARE AccountNumber, Choice : INTEGER
DECLARE Valid, Exit : BOOLEAN

OUTPUT "Please enter your account number "
INPUT AccountNumber
CheckDetails(AccountNumber)

IF Valid
    THEN
        REPEAT
            OUTPUT "Menu"
            OUTPUT "1. display balance"
            OUTPUT "2. withdraw money"
            OUTPUT "3. deposit money"
            OUTPUT "4. exit"
            OUTPUT "please choose 1, 2, 3 or 4"
            INPUT Choice
            CASE OF Choice
                1 : Balance(AccountNumber)
                2 : Withdrawal(AccountNumber)
                3 : Deposit(AccountNumber)
                4 : Exit ← TRUE
            OTHERWISE OUTPUT "Invalid choice"
            ENDCASE
        UNTIL Choice = 4
    ELSE
        OUTPUT "Invalid account number "
    ENDIF
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 3:

Marking Instructions in italics			
AO2: Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems			
0	1–3	4–6	7–9
No creditable response.	At least one programming technique has been used. <i>Any use of selection, iteration, counting, totalling, input and output.</i>	Some programming techniques used are appropriate to the problem. <i>More than one technique seen applied to the scenario, check the list of techniques needed.</i>	The range of programming techniques used is appropriate to the problem. <i>All criteria stated for the scenario have been covered by the use of appropriate programming techniques, check the list of techniques needed.</i>
	Some data has been stored but not appropriately. <i>Any use of variables or arrays or other language dependent data structures e.g. Python lists.</i>	Some of the data structures chosen are appropriate and store some of the data required. <i>More than one data structure used to store data required by the scenario.</i>	The data structures chosen are appropriate and store all the data required. <i>The data structures used store all the data required by the scenario.</i>

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Question 4:

- 11** A two-dimensional (2D) array `Contacts[]` is used to store names and telephone numbers. All the data is stored as strings. The array must have the capacity to store 100 contacts in the form of:
- column 1 – contact names as: last name, first name
for example: Smith, John
 - column 2 – telephone numbers.

The variable `CurrentSize` shows how many contacts are in the array.

Write a program that meets the following requirements:

- display a menu of choices:
 - enter new contact details
 - display all the contact details
 - delete all the contact details
- validate the menu input
- allow up to a maximum of five new contacts to be added to the array at any one time
- do **not** allow more than 100 contacts in total
- after new contacts have been added, sort the array by contact name, as long as there are at least two contacts in the array
- output the whole of the array
- delete the contents of the array.

You must use pseudocode or program code **and** add comments to explain how your code works.

You do **not** need to declare any arrays, variables or constants; you may assume that this has already been done.

All inputs and outputs must contain suitable messages.

You do **not** need to initialise the data in the array `Contacts[]` and the variable `CurrentSize`

.....

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 4:

Question	Answer	Marks
11	Read the whole answer: Check if each requirement listed below has been met. Requirements may be met using a suitable built-in function from the programming language used (Python, VB.NET or Java). Mark SEEN on script if requirement met, cross if no attempt seen, NE if partially met (see marked scripts). Use the tables for A02 and A03 below to award a mark in a suitable band using a best fit approach. Then add up the total. Marks are available for: • A02 (maximum 9 marks) • A03 (maximum 6 marks) Data structures required: The names underlined must match those given in the scenario: Arrays or lists <u>Contacts[]</u> Variables <u>CurrentSize</u>, Cont, Choice, NewContacts, Count, Count2, Flag Requirements (techniques): R1 Output menu and input choice, with validation (range check, output with messages, input with prompts). R2 Input number of new entries, within limits, update current size of contacts, input new data and sort the array (range check, totalling, iteration and bubble sort). R3 Output array whole contents and delete contents of array (iteration, output with labelling/messages, array initialisation).	15

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 4:

Example 15 mark answer in pseudocode

```
// meaningful identifiers and appropriate data structures for
// all data required
DECLARE Contacts : ARRAY[1:100, 1:2] OF STRING
DECLARE CurrentSize : INTEGER
DECLARE Cont : BOOLEAN
DECLARE Choice : INTEGER
DECLARE NewContacts : INTEGER
DECLARE Count : INTEGER
DECLARE Count2 : INTEGER
DECLARE Flag : BOOLEAN
DECLARE Temp1 : STRING
DECLARE Temp2 : STRING

// the number of contacts in the array
CurrentSize ← 0

// to allow program to continue indefinitely
Cont ← TRUE
WHILE Cont DO
// display menu
    OUTPUT "Please choose one of the following: "
    OUTPUT "Press 1 to enter new contacts "
    OUTPUT "Press 2 to display your contacts "
    OUTPUT "Press 3 to delete all contacts "
    INPUT Choice
// validate choice as 1, 2 or 3
    WHILE Choice = 1 AND CurrentSize = 100 DO
        OUTPUT "Your contacts are full, please enter 2 or 3"
        INPUT Choice
    ENDWHILE
    WHILE Choice < 1 OR Choice > 3 DO
        OUTPUT "Incorrect entry - please enter 1, 2, or 3"
        INPUT Choice
    ENDWHILE
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 4:

```
// enter new contacts
  IF Choice = 1
    THEN
      OUTPUT "How many contacts (1 to 5 only)?"
      INPUT NewContacts
// validates new contacts input
      WHILE NewContacts < 1 OR NewContacts > 5 DO
        OUTPUT "You may only enter between 1 and 5 contacts. Please try again"
        INPUT NewContacts
      ENDWHILE
// checks the maximum size is not exceeded
      WHILE CurrentSize + NewContacts > 100
        OUTPUT "Not enough space in your contacts"
        OUTPUT "The maximum number you may input is ", 100 - CurrentSize
        INPUT NewContacts
      ENDWHILE
      FOR Count ← CurrentSize + 1 TO CurrentSize + NewContacts
        OUTPUT "Enter the contact name as last name, first name"
        INPUT Contacts[Count, 1]
        OUTPUT "Enter the telephone number"
        INPUT Contacts[Count, 2]
      NEXT Count
      CurrentSize ← CurrentSize + NewContacts
// bubble sort to sort array if it contains 2 or more contacts
      IF CurrentSize >= 2
        THEN
          REPEAT
            Flag ← FALSE
            FOR Count ← 1 TO CurrentSize-1
              IF Contacts[Count + 1, 1] <
                Contacts[Count, 1]
                THEN
                  Flag ← TRUE
                  Temp1 ← Contacts[Count, 1]
                  Temp2 ← Contacts[Count, 2]
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 4:

```

                                Contacts[Count, 1] ← Contacts[Count + 1, 1]
                                Contacts[Count, 2] ← Contacts[Count + 1, 2]
                                Contacts[Count + 1, 1] ← Temp1
                                Contacts[Count + 1, 2] ← Temp2

                                ENDIF
                                NEXT Count
                                UNTIL NOT Flag
                                ENDIF
                                ENDIF
// display all contacts
    IF Choice = 2
    THEN
        IF CurrentSize > 0
        THEN
            OUTPUT "Name and Telephone Number"
            FOR Count ← 1 TO CurrentSize
                OUTPUT Contacts[Count, 1], "    ", Contacts[Count, 2]
            NEXT Count
        ENDIF
    ENDIF
// delete all contacts
    IF Choice = 3
    THEN
        FOR Count ← 1 TO 100
            FOR Count2 ← 1 TO 2
                Contacts[Count, Count2] ← ""
            NEXT Count2
        NEXT Count
    ENDIF
ENDWHILE
```

8.2 Arrays

Past Paper 2 exams Questions and Answers:

Answer 4:

Marking Instructions in italics			
AO2: Apply knowledge and understanding of the principles and concepts of computer science to a given context, including the analysis and design of computational or programming problems			
0	1–3	4–6	7–9
No creditable response.	At least one programming technique has been used. <i>Any use of selection, iteration, counting, totalling, input and output.</i>	Some programming techniques used are appropriate to the problem. <i>More than one technique seen applied to the scenario, check the list of techniques needed.</i>	The range of programming techniques used is appropriate to the problem. <i>All criteria stated for the scenario have been covered by the use of appropriate programming techniques, check the list of techniques needed.</i>
	Some data has been stored but not appropriately. <i>Any use of variables or arrays or other language dependent data structures e.g. Python lists.</i>	Some of the data structures chosen are appropriate and store some of the data required. <i>More than one data structure used to store data required by the scenario.</i>	The data structures chosen are appropriate and store all the data required. <i>The data structures used store all the data required by the scenario.</i>

8.3 File handling

Candidates should be able to:

- 1 Understand the purpose of storing data in a file to be used by a program
- 2 Open, close and use a file for reading and writing

Notes and guidance

- Including:
 - read and write single items of data
 - read and write a line of text

8.3 File handling

[Click the link below to watch the video for the use of basic file handling operations](https://www.youtube.com/watch?v=wHgQdK78j-U)

<https://www.youtube.com/watch?v=wHgQdK78j-U>

8.3 File handling

The use of basic file
handling operations



CAMBRIDGE
IGCSE
(0478/0984)

6:52
ave.c

8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
```

```
    name = f.readline().strip()
```

```
    health = f.readline().strip()
```

```
    stamina = f.readline().strip()
```

```
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
```

```
    print(name)
```

```
    print("Health:", health)
```

```
    print("Stamina:", stamina)
```

```
    print("Hunger:", hunger)
```

```
if name == "":
```

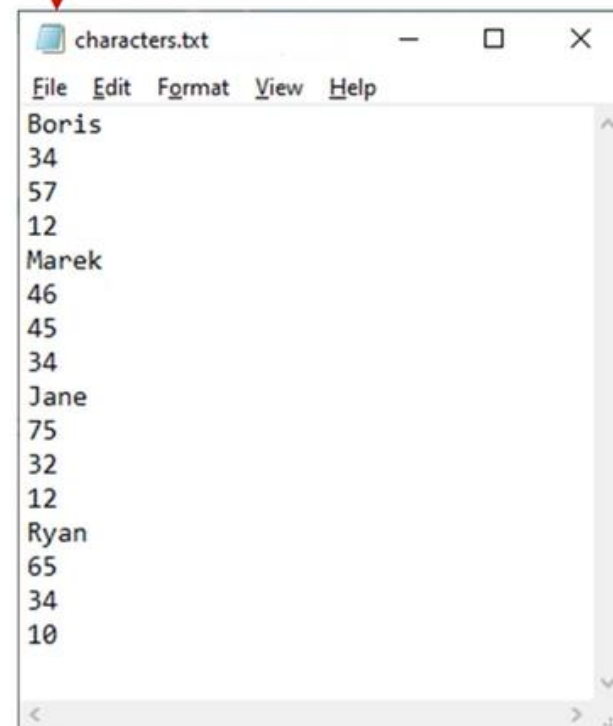
```
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

This program asks for the name of a character, looks up the name of that character in a text file and outputs all their attributes:

- Health
- Stamina
- Hunger



8.3 File handling

Reading data from text files

```
#Input the character to find in the file
character = input("Which character do you wish to output? ")

#Step 1 - Open the file for reading/writing
f = open("characters.txt", "r")

#Step 2 - Use a flag to indicate if the end of file is reached
end_of_file = False

#Step 3 - Iterate through all the data in the file
while not end_of_file:
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()

    #Step 4 - End of file or output data from file
    if character == name:
        print(name)
        print("Health:", health)
        print("Stamina:", stamina)
        print("Hunger:", hunger)
    if name == "":
        end_of_file = True

#Step 5 - Close the file
f.close()
```

This program asks for the name of a character, looks up the name of that character in a text file and outputs all their attributes:

- Health
- Stamina
- Hunger



8.3 File handling

Reading data from text files

#Input the character to find in the file
character = input("Which character do you wish to output? ")

#Step 1 - Open the file for reading/writing
f = open("characters.txt", "r")

#Step 2 - Use a flag to indicate if the end of file is reached
end_of_file = False

#Step 3 - Iterate through all the data in the file
while not end_of_file:

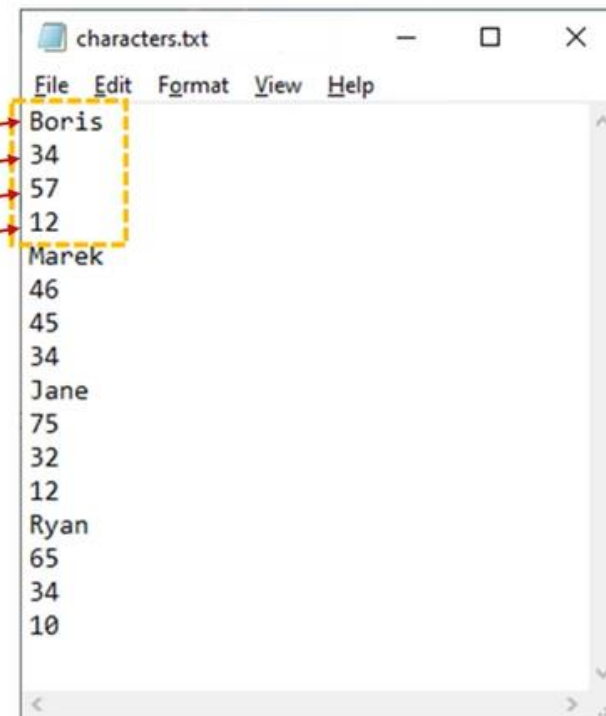
```
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
    print(name)
    print("Health:", health)
    print("Stamina:", stamina)
    print("Hunger:", hunger)
if name == "":
    end_of_file = True
```

#Step 5 - Close the file
f.close()

Our program requires the items from the text file to be in a very certain order – known as *data dependency*.



8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()
```

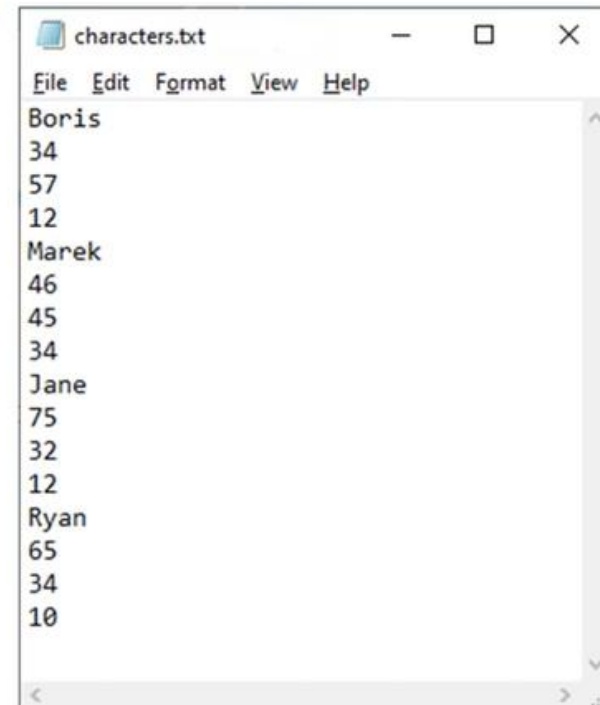
#Step 4 - End of file or output data from file

```
if character == name:
    print(name)
    print("Health:", health)
    print("Stamina:", stamina)
    print("Hunger:", hunger)
if name == "":
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

First, we ask the user which character they want to output.



8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
```

```
    name = f.readline().strip()
```

```
    health = f.readline().strip()
```

```
    stamina = f.readline().strip()
```

```
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
```

```
    print(name)
```

```
    print("Health:", health)
```

```
    print("Stamina:", stamina)
```

```
    print("Hunger:", hunger)
```

```
if name == "":
```

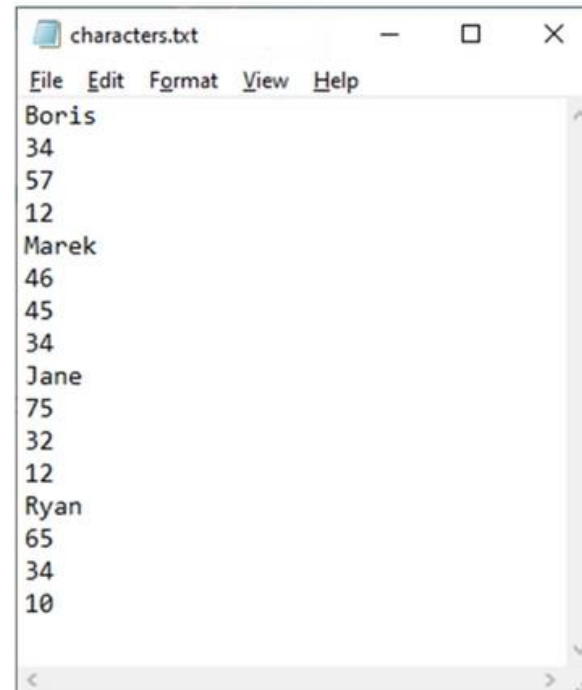
```
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

Next, we open the file for reading. The open command takes two parameters:

- "characters.txt" – the name of the file to open.
- "r" – what we want to do with the file (e.g., read).



8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
```

```
    name = f.readline().strip()
```

```
    health = f.readline().strip()
```

```
    stamina = f.readline().strip()
```

```
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
```

```
    print(name)
```

```
    print("Health:", health)
```

```
    print("Stamina:", stamina)
```

```
    print("Hunger:", hunger)
```

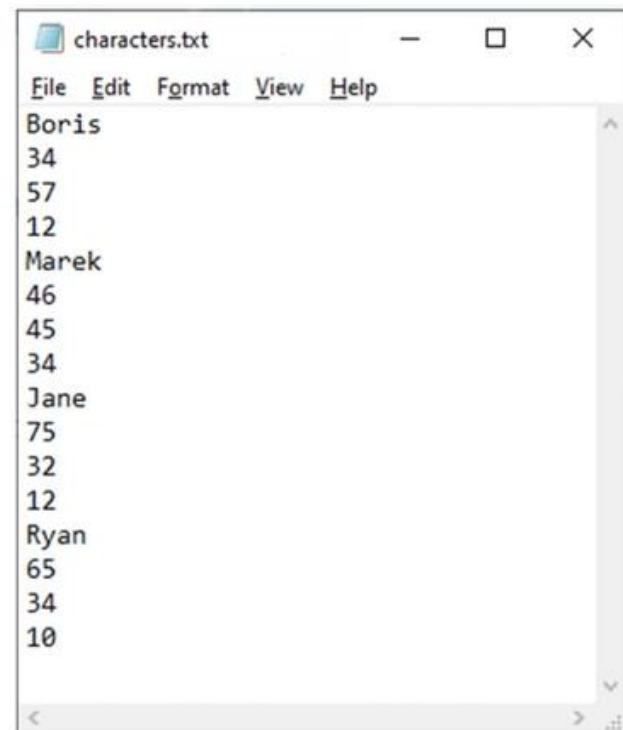
```
if name == "":
```

```
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

Set a Boolean variable to *false* to indicate that we haven't yet reached the end of the file.



8.3 File handling

Reading data from text files

```
#Input the character to find in the file
character = input("Which character do you wish to output? ")

#Step 1 - Open the file for reading/writing
f = open("characters.txt", "r")

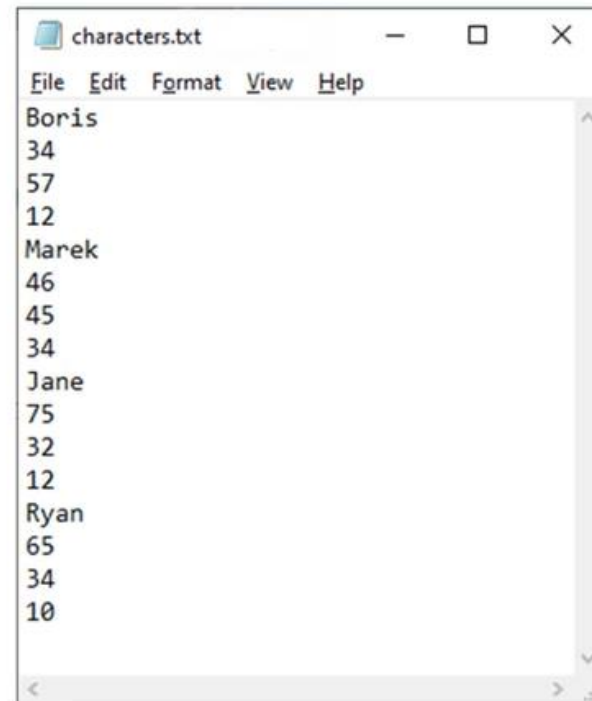
#Step 2 - Use a flag to indicate if the end of file is reached
end_of_file = False

#Step 3 - Iterate through all the data in the file
while not end_of_file:
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()

    #Step 4 - End of file or output data from file
    if character == name:
        print(name)
        print("Health:", health)
        print("Stamina:", stamina)
        print("Hunger:", hunger)
    if name == "":
        end_of_file = True

#Step 5 - Close the file
f.close()
```

Iterate through the file until such time as we reach the end of the file. We will change this `end_of_file` flag to `true` later on if we reach the end of the file.



8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
```

```
    name = f.readline().strip()
```

```
    health = f.readline().strip()
```

```
    stamina = f.readline().strip()
```

```
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
```

```
    print(name)
```

```
    print("Health:",health)
```

```
    print("Stamina:",stamina)
```

```
    print("Hunger:",hunger)
```

```
if name == "":
```

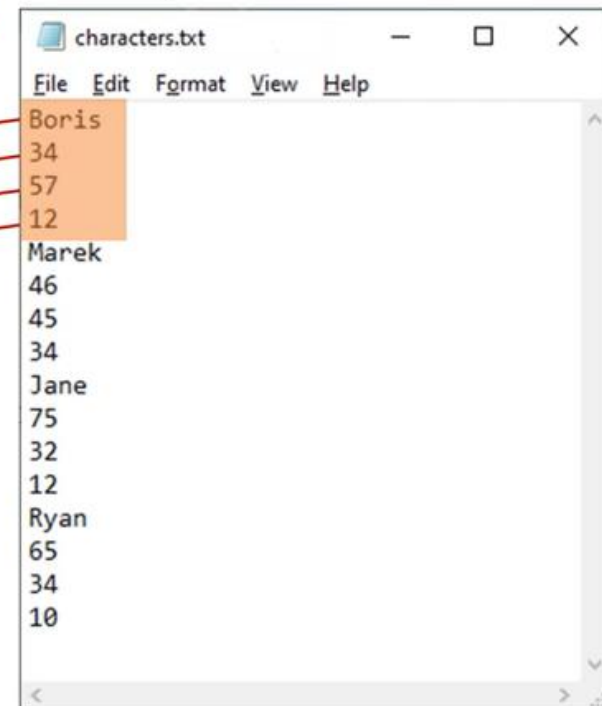
```
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

We now read in four lines from the file, one after the other. Each line read in is placed into the appropriate variable:

- Name
- Health
- Stamina
- Hunger



8.3 File handling

Reading data from text files

```
#Input the character to find in the file
character = input("Which character do you wish to output? ")

#Step 1 - Open the file for reading/writing
f = open("characters.txt", "r")

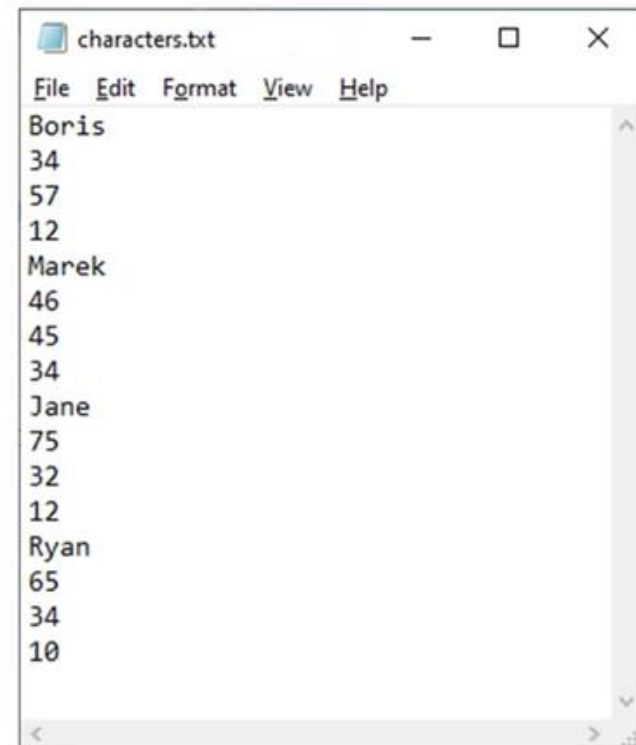
#Step 2 - Use a flag to indicate if the end of file is reached
end_of_file = False

#Step 3 - Iterate through all the data in the file
while not end_of_file:
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()

    #Step 4 - End of file or output data from file
    if character == name:
        print(name)
        print("Health:", health)
        print("Stamina:", stamina)
        print("Hunger:", hunger)
    if name == "":
        end_of_file = True

#Step 5 - Close the file
f.close()
```

Now we check if the character we read in from the file is the one we are looking for. If they are, we output the data.



8.3 File handling

Reading data from text files

#Input the character to find in the file

```
character = input("Which character do you wish to output? ")
```

#Step 1 - Open the file for reading/writing

```
f = open("characters.txt", "r")
```

#Step 2 - Use a flag to indicate if the end of file is reached

```
end_of_file = False
```

#Step 3 - Iterate through all the data in the file

```
while not end_of_file:
```

```
    name = f.readline().strip()
```

```
    health = f.readline().strip()
```

```
    stamina = f.readline().strip()
```

```
    hunger = f.readline().strip()
```

#Step 4 - End of file or output data from file

```
if character == name:
```

```
    print(name)
```

```
    print("Health:", health)
```

```
    print("Stamina:", stamina)
```

```
    print("Hunger:", hunger)
```

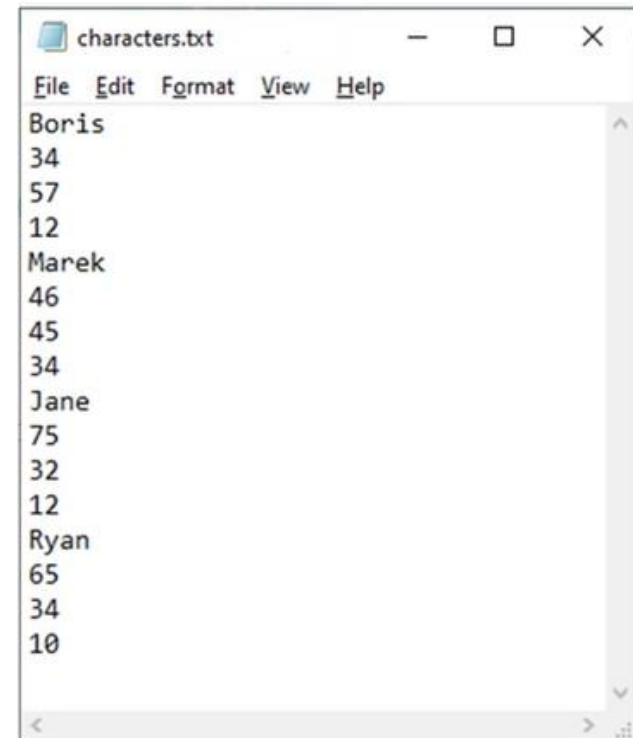
```
if name == "":
```

```
    end_of_file = True
```

#Step 5 - Close the file

```
f.close()
```

If the name we read in contains no data, we know we have reached the end of the file. In this case, we set the `end_of_file` flag to `true`, which will cause the WHILE loop to terminate.



8.3 File handling

Reading data from text files

```
#Input the character to find in the file
character = input("Which character do you wish to output? ")

#Step 1 - Open the file for reading/writing
f = open("characters.txt", "r")

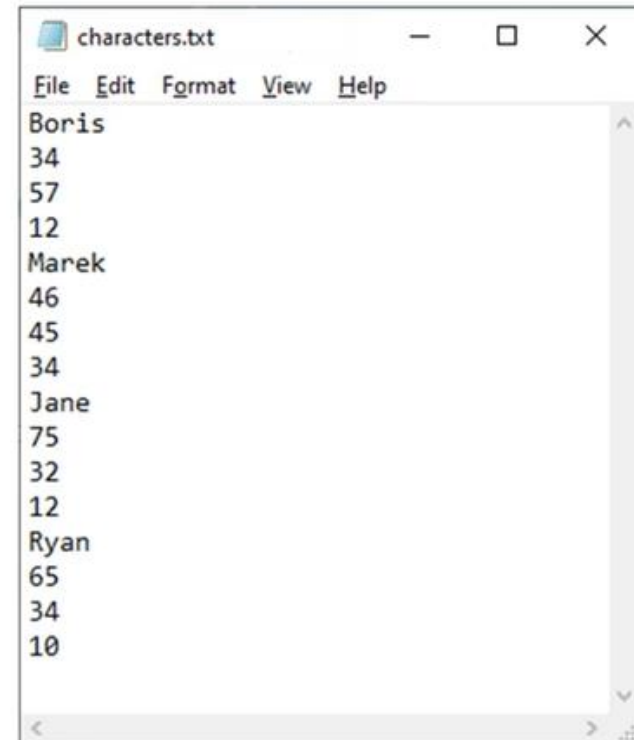
#Step 2 - Use a flag to indicate if the end of file is reached
end_of_file = False

#Step 3 - Iterate through all the data in the file
while not end_of_file:
    name = f.readline().strip()
    health = f.readline().strip()
    stamina = f.readline().strip()
    hunger = f.readline().strip()

    #Step 4 - End of file or output data from file
    if character == name:
        print(name)
        print("Health:", health)
        print("Stamina:", stamina)
        print("Hunger:", hunger)
    if name == "":
        end_of_file = True

#Step 5 - Close the file
f.close()
```

Finally, we need to close the file.



8.3 File handling

Writing data to text files

```
name = input("Enter the name of the character: ")
health = input("Enter the health of the character: ")
stamina = input("Enter the stamina of the character: ")
hunger = input("Enter the hunger of the character: ")
```

#Step 1 - Open the file for characters

```
f = open("characters.txt", "a")
```

#Step 2 - Write the character data to the file

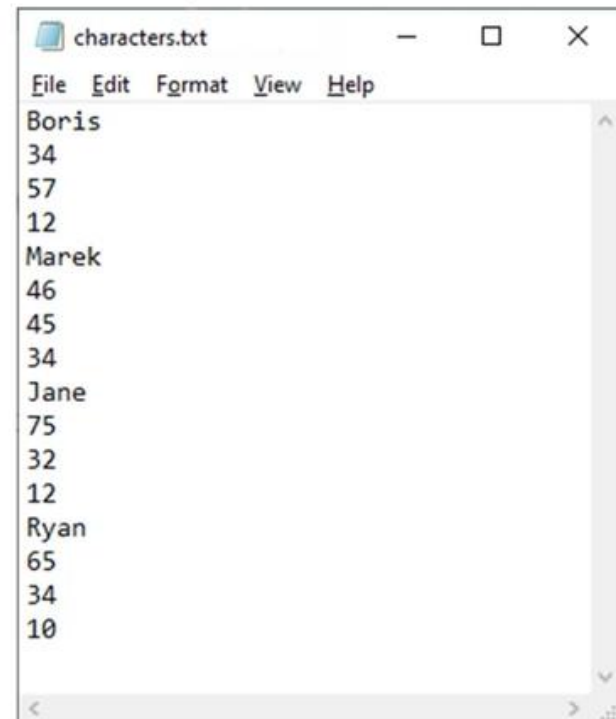
```
f.write(name+"\n")
f.write(health+"\n")
f.write(stamina+"\n")
f.write(hunger+"\n")
```

#Step 3 - Close the file of characters

```
f.close()
```

Get the user to input the details for a character and store those details in variables:

- Name
- Health
- Stamina
- Hunger



8.3 File handling

Writing data to text files

```
name = input("Enter the name of the character: ")
health = input("Enter the health of the character: ")
stamina = input("Enter the stamina of the character: ")
hunger = input("Enter the hunger of the character: ")
```

#Step 1 - Open the file for characters

```
f = open("characters.txt", "a")
```

#Step 2 - Write the character data to the file

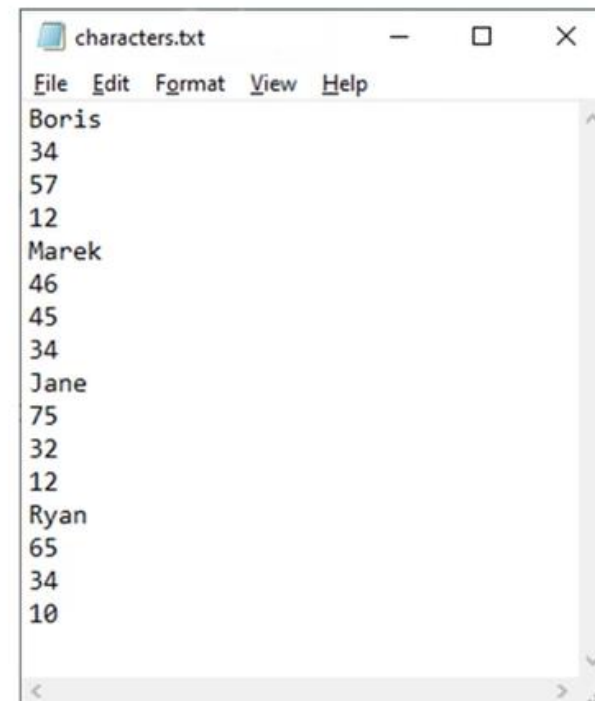
```
f.write(name+"\n")
f.write(health+"\n")
f.write(stamina+"\n")
f.write(hunger+"\n")
```

#Step 3 - Close the file of characters

```
f.close()
```

Next, we open the file for writing. The open command takes two parameters:

- "characters.txt" – the name of the file to open.
- "a" – what we want to do with the file (e.g., append).
- "w" – would create a new file/overwrite contents.



8.3 File handling

Writing data to text files

```
name = input("Enter the name of the character: ")
health = input("Enter the health of the character: ")
stamina = input("Enter the stamina of the character: ")
hunger = input("Enter the hunger of the character: ")
```

#Step 1 - Open the file for characters

```
f = open("characters.txt", "a")
```

#Step 2 - Write the character data to the file

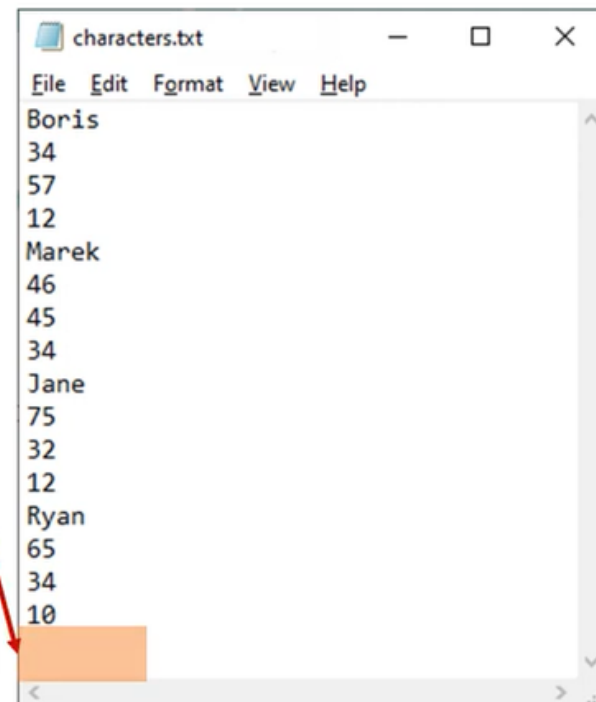
```
f.write(name+"\n")
f.write(health+"\n")
f.write(stamina+"\n")
f.write(hunger+"\n")
```

#Step 3 - Close the file of characters

```
f.close()
```

Next, we write the contents of each variable on a new line in the *characters* text file.

Note how we add the string `"\n"`, which means *start a new line* – as if you had pressed enter.



8.3 File handling

Writing data to text files

```
name = input("Enter the name of the character: ")
health = input("Enter the health of the character: ")
stamina = input("Enter the stamina of the character: ")
hunger = input("Enter the hunger of the character: ")
```

#Step 1 - Open the file for characters

```
f = open("characters.txt", "a")
```

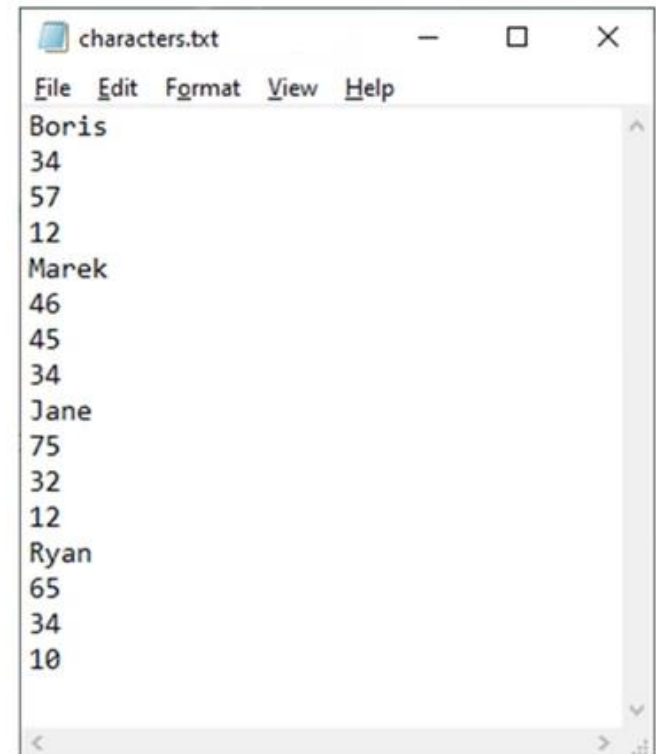
#Step 2 - Write the character data to the file

```
f.write(name+"\n")
f.write(health+"\n")
f.write(stamina+"\n")
f.write(hunger+"\n")
```

#Step 3 - Close the file of characters

```
f.close()
```

Finally, we need to close the file.



8.3 File handling

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Handling files

It is good practice to explicitly open a file ready for writing:

```
OPENFILE <File identifier> FOR <File mode>
```

The file identifier will be the name of the file with data type string. The following file modes are used:

- READ for data to be read from the file
- WRITE for data to be written to the file. A new file will be created and any existing data in the file will be lost.

Data is read from the file (after the file has been opened in READ mode) using the READFILE command as follows:

```
READFILE <File Identifier>, <Variable>
```

Data is written into the file after the file has been opened using the WRITEFILE command as follows:

```
WRITEFILE <File identifier>, <Variable>
```

Files should be closed when they are no longer needed using the CLOSEFILE command as follows:

```
CLOSEFILE <File identifier>
```

8.3 File handling

Cambridge IGCSE pseudocode

Here is an example of how these concepts will look in exam papers following the Cambridge IGCSE pseudocode format, which is listed towards the back of the syllabus.

Handling files

This example uses the operations together to copy a line of text from *FileA.txt* to *FileB.txt*

```
DECLARE LineOfText : STRING
```

```
OPENFILE FileA.txt FOR READ
```

```
OPENFILE FileB.txt FOR WRITE
```

```
READFILE FileA.txt, LineOfText
```

```
WRITEFILE FileB.txt, LineOfText
```

```
CLOSEFILE FileA.txt
```

```
CLOSEFILE FileB.txt
```

8.3 File handling



Reading and writing data to text files

The stages of writing data to a file are:

- Open the file for creating/overwriting or appending to a file.
- Write the data to the file.
- Close the file.

The stages of reading data from a file are:

- Open the file for reading data.
- Set a Boolean variable to *false* to indicate the end of file has not been reached.
- While the `end_of_file` flag is false and the search item has not been found:
 - Read the data from the file.
 - If the data matches what is being searched for, assign the data to variables or output.
 - Check if the end of the file has been reached and if it has, set the Boolean variable to *true*.
- Close the file.

8.3.1 Understand the purpose of **storing data** in a **file** to be **used by a program**

File handling is a crucial aspect of programming, allowing programs to store and retrieve data for various purposes.

Storing data in a file provides a mechanism for data persistence, sharing, backup, configuration, handling large datasets, logging, input/output operations, serialization, and interaction with databases. It enhances the flexibility and functionality of programs, contributing to efficient data management and utilization.

In next slide:

- We have a list **data_to_store** containing various pieces of information.
- We use the with **open(...)** syntax **to open the file** (**user_data.txt**) in **write mode** ("**w**"), and we write each piece of data to a new line in the file.
- We then open the same file in **read mode** ("**r**") and read each line to retrieve the data. The retrieved data is stored in the list **retrieved_data**.
- Finally, we display the retrieved data.

8.3.1 Understand the purpose of **storing data** in a **file** to be **used by a program**

 Storing Data in a File.py - C:/Program Files/Python312/Storing Data in a File.py (3.12.0)

— □

File Edit Format Run Options Window Help

```
# Storing Data in a File
```

```
# Data to be stored
```

```
data_to_store = ["Noureddine", "Tadjerout", 35, "Noureddine.Tad@email.com"]
```

```
# File path
```

```
file_path = "user_data.txt"
```

```
# Open the file in write mode
```

```
with open(file_path, "w") as file:
```

```
    # Write each piece of data to a new line
```

```
    for data in data_to_store:
```

```
        file.write(str(data) + "\n")
```

```
print("Data has been stored in the file.")
```

```
# Reading Data from the File
```

```
# Empty list to store retrieved data
```

```
retrieved_data = []
```

```
# Open the file in read mode
```

```
with open(file_path, "r") as file:
```

```
    # Read each line from the file
```

```
    for line in file:
```

```
        # Remove newline characters and append to the list
```

```
        retrieved_data.append(line.strip())
```

```
# Display the retrieved data
```

```
print("Data retrieved from the file:")
```

```
print(retrieved_data)
```

```
= RESTART: C:/Program Files/Python312/Storing Data in a File.py
```

```
Data has been stored in the file.
```

```
Data retrieved from the file:
```

```
['Noureddine', 'Tadjerout', '35', 'Noureddine.Tad@email.com']
```

8.3.1 Open, close and use a file for reading and writing

Including: read and write single items of data and read and write a line of text

Openand Close File.py - C:/Program Files/Python312/Openand Close File.py (3.12.0)

File Edit Format Run Options Window Help

```
# Open a file for writing
with open('example.txt', 'w') as file:
    # Write a single item of data
    file.write('Hello, Nouredine!\n')

    # Write multiple names of data (as a string)
    data_list = ['Nuriya', 'Artur', 'Danial', 'Inaaya', 'Harsh', ' Ivan', 'Danial', 'Beiza', 'Nicholas', 'Darius', 'Samina', 'Emir' ]
    file.write(','.join(data_list) + '\n')

# Open the same file for reading
with open('example.txt', 'r') as file:
    # Read a single item of data
    single_item = file.read(5) # Read the first 5 characters
    print("Read single item:", single_item)

    # Read a line of text
    line_of_text = file.readline().strip() # Remove the newline character
    print("Read line of text:", line_of_text)

    # Read multiple names of data (splitting by comma)
    name_list = file.readline().strip().split(',')
    print("Read multiple names:", name_list)
```

= RESTART: C:/Program Files/Python312/Openand Close File.py

Read single item: Hello

Read line of text: , Nouredine!

Read multiple names: ['Nuriya', 'Artur', 'Danial', 'Inaaya', 'Harsh', ' Ivan', 'Danial', 'Beiza', 'Nicholas', 'Darius', 'Samina', 'Emir']

8.3 File handling

Past Paper 2 Exams Questions and Answers

Question 1

Circle file-handling operations.

calculate

close

count

create

input

open

output

print

read

sort

search

test

total

write